# TrustGraph ??? The Complete Guide v2.2

Table of Contents

## TrustGraph — The Complete Guide

### From Zero to Production: Full Technical Depth, Real Tested Code, Business Case, and the Complete Long-Term Vision

**Version 2.2 — Clarification Update**

54 chapters across 5 parts. Refines Chapter 33's Feedback & Adjudication Loop (introduced in v2.1): feedback is now classified before adjudication (different report types route to different engineering queues — a "too slow" report never touches verification machinery); adjudication produces a confidence score, not just a verdict; the system explicitly routes work rather than "learning" automatically; and no feedback ever directly updates the knowledge base — corrections flow through a Candidate → Verified → Approved pipeline, matching the source-authority governance already established elsewhere in this book. Adds a Trust Recovery research direction, measuring how reliably the system corrects itself over time rather than only its state at one snapshot. This remains a design specification only — v0.1 implementation scope is unchanged.

Every claim in this book is explicit, throughout, about what was actually run and measured versus what is designed but not yet built.

---

## Preface

### Who this book is for

You do not need prior experience with RAG, retrieval, or LLM verification to read this book. If you can write basic Python and have used a REST API before, you can build TrustGraph-Lite — a small, fully working version of the platform — by the end of Part B. Part C then shows exactly how that toy version maps onto the production-grade, 10-engineer architecture described in the rest of this book.

Every piece of code in this book has been run and tested before being written down. Nothing here is aspirational pseudocode. If you type it in (or copy it) and follow the setup steps, it will work — including the parts designed to *fail* on purpose, so you can see verification catching a bad answer in real time.

## What makes TrustGraph different

Most "RAG tutorials" stop at: retrieve some text, stuff it into a prompt, generate an answer. That gets you a chatbot. It does not get you a system anyone can trust with a real business decision, because nothing in that pipeline checks whether the generated answer is actually true.

TrustGraph adds the missing piece: after an answer is generated, it is broken into individual factual claims, and each claim is checked — using plain code (SQL/Pandas) for anything numeric, and a language model only for the claims that genuinely require judgment about meaning. The result is a trust score built from real evidence, not a single confidence number the model made up.

## How the book is organized

- **Part A — Foundations.** What RAG is, why it hallucinates, what "verification" and "trust" actually mean in an AI system, and the tools you'll use. Written for a complete fresher.
- **Part B — Build It Yourself.** A step-by-step tutorial that builds TrustGraph-Lite: a small, real, runnable version of every core service, wired together into one API you can call with `curl`. By the end of this part, you will have run the system and watched it catch a hallucinated number with your own eyes.
- **Part C — The Production Architecture.** The full frozen architecture, data contracts, API specification, and engineering standards used by the real 10-engineer team building TrustGraph at scale — and an explicit mapping showing how each piece of your Part B toy code corresponds to its production-grade counterpart.
- **Part D — Vision, Process, and Execution.** The long-term roadmap, sprint plan, and release criteria that govern how the real team decides what to build next and when something counts as "done."
- **Part E — Reference.** Glossary, FAQ, troubleshooting, and further reading.

## A note on honesty

This book is deliberately upfront about what is a teaching simplification and what is production-grade. Part B's retriever uses TF-IDF and a small in-memory document list instead of a real embedding model and vector database; its "LLM verifier" uses word overlap instead of an actual API call. Every one of these simplifications is labeled clearly at the point it appears, with a note on what to swap in for production. The goal is that you understand *why* each production component exists, not that you mistake the toy version for the real thing.

---

# Part A: Foundations

## Chapter 1: What Is RAG, and Why Does It Hallucinate?

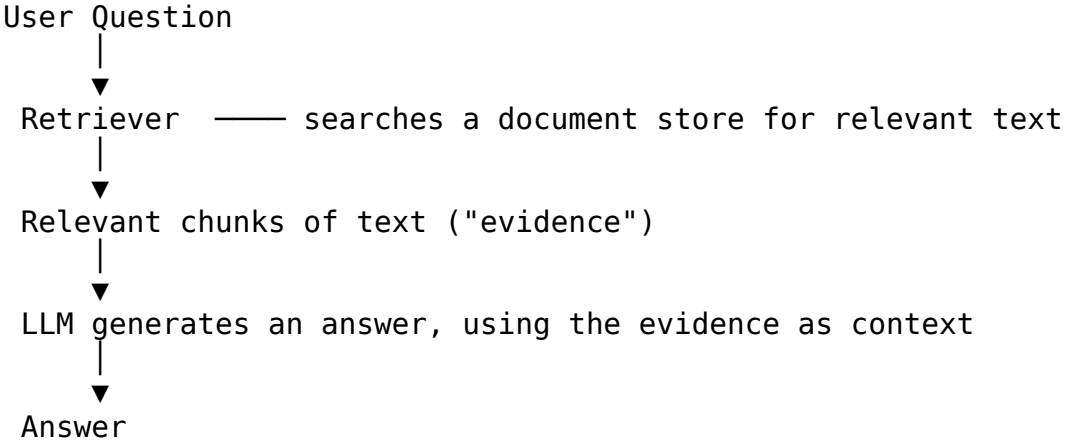
### The problem large language models have

A large language model (LLM) like Claude or GPT is trained to predict plausible next words given everything it has seen before. It has no built-in mechanism to check whether a specific fact it states is true — it generates the most statistically likely continuation of text, which is usually correct for well-known facts, and often wrong or fabricated for anything specific, recent, or obscure. This fabrication is called **hallucination**, and it is not a bug that better training fixes — it is a structural property of how these models work.

Ask an LLM "what was Acme Corp's Q3 2025 revenue?" and, if it doesn't know, it will often produce a confident-sounding number anyway, because a confident-sounding number is a more statistically plausible sentence completion than "I don't know."

### Retrieval-Augmented Generation (RAG)

RAG tries to fix this by giving the model real source material to work from before it answers:

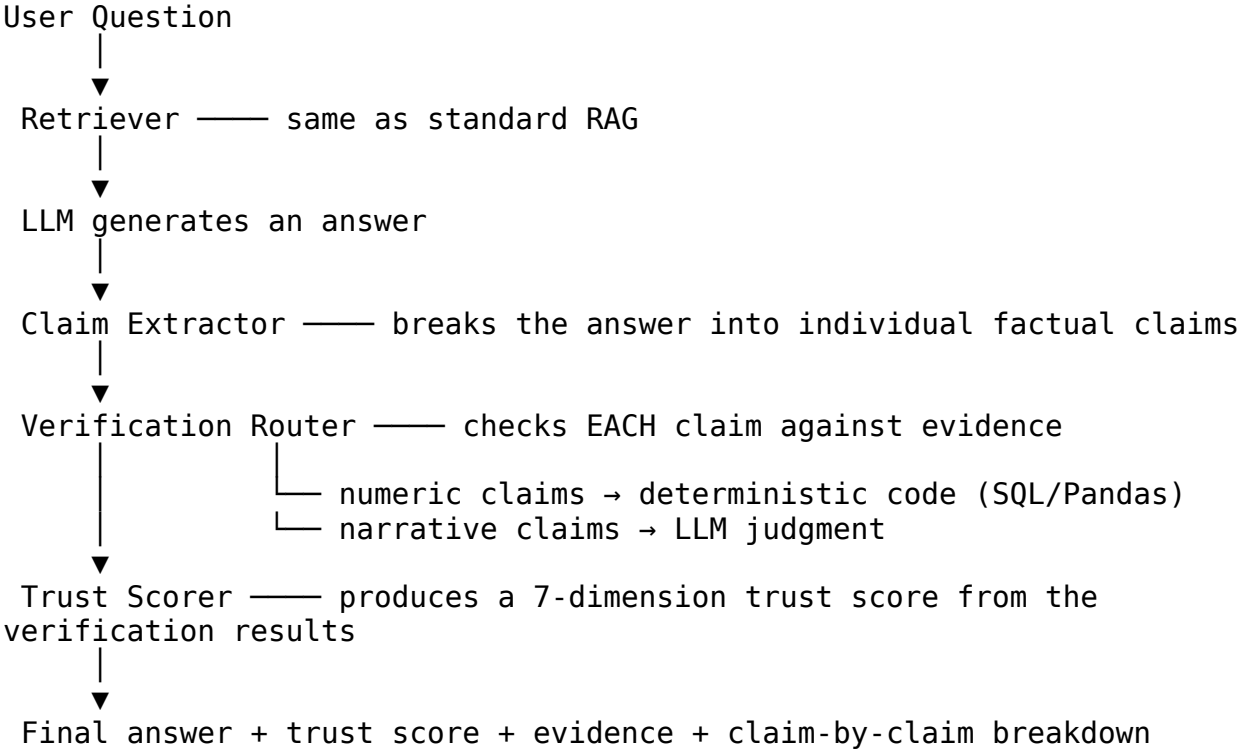



This helps a lot — the model is now grounding its answer in real text instead of pure memory. But it does not fully solve hallucination, for three reasons:

1. **The retriever might miss the right evidence.** If the real answer isn't retrieved, the model may still guess.
2. **The model might misread the evidence.** LLMs sometimes paraphrase numbers incorrectly, combine facts from different documents incorrectly, or draw a conclusion the evidence doesn't actually support.
3. **Nothing checks the output afterward.** Standard RAG has no step that goes back and confirms the final answer's claims are actually supported by what was retrieved. The evidence was an input to generation, not a check on the output.

## What TrustGraph adds

TrustGraph inserts a verification stage *after* generation:




This is the whole idea of the book in one diagram. Standard RAG generates and hopes. TrustGraph generates, then checks its own work, claim by claim, before it's shown to a user.

## A concrete example

Suppose the retrieved evidence says: *"Revenue grew 12% to $4.2 million in 2025."*

A standard RAG system might generate: *"Revenue grew 45% to $9 million in 2025"* — a plausible-sounding hallucination that happens to be entirely wrong, and nothing in a standard pipeline would catch it before the user sees it.

TrustGraph's claim extractor pulls out two numeric claims (`45%` and `$9,000,000`), the verification router sends both to the deterministic SQL verifier, and the verifier compares them against the ground-truth numbers from the evidence. Both are flagged `CONTRADICTED`, the trust score drops sharply, and the failure is visible to the user — instead of silently shipped as fact.

You will build and run exactly this scenario yourself in Part B, Chapter 10.

## Why this matters more as stakes rise

For a casual chatbot, an occasional wrong fact is a minor annoyance. For a system used in finance, legal, healthcare, or any enterprise decision-making, an unverified hallucinated number can cause real damage — a wrong revenue figure in a report, a misstated compliance

requirement, an incorrect medical dosage reference. This is why TrustGraph exists: not as an incremental RAG improvement, but as an answer to the question "how do I know I can trust this system's output enough to act on it?"

---

## Chapter 2: How Large Language Models Actually Work

Chapter 1 explained *why* LLMs hallucinate. This chapter explains, at a level appropriate for a complete fresher, *how* they work at all — enough to make the rest of the book's design decisions feel motivated rather than arbitrary.

### Tokens, not words

A language model doesn't read text as words — it reads it as **tokens**, which are often sub-word pieces. "Unhappiness" might become three tokens: "un", "happi", "ness". This matters for two very practical reasons that show up throughout this book: cost is measured in tokens (Chapter 17's `PipelineEvent.token_usage`), and a model's context window (how much text it can consider at once) is a token limit, not a word-count limit — which is exactly why Chapter 13's "Context Intelligence" capability includes token budgeting as a real, necessary function.

### The next-token prediction game

At its core, an LLM is trained to do one thing: given a sequence of tokens, predict the most likely next token. "The capital of France is ___" → the model has seen this pattern enough times that "Paris" is overwhelmingly the most probable completion. Generation is just this process repeated — predict one token, add it to the sequence, predict the next one, and so on.

This single fact explains hallucination completely (Chapter 1 covered the consequence; this is the mechanism): the model isn't consulting a fact database when it predicts "Paris" — it's producing the statistically likely continuation. For well-known facts, the statistically likely continuation and the true fact are almost always the same thing, which is why LLMs seem to "know" so much. For obscure, recent, or highly specific facts (like your company's exact Q3 revenue), the statistically likely continuation is just whatever sounds plausible — and a plausible-sounding wrong number is exactly what a next-token predictor is optimized to produce when it doesn't actually know.

### Attention: how a model decides what's relevant

Modern LLMs use an architecture called the Transformer, whose key mechanism is called **attention**: for every token being generated, the model computes how much "attention" to pay to every other token in its input, and weighs their influence accordingly. This is *why* retrieval-augmented generation (Chapter 1) works at all — feeding relevant evidence into the model's input gives the attention mechanism something real to weigh, rather than forcing generation purely from the model's trained-in statistical patterns.

### Why this book treats the LLM as an "explanation layer," not a source of truth

Given the above, a design principle from Chapter 3 should now feel motivated rather than just asserted: an LLM is fundamentally a very sophisticated pattern-completion engine, not a database or a calculator. It is excellent at language — summarizing, rephrasing, explaining, judging whether a sentence's meaning is supported by other text — and structurally unsuited to being the sole authority on a specific fact, a specific number, or a specific legal clause, because none of those are what its training process was actually optimizing for. That's the precise, mechanical reason Chapter 8's Verification Router sends numeric claims to deterministic code and reserves the LLM verifier for claims that are genuinely about meaning rather than fact lookup.

### A note on model size and capability

Larger, more heavily trained models are generally better at this pattern-completion game — they've seen more data and can model more subtle statistical relationships, which is why they write more fluently and reason more capably. But more capable pattern completion is still pattern completion. A larger model hallucinates less often on obscure facts than a smaller one, but "less often" is not "never," and no amount of scale changes the fundamental mechanism described in this chapter. This is why TrustGraph's core bet — verify after generation, rather than trust a bigger model to generate correctly in the first place — doesn't become less relevant as models improve. It becomes a permanent architectural position, not a stopgap for today's model quality.

---

## Chapter 3: Embeddings, Vector Search, and Hybrid Retrieval

### The retrieval problem

Given a user's question, how do you find the right documents out of possibly millions? The naive approach — matching exact keywords — misses paraphrases ("revenue" vs "sales" vs "top line"). The modern approach uses **embeddings**: numerical representations of meaning.

### What is an embedding?

An embedding model converts a piece of text into a list of numbers (a vector) such that texts with similar meaning produce vectors that are close together in that numerical space, and texts with different meaning produce vectors that are far apart. For example, "the company's revenue increased" and "sales went up" would produce similar vectors even though they share no words, because an embedding model has learned that these phrases mean similar things.

In production, you'd use a real embedding model (OpenAI's `text-embedding-3`, Cohere's embeddings, or similar) via an API call. This book's teaching code uses TF-IDF (term frequency statistics) combined with a dimensionality-reduction technique called SVD as a simplified stand-in — it captures a rough notion of topical similarity without needing an external API call, which keeps Part B's code runnable offline. It is meaningfully weaker than a real embedding

model at capturing paraphrase and synonym relationships, and Chapter 12 explains exactly how to swap in a real one.

## Dense retrieval

Once every document is embedded as a vector, and the user's query is also embedded, you can find the most relevant documents by computing the **cosine similarity** (a measure of how closely two vectors point in the same direction) between the query vector and every document vector, and returning the closest matches. This is called **dense retrieval**, because the vectors are dense (mostly non-zero) numerical arrays.

Dense retrieval is excellent at catching semantic similarity — synonyms, paraphrases, related concepts. It is comparatively weak at exact matches: specific product codes, names, IDs, or precise numbers can get "blurred" in the embedding space, because the model is optimizing for meaning, not exact string matching.

## Sparse retrieval (BM25)

**Sparse retrieval** works differently: it's a statistical scoring method (the most common algorithm is called BM25) that scores documents based on how often the query's exact words appear in them, adjusted for how common or rare those words are across the whole document collection. A document containing the exact phrase "Q3 2025 revenue" will score very highly against a query containing those same words, regardless of whether the surrounding context is semantically similar.

Sparse retrieval is excellent at exact-match precision — the thing dense retrieval is weakest at.

## Why combine both: Hybrid Retrieval

Because dense and sparse retrieval have complementary strengths and weaknesses, production RAG systems combine them. This book's code does this using **Reciprocal Rank Fusion (RRF)**, a simple and effective combination method:

1. Rank all documents using the sparse (BM25) method — get a ranked list.
2. Rank all documents using the dense (embedding) method — get a second ranked list.
3. For each document, compute a combined score: `1 / (k + rank_in_list_1) + 1 / (k + rank_in_list_2)`, where `k` is a small constant (60 is a common default) that prevents documents ranked #1 from dominating too heavily.
4. Sort by this combined score.

RRF works well because it combines *rankings* rather than raw scores — BM25 scores and cosine similarity scores are on completely different numerical scales, so averaging them directly would be meaningless. Averaging their *ranks* sidesteps that problem entirely.

## Reranking

After hybrid retrieval narrows a large document collection down to, say, the top 50-100 candidates, production systems often add a second, more expensive but more accurate step: a **cross-encoder reranker**. Unlike the retrieval step (which compares a query vector to each

document vector independently), a cross-encoder looks at the query and each candidate document *together* and produces a more precise relevance score. This is too slow to run against millions of documents, but very effective for re-scoring a small shortlist. Part C describes exactly where this fits into the production architecture; the teaching code in Part B skips this step for simplicity, since the toy document collection is small enough that it isn't needed to get sensible results.

### What you'll build in Part B

In Chapter 6, you will implement exactly the hybrid retrieval approach described in this chapter — BM25 for sparse, TF-IDF+SVD for dense, combined via RRF — as a self-contained, runnable Python class. You'll see it correctly retrieve the right evidence for a revenue question even when the query uses different words than the source document.

---

## Chapter 4: Vector Databases — A Practical Comparison

Chapter 4 mentioned that production TrustGraph uses PostgreSQL with the pgvector extension. This chapter explains what a vector database actually does and why several real options exist, each with different trade-offs.

### What a vector database has to do well

Given millions of stored embedding vectors (Chapter 2) and a new query vector, find the nearest ones fast — much faster than comparing the query against every single stored vector one by one (which is what Chapter 6's teaching retriever does, and why it's fine for a 5-document corpus and would be far too slow for a 5-million-document one). This requires an approximate nearest-neighbor (ANN) index — a data structure that trades a small amount of accuracy for a very large amount of speed.

### The main indexing approaches

**HNSW (Hierarchical Navigable Small World)** builds a multi-layer graph where each vector is connected to its approximate neighbors, and search descends through the layers, narrowing in on close matches quickly. Most production vector databases (including pgvector) support this, because it typically gives the best accuracy-to-speed trade-off for a moderate memory cost.

**IVF (Inverted File Index)** clusters vectors into buckets during indexing, and search only scans the buckets nearest the query — much less memory than HNSW, generally somewhat lower accuracy at the same speed.

### Comparing the real options

| Option | What it is | When it fits |
|---|---|---|
| **pgvector** | A PostgreSQL extension adding vector columns and similarity search to an | You already run Postgres, want vectors alongside relational data in the same |

| Option | What it is | When it fits |
|---|---|---|
| | ordinary relational database | transactional store, and don't yet have evidence you need a dedicated system — this is what TrustGraph's production architecture uses by default (Chapter 18, Section 9) |
| **Pinecone** | A managed, dedicated vector database service | You want a fully managed service with minimal operational overhead and are comfortable with an external dependency and its cost model |
| **Weaviate** | An open-source vector database with built-in hybrid search and schema support | You want hybrid dense+sparse search (Chapter 2) natively in the database layer rather than combined at the application layer, as Chapter 6's `HybridRetriever` does |
| **Qdrant** | An open-source, Rust-based vector database focused on performance and filtering | You need fast metadata filtering alongside vector search (e.g., "find similar documents, but only ones this user is authorized to see") |

## Why TrustGraph defaults to pgvector

Per the TRD's deployment architecture (Chapter 18) and the Architecture Freeze Policy (Chapter 22), the guiding rule is: don't add infrastructure complexity speculatively. pgvector means one database to operate, one connection pool, one backup strategy, and vector search that can be joined directly against relational metadata in the same query — genuinely useful for TrustGraph specifically, since Evidence objects (Chapter 17) carry both a vector-searchable text field and structured metadata (authority score, freshness timestamp) that benefit from being queryable together. A dedicated vector database becomes justified only if a real, measured bottleneck shows pgvector's performance is insufficient at the corpus size actually in production — not because a dedicated system sounds more sophisticated on a slide.

## How this connects back to Chapter 6's teaching code

The `HybridRetriever` class combines a TF-IDF+SVD "dense" stand-in with BM25 sparse search, computing similarity by brute-force comparison against every document in memory. Swapping in pgvector means: (1) writing embeddings to a `vector` column via a real embedding model instead of TF-IDF+SVD, (2) letting Postgres's HNSW index handle the nearest-neighbor

search instead of the manual cosine similarity loop, and (3) keeping the BM25 sparse side and the Reciprocal Rank Fusion combination logic exactly as they are — those two pieces don't change based on which vector store backs the dense side. This is precisely why the `Retriever` interface (Chapter 17) is defined as an abstract contract rather than tied to any specific implementation: swapping pgvector in for the teaching stand-in shouldn't require touching any other service.

---

# Chapter 5: What "Verification" and "Trust" Actually Mean

## Confidence is not the same as correctness

When an LLM says "I'm 95% confident," that number is not measuring the same thing a scientist means by 95% confidence. It's typically the model's own estimate of how plausible its answer sounds, generated the same way it generates everything else — as a statistically likely piece of text. This number can be wrong in either direction: models can be very confident about hallucinated facts, and uncertain about facts they actually got right.

Real trust needs to come from checking claims against something external to the model — actual source documents, actual database values, actual business logic — not from asking the model how sure it feels.

## Claim-level verification, not answer-level

Most systems that attempt "fact-checking" treat the whole generated answer as one unit: either the answer passes a check, or it doesn't. This is too coarse. A single paragraph might contain five factual claims, four of which are perfectly correct and one of which is wrong. Answer-level checking either (a) rejects a mostly-correct answer entirely, throwing away good information, or (b) passes it anyway because most of it checked out, silently letting the one wrong claim through.

TrustGraph instead decomposes an answer into **atomic claims** — individually verifiable statements — and checks each one separately. A claim like *"revenue grew 12% to $4.2 million in 2025"* actually contains two separate numeric claims (the percentage and the dollar amount), each independently checkable.

## Why numbers and language need different verification methods

This is the single most important design idea in the whole book: **numeric claims should be verified with code, and narrative claims should be verified with judgment.**

If a claim says "revenue grew 12%," you don't need to ask a language model whether that's true — you can look up the actual revenue figures and compute the percentage change with arithmetic. This is fast, cheap, deterministic, and 100% reliable (as reliable as your source data, at least) — asking an LLM to verify a number is strictly worse than just computing it, because you're introducing a second chance for the LLM to make an error.

If a claim says "customers generally reported higher satisfaction," there's no formula to check that — it requires reading the actual evidence text and judging whether it supports the statement. This does require a language model (or a human), because it's a question about *meaning*, not arithmetic.

Routing each claim type to the appropriate verification method — deterministic code for numbers, LLM judgment for narrative claims — is what the book's **Verification Router** does. You'll build this in Chapter 8.

## Trust scores should be explainable, not a single number

A system that reports "Trust: 87%" and nothing else is not meaningfully more trustworthy than one that reports nothing — the number is just as opaque as the LLM's own confidence claim.

TrustGraph instead reports a **breakdown across seven dimensions**:

| Dimension | What it measures |
|---|---|
| Retrieval quality | Did the retriever likely find the relevant evidence? |
| Evidence coverage | What fraction of claims had any relevant evidence available at all? |
| Claim verification rate | What fraction of claims were actually confirmed correct? |
| Freshness | How current are the sources the answer relies on? |
| Conflict score | Do the sources agree with each other and the answer, or contradict? |
| Source authority | How trustworthy are the specific sources used? |
| Deterministic verification rate | What fraction of claims were checked by code rather than by an LLM's judgment? |

A reviewer looking at this breakdown can immediately see, for example, "this answer has a high claim verification rate but low deterministic verification rate" — meaning most of the trust here is riding on an LLM's judgment calls, not hard numeric checks, which is a meaningfully different risk profile than the reverse.

## Failure should be categorized, not just flagged

When a claim fails verification, TrustGraph records *why*, using a fixed set of failure codes (`NUMERIC_MISMATCH`, `NO_SUPPORT`, `RETRIEVAL_MISS`, `CONFLICTING_EVIDENCE`, and others — the full list is in Part C). This turns failures into data you can analyze in aggregate: "80% of our verification failures last month were `RETRIEVAL_MISS`" tells you to invest in the retriever, not the verifier. A system that just says "verification failed" gives you nothing to act on.

### What "trust" means, precisely, in this book

To be precise about what TrustGraph can and cannot claim: it cannot prove a statement is objectively, universally true. It can only verify that a statement is **consistent with the evidence available to it** — deterministically for numbers, and via LLM judgment for language. If the underlying data is wrong, or the evidence is incomplete, TrustGraph's verification will reflect that limitation faithfully (ideally reporting low evidence coverage or low retrieval quality) rather than hiding it behind a falsely confident score.

---

## Chapter 6: The Mathematics of Retrieval and Trust

Every algorithm used in this book has a precise mathematical definition. This chapter works through each one by hand, on small numbers, so the code in Chapters 6-9 is never a black box.

### Cosine similarity (dense retrieval)

Given two vectors **a** and **b**, cosine similarity measures the angle between them, ignoring their length:

```
cosine_similarity(a, b) = (a · b) / (||a|| × ||b||)
```

Worked example, with two 2-dimensional toy vectors:

```
a = [3, 4]      ||a|| = sqrt(3² + 4²) = sqrt(25) = 5
b = [6, 8]      ||b|| = sqrt(6² + 8²) = sqrt(100) = 10

a · b = (3×6) + (4×8) = 18 + 32 = 50

cosine_similarity = 50 / (5 × 10) = 50 / 50 = 1.0
```

A result of 1.0 means the vectors point in exactly the same direction — b is just a scaled-up version of a. This is why cosine similarity captures *semantic* similarity: it doesn't care about magnitude, only direction, which corresponds to "topic" in embedding space.

### TF-IDF (the teaching stand-in for dense embeddings)

Term Frequency-Inverse Document Frequency scores how important a word is to a specific document within a larger collection:

```
TF-IDF(word, doc) = TF(word, doc) × IDF(word)

TF(word, doc) = (times word appears in doc) / (total words in doc)

IDF(word) = log( (total documents) / (documents containing word) )
```

Worked example: the word "revenue" appears once in a 10-word document, out of a 5-document collection where "revenue" appears in 2 documents:

```
TF = 1 / 10 = 0.1
IDF = log(5 / 2) = log(2.5) ≈ 0.916

TF-IDF = 0.1 × 0.916 ≈ 0.0916
```

A word that appears in every document (like "the") gets `IDF = log(5/5) = log(1) = 0`, contributing nothing to the score — exactly the intended behavior, since a word present everywhere carries no distinguishing information.

## BM25 (sparse retrieval)

BM25 is TF-IDF's more sophisticated cousin, adding two corrections: it caps out how much repeating a word helps (diminishing returns), and it normalizes for document length (so a keyword match in a short document counts for more than the same match diluted across a long one):

```
BM25(word, doc) = IDF(word) × [ TF(word,doc) × (k1+1) ] /
[ TF(word,doc) + k1 × (1-b+b × |doc|/avgdl) ]
```

where `k1` (commonly 1.2-2.0) controls the diminishing-returns curve, and `b` (commonly 0.75) controls how strongly document length is penalized. You don't need to hand-derive this one — the `rank_bm25` library in Chapter 6 implements it correctly — but knowing the shape explains *why* BM25 outperforms plain TF-IDF for keyword search: it's TF-IDF corrected for two real-world distortions.

## Reciprocal Rank Fusion (combining dense + sparse)

```
RRF_score(doc) = Σ  1 / (k + rank_in_list)
```

summed across every ranked list the document appears in, where `k` (commonly 60) softens the impact of any single list's #1 position.

Worked example — a document ranked #1 by BM25 and #3 by the dense retriever, with k=60:

```
RRF_score = 1/(60+1) + 1/(60+3) = 1/61 + 1/63 ≈ 0.01639 + 0.01587 ≈
0.03226
```

Compare a document ranked #2 in both lists:

```
RRF_score = 1/(60+2) + 1/(60+2) = 2 × (1/62) ≈ 0.03226
```

Interesting — these come out almost identical, illustrating RRF's key property: consistent middling relevance across both methods can score comparably to being #1 in one and mediocre in the other, which is usually the desired behavior for a system trying to reward broad agreement over a single lucky match.

## Brier score (trust calibration)

```
Brier score = (1/n) × Σ (predicted_probability − actual_outcome)²
```

where `actual_outcome` is 1 if the claim was actually correct, 0 if not. Lower is better; 0 is a perfect calibration.

Worked example — three predictions:

```
Predicted 0.9, actually correct (1):    (0.9 − 1)² = 0.01
Predicted 0.9, actually correct (1):    (0.9 − 1)² = 0.01
Predicted 0.9, actually WRONG (0):      (0.9 − 0)² = 0.81

Brier score = (0.01 + 0.01 + 0.81) / 3 = 0.83 / 3 ≈ 0.277
```

This shows why calibration matters, not just accuracy: a system that's "90% confident" needs to actually be right about 90% of the time across many predictions, not just sound confident every time. One badly wrong high-confidence prediction (the third case above) damages the Brier score far more than a cautious, correctly-hedged one would.

### Precision, Recall, and F1 (verification quality)

```
Precision = True Positives / (True Positives + False Positives)
Recall    = True Positives / (True Positives + False Negatives)
F1        = 2 × (Precision × Recall) / (Precision + Recall)
```

Applied to Chapter 11's actual first benchmark run — 7 true positives (real errors correctly caught), 5 false positives (correct claims wrongly flagged), 0 false negatives (no real error missed):

```
Precision = 7 / (7 + 5) = 7/12 ≈ 0.583
Recall    = 7 / (7 + 0) = 7/7 = 1.000
F1        = 2 × (0.583 × 1.000) / (0.583 + 1.000) = 1.166 / 1.583 ≈
0.737
```

This is exactly the 0.737 reported in Chapter 11 — worked by hand here so the number is never just "what the code printed."

### Why recall mattered more than precision in that result

F1 treats precision and recall symmetrically, but the actual engineering priority didn't: a recall of 1.000 (catching every real hallucination) with mediocre precision (some false alarms) is a far safer failure mode for a v0.1 system than the reverse — a human reviewing a false alarm loses a little time; a missed hallucination reaches a user unchecked. This is why Chapter 11 treated the 0.583 precision as an acceptable, expected weakness to fix later, rather than a blocking failure.

---

## Chapter 7: The Stack — What You Need to Know Before You Start

You do not need to be an expert in any of these before starting Part B — this chapter gives you just enough context to follow the code. Where something is unfamiliar, a link or note explains where to learn more.

## Python 3.12

TrustGraph's backend services are all written in Python, chosen for its strong ecosystem in both data/ML work (pandas, scikit-learn) and web APIs (FastAPI). If you know basic Python — functions, classes, loops, dictionaries — you have everything you need. The teaching code introduces `dataclasses` (a way to define simple structured objects with less boilerplate than a full class) and type hints (`def foo(x: int) -> str:`), both of which are explained inline the first time they appear.

## Pydantic

Pydantic is a library for defining data structures with automatic validation. Instead of a plain Python dictionary (where nothing stops you from accidentally putting a string where a number should be), a Pydantic model declares exactly what fields exist and what type each one must be — and it raises a clear error immediately if something doesn't match, rather than causing a confusing bug three steps later. Every shared data contract in this book — Claim, Evidence, TrustScore, and so on — is defined as a Pydantic model.

```
from pydantic import BaseModel

class Claim(BaseModel):
    text: str
    confidence: float

# This works:
c = Claim(text="Revenue grew 12%", confidence=0.95)

# This raises a clear validation error immediately, instead of failing
silently later:
c = Claim(text="Revenue grew 12%", confidence="high")
```

## FastAPI

FastAPI is a Python web framework for building REST APIs — the kind of API you interact with using tools like `curl` or Postman, where a client sends an HTTP request and gets a JSON response back. It's built on top of Pydantic, so the same data models you use internally can define exactly what shape of JSON your API expects and returns, with automatic validation and automatic interactive documentation. You'll write your first FastAPI endpoint in Chapter 10.

## BM25 and TF-IDF (via `rank_bm25` and `scikit-learn`)

These are the two retrieval algorithms discussed conceptually in Chapter 2. You'll use existing, well-tested Python libraries (`rank_bm25` for BM25, `scikit-learn` for TF-IDF) rather than implementing the underlying math yourself — the goal of this book is to understand *how to combine and use* these techniques correctly, not to reimplement decades of information-retrieval research from scratch.

### Pandas

Pandas is Python's standard library for working with tabular data (rows and columns, like a spreadsheet). In this book, it plays a specific and important role: it's the "ground truth" table that the deterministic numeric verifier checks claims against — standing in for a real SQL database or data warehouse table in production.

### PostgreSQL + pgvector (production only — not needed for Part B)

In the production architecture (Part C), the vector embeddings and metadata are stored in PostgreSQL with the `pgvector` extension, which lets a standard relational database also perform fast vector similarity search. Part B's teaching code uses an in-memory Python list instead, so you don't need to install or configure a database to follow along.

### Docker (production only — not needed for Part B)

Docker packages an application and everything it needs to run (dependencies, configuration) into a portable container, ensuring "it works on my machine" also means it works on every other machine. The production repository scaffold (Part C) includes a `docker-compose.yml` file for running Postgres and Redis locally. It is not required for Part B, which runs directly on your machine with plain Python.

### What you need installed for Part B

```
pip install fastapi uvicorn pydantic rank-bm25 numpy scikit-learn
pandas
```

That's the entire dependency list for the teaching implementation you'll build in Part B. No API keys, no external services, no database setup required.

---

# Part B: Build It Yourself

## Chapter 8: Environment Setup

### Prerequisites

- Python 3.10 or newer installed (`python3 --version` to check)
- Basic command-line familiarity (navigating folders, running commands)
- A text editor (VS Code, PyCharm, or even a plain editor works fine)

### Step 1: Create a project folder

```
mkdir trustgraph-lite
cd trustgraph-lite
mkdir -p retriever claim_extractor verifiers trust_scorer api
```

This mirrors the production repository's `services/` structure (Part C), just flattened for a single-person tutorial project.

### Step 2: Create a virtual environment

A virtual environment keeps this project's dependencies separate from other Python projects on your machine.

```
python3 -m venv venv
source venv/bin/activate       # on Windows: venv\Scripts\activate
```

### Step 3: Install dependencies

```
pip install fastapi uvicorn pydantic rank-bm25 numpy scikit-learn
pandas
```

### Step 4: Verify your setup

```
python3 -c "import fastapi, pydantic, rank_bm25, numpy, sklearn,
pandas; print('All dependencies installed correctly.')"
```

If this prints the success message with no errors, you're ready for Chapter 6.

### What you're about to build

By the end of Part B, your folder will contain five small Python files — one per service — plus a `main.py` that wires them together into a single API. Each chapter builds and tests one file in isolation before moving to the next, exactly the same order the real 10-engineer team builds and tests their production services in Part C.

```
trustgraph-lite/
├── retriever/
│   └── retriever.py        (Chapter 6)
├── claim_extractor/
│   └── claim_extractor.py  (Chapter 7)
├── verifiers/
│   └── verifiers.py         (Chapter 8)
├── trust_scorer/
│   └── trust_scorer.py      (Chapter 9)
└── api/
    └── main.py               (Chapter 10)
```

---

## Chapter 9: Building the Retriever

This chapter implements exactly the hybrid retrieval approach from Chapter 2 — BM25 (sparse) + TF-IDF/SVD (dense stand-in), combined via Reciprocal Rank Fusion — as a small, self-contained, tested Python class.

## The code

Create `retriever/retriever.py`:

```
"""
TrustGraph-Lite — Retriever
===========================
Teaching implementation of hybrid retrieval: dense (TF-IDF + SVD,
standing in
for a real embedding model like OpenAI/Cohere embeddings) + sparse
(BM25),
combined via Reciprocal Rank Fusion (RRF).

In production, replace the dense vectorizer with a real embedding
model and
a vector database (pgvector, Pinecone, Weaviate). The interface below
stays
identical — that's the point of the typed contract in Part 3 of the
book.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field

import numpy as np
from rank_bm25 import BM25Okapi
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer


@dataclass
class Document:
    document_id: str
    chunk_id: str
    text: str
    metadata: dict = field(default_factory=dict)


@dataclass
class RetrievedEvidence:
    document_id: str
    chunk_id: str
    text: str
    retrieval_score: float
    retrieval_method: str


def _tokenize(text: str) -> list[str]:
```

```
    return text.lower().replace(",", " ").replace(".", " ").split()


class HybridRetriever:
    """
    A small, self-contained hybrid retriever.

    - Sparse side: BM25Okapi over tokenized text (this is exact-match
strong —
      catches IDs, names, numbers that dense embeddings often blur).
    - Dense side: TF-IDF + TruncatedSVD as a stand-in "embedding"
(captures
      semantic similarity without needing an external embedding API
call,
      so this chapter's code runs completely offline).
    - Combination: Reciprocal Rank Fusion (RRF), which combines two
ranked
      lists without needing to normalize incompatible score scales.
    """

    def __init__(self, documents: list[Document]):
        self.documents = documents
        self.texts = [d.text for d in documents]

        # Sparse (BM25)
        self.tokenized_corpus = [_tokenize(t) for t in self.texts]
        self.bm25 = BM25Okapi(self.tokenized_corpus)

        # Dense (TF-IDF -> SVD as a toy embedding space)
        self.vectorizer = TfidfVectorizer()
        tfidf_matrix = self.vectorizer.fit_transform(self.texts)
        n_components = min(50, tfidf_matrix.shape[1] - 1,
tfidf_matrix.shape[0] - 1)
        n_components = max(n_components, 1)
        self.svd = TruncatedSVD(n_components=n_components,
random_state=42)
        self.dense_matrix = self.svd.fit_transform(tfidf_matrix)

    def _dense_query_vector(self, query: str) -> np.ndarray:
        q_tfidf = self.vectorizer.transform([query])
        return self.svd.transform(q_tfidf)[0]

    def _cosine_sim(self, a: np.ndarray, b: np.ndarray) -> float:
        denom = (np.linalg.norm(a) * np.linalg.norm(b)) or 1e-9
        return float(np.dot(a, b) / denom)

    def retrieve(self, query: str, top_k: int = 5, rrf_k: int = 60) ->
list[RetrievedEvidence]:
        # --- Sparse ranking ---
```

```
        bm25_scores = self.bm25.get_scores(_tokenize(query))
        sparse_ranked = sorted(
            range(len(self.documents)), key=lambda i: bm25_scores[i],
reverse=True
        )

        # --- Dense ranking ---
        q_vec = self._dense_query_vector(query)
        dense_scores = [self._cosine_sim(q_vec, self.dense_matrix[i])
for i in range(len(self.documents))]
        dense_ranked = sorted(
            range(len(self.documents)), key=lambda i: dense_scores[i],
reverse=True
        )

        # --- Reciprocal Rank Fusion ---
        # RRF score for doc i = sum over each ranked list of 1 /
(rrf_k + rank_of_i)
        rrf_scores: dict[int, float] = {}
        for rank, doc_idx in enumerate(sparse_ranked):
            rrf_scores[doc_idx] = rrf_scores.get(doc_idx, 0.0) + 1.0 /
(rrf_k + rank + 1)
        for rank, doc_idx in enumerate(dense_ranked):
            rrf_scores[doc_idx] = rrf_scores.get(doc_idx, 0.0) + 1.0 /
(rrf_k + rank + 1)

        final_ranked = sorted(rrf_scores.items(), key=lambda kv:
kv[1], reverse=True)[:top_k]

        results = []
        for doc_idx, score in final_ranked:
            doc = self.documents[doc_idx]
            results.append(
                RetrievedEvidence(
                    document_id=doc.document_id,
                    chunk_id=doc.chunk_id,
                    text=doc.text,
                    retrieval_score=round(score, 5),
                    retrieval_method="hybrid_rrf",
                )
            )
        return results


if __name__ == "__main__":
    docs = [
        Document("doc1", "c1", "TrustGraph revenue grew 12% year over
year to reach $4.2 million in 2025."),
        Document("doc2", "c2", "The company's headquarters moved to
Noida in early 2024."),
```

```
        Document("doc3", "c3", "Customer churn rate decreased from 8%
to 5% after the Q3 product launch."),
        Document("doc4", "c4", "Revenue for the prior fiscal year was
$3.75 million, showing steady growth."),
        Document("doc5", "c5", "The engineering team is organized into
five pods of two people each."),
    ]
    retriever = HybridRetriever(docs)
    results = retriever.retrieve("What was the revenue growth?",
top_k=3)
    for r in results:
        print(f"[{r.retrieval_score}] {r.document_id}: {r.text}")
```

### Run it

```
cd retriever && python3 retriever.py
```

### Expected output

```
[0.03279] doc4: Revenue for the prior fiscal year was $3.75 million,
showing steady growth.
[0.03226] doc1: TrustGraph revenue grew 12% year over year to reach
$4.2 million in 2025.
[0.0315] doc5: The engineering team is organized into five pods of two
people each.
```

### What just happened

Notice that the query "What was the revenue growth?" correctly surfaced the two documents that discuss revenue (doc4 and doc1), even though neither document uses the word "growth" directly (doc1 says "grew," doc4 says "growth" only once) — this is the dense/semantic side of hybrid retrieval doing its job, catching related vocabulary rather than requiring an exact keyword match. The BM25 sparse side reinforces this because both documents also share meaningful overlapping words ("revenue," "million").

The unrelated document about team structure (doc5) still appears at rank 3 with a much lower score — a reminder that retrieval is a ranking, not a hard filter. In production, you would set a similarity threshold or rely on `top_k` to only pass the highest-confidence results downstream.

### Try it yourself

Change the query in the `if __name__ == "__main__":` block to `"What is the team structure?"` and re-run. You should see doc5 jump to the top of the ranking instead.

---

## Chapter 10: Building the Claim Extractor

This is where the "verify numbers with code, not language models" philosophy from Chapter 3 starts taking shape. This chapter's extractor splits a generated answer into individual claims

and classifies each one as numeric, entity, or narrative — the classification the Verification Router (Chapter 8) uses to decide how to check each claim.

## The code

Create `claim_extractor/claim_extractor.py`:

```
"""
TrustGraph-Lite — Claim Extractor
==================================
Teaching implementation of hybrid claim extraction: a rule-based
numeric/entity
extractor (standing in for spaCy NER + dependency parsing) combined
with an
LLM-style extractor for narrative claims. This is deliberately
"hybrid" per
the book's core design principle: never trust a single LLM pass to
define
claim structure for numeric facts that can be extracted
deterministically.

In production, swap the rule-based layer for spaCy's NER + dependency
parser,
and the "LLM extractor" stub for a real call to the Anthropic API.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field
from enum import Enum


class ClaimType(str, Enum):
    NUMERIC = "numeric"
    ENTITY = "entity"
    NARRATIVE = "narrative"


@dataclass
class Claim:
    text: str
    claim_type: ClaimType
    entities: list[str] = field(default_factory=list)
    numeric_values: list[dict] = field(default_factory=list)
    extraction_method: str = "hybrid"
    extraction_confidence: float = 0.9


# --- Deterministic numeric extraction
```

```
------------------------------------

_PERCENT_PATTERN = re.compile(r"(\d+(?:\.\d+)?)\s?%")
_MONEY_PATTERN = re.compile(r"\$\s?(\d+(?:\.\d+)?)\s?(million|billion|
thousand)?", re.IGNORECASE)


def _extract_numeric_claims(sentence: str) -> list[Claim]:
    claims = []

    for match in _PERCENT_PATTERN.finditer(sentence):
        value = float(match.group(1))
        claims.append(
            Claim(
                text=sentence.strip(),
                claim_type=ClaimType.NUMERIC,
                numeric_values=[{"value": value, "unit": "percent"}],
                extraction_method="rule_based",
                extraction_confidence=0.98,
            )
        )

    for match in _MONEY_PATTERN.finditer(sentence):
        value = float(match.group(1))
        unit = (match.group(2) or "").lower()
        multiplier = {"thousand": 1_000, "million": 1_000_000,
"billion": 1_000_000_000}.get(unit, 1)
        claims.append(
            Claim(
                text=sentence.strip(),
                claim_type=ClaimType.NUMERIC,
                numeric_values=[{"value": value * multiplier, "unit":
"usd"}],
                extraction_method="rule_based",
                extraction_confidence=0.98,
            )
        )

    return claims


# --- Simple entity extraction (toy NER)
----------------------------------

_CAPITALIZED_PHRASE = re.compile(r"\b([A-Z][a-zA-Z]+(?:\s[A-Z][a-zA-Z]
+)*)\b")
_STOPWORDS_AT_START = {"The", "This", "That", "It", "A", "An"}
_COMMON_SENTENCE_STARTERS = {
    "Customers", "Customer", "Users", "User", "People", "Revenue",
```

```
"Growth",
    "Sales", "Prices", "Costs", "Teams", "Employees",
}


def _extract_entity_claims(sentence: str) -> list[Claim]:
    matches = _CAPITALIZED_PHRASE.findall(sentence)
    # Drop the sentence-initial word if it's a common noun rather than
a proper
    # noun — a toy heuristic. Multi-word capitalized phrases (e.g.
"New York")
    # are almost always genuine entities and are kept regardless of
position.
    entities = []
    for m in matches:
        is_multiword = " " in m
        is_sentence_start_common_word = (
            sentence.strip().startswith(m) and m in
(_STOPWORDS_AT_START | _COMMON_SENTENCE_STARTERS)
        )
        if is_sentence_start_common_word and not is_multiword:
            continue
        entities.append(m)

    if not entities:
        return []
    return [
        Claim(
            text=sentence.strip(),
            claim_type=ClaimType.ENTITY,
            entities=entities,
            extraction_method="rule_based_ner",
            extraction_confidence=0.75,
        )
    ]


# --- Narrative fallback (would call an LLM in production)
------------------

def _extract_narrative_claim(sentence: str) -> Claim:
    """
    Any sentence that isn't purely numeric or a simple entity
statement is
    treated as a narrative claim requiring LLM-based verification. In
    production this is where you'd call the Anthropic API to confirm
the
    sentence expresses a single coherent claim (splitting compound
sentences
    if needed).
```

```
    """
    return Claim(
        text=sentence.strip(),
        claim_type=ClaimType.NARRATIVE,
        extraction_method="llm_stub",
        extraction_confidence=0.85,
    )


def extract_claims(response_text: str) -> list[Claim]:
    """
    Hybrid extraction pipeline: split into sentences, run numeric +
entity
    extractors first (deterministic, high precision), then fall back
to a
    narrative claim for anything left un-typed.
    """
    sentences = [s.strip() for s in re.split(r"(?<=[.!?])\s+",
response_text) if s.strip()]
    claims: list[Claim] = []

    for sentence in sentences:
        numeric = _extract_numeric_claims(sentence)
        entity = _extract_entity_claims(sentence)

        if numeric:
            claims.extend(numeric)
        elif entity:
            claims.extend(entity)
        else:
            claims.append(_extract_narrative_claim(sentence))

    return claims


if __name__ == "__main__":
    answer = (
        "TrustGraph revenue grew 12% year over year to reach $4.2
million in 2025. "
        "The company's headquarters moved to Noida in early 2024. "
        "Customers generally reported higher satisfaction after the
redesign."
    )
    for c in extract_claims(answer):
        print(f"[{c.claim_type.value:9s} |
{c.extraction_confidence:.2f}] {c.text}")
        if c.numeric_values:
            print(f"           numeric: {c.numeric_values}")
        if c.entities:
            print(f"           entities: {c.entities}")
```

## Run it

```
cd claim_extractor && python3 claim_extractor.py
```

## Expected output

```
[numeric   | 0.98] TrustGraph revenue grew 12% year over year to reach
$4.2 million in 2025.
           numeric: [{'value': 12.0, 'unit': 'percent'}]
[numeric   | 0.98] TrustGraph revenue grew 12% year over year to reach
$4.2 million in 2025.
           numeric: [{'value': 4200000.0, 'unit': 'usd'}]
[entity    | 0.75] The company's headquarters moved to Noida in early
2024.
           entities: ['Noida']
[narrative | 0.85] Customers generally reported higher satisfaction
after the redesign.
```

## What just happened

The first sentence produced *two* numeric claims — one for the percentage, one for the dollar amount — because each is an independently verifiable fact even though they appear in the same sentence. This is exactly the atomic decomposition described in Chapter 3: a single sentence can contain multiple claims that should each be checked on their own.

The second sentence was correctly classified as an entity claim (Noida), and the third — which contains no numbers and no clean entity — correctly fell through to the narrative fallback, which in production would be verified by an LLM checking whether the retrieved evidence actually supports the sentiment claimed.

## A note on the toy NER

The entity extractor here is a simple regex looking for capitalized words, with a small hand-built exception list to avoid treating common sentence-starting words ("Customers," "The") as entities. This is a real limitation — a proper implementation uses a trained NER model (like spaCy's), which understands grammatical structure well enough to avoid this kind of false positive far more reliably. Part C, Chapter 12 shows exactly what to swap in for production.

## Try it yourself

Feed it a sentence with a claim your rules don't yet handle — e.g., a date ("The report was published in March 2025") — and see it fall through to the narrative bucket. This is a good exercise in noticing where a rule-based extractor's coverage ends, which is precisely why production systems use *hybrid* extraction rather than rules alone.

---

## Chapter 11: Building the Verification Router and Verifiers

This is the most important chapter in the book. Everything else exists to feed claims into this component and use its output.

### The code

Create `verifiers/verifiers.py`:

```
"""
TrustGraph-Lite — Verification Router + Verifiers
==================================================
This is the single most important file in the teaching codebase. It
implements the book's core idea: NUMBERS ARE VERIFIED WITH CODE, NOT
WITH
LANGUAGE MODELS. Only narrative claims — ones that can't be checked by
a
deterministic lookup — go to an LLM verifier.

Three verifiers are shown:
  1. SQLVerifier      — for numeric claims, checks the claimed number
                        against a "ground truth" table (a pandas
DataFrame
                        standing in for a real data warehouse table).
  2. RulesVerifier    — for entity claims, checks whether the named
entity
                        appears in a known-entities list (standing in
for
                        a knowledge graph lookup).
  3. LLMVerifier      — for narrative claims only, asks an LLM
whether the
                        claim is supported by the retrieved evidence
text.
                        (Stubbed here with a simple heuristic so the
whole
                        chapter runs without needing an API key; swap
in a
                        real Anthropic API call for production.)
"""

from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum

import pandas as pd

import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
```

```
"claim_extractor"))
from claim_extractor import Claim, ClaimType  # noqa: E402


class VerificationStatus(str, Enum):
    VERIFIED = "verified"
    CONTRADICTED = "contradicted"
    UNSUPPORTED = "unsupported"
    INSUFFICIENT_EVIDENCE = "insufficient_evidence"


class FailureCode(str, Enum):
    NUMERIC_MISMATCH = "NUMERIC_MISMATCH"
    NO_SUPPORT = "NO_SUPPORT"
    RETRIEVAL_MISS = "RETRIEVAL_MISS"
    AMBIGUOUS_CLAIM = "AMBIGUOUS_CLAIM"


@dataclass
class VerificationResult:
    claim_text: str
    status: VerificationStatus
    verification_method: str
    confidence: float
    failure_code: FailureCode | None = None
    reason: str = ""


# --- Deterministic numeric verifier (the "SQL/Pandas verifier")
-----------

class SQLVerifier:
    """
    Verifies numeric claims against a ground-truth table. In
production this
    runs a real SQL query against the warehouse; here it's a pandas
lookup
    against a small in-memory "facts" table, which is exactly the same
idea
    at teaching scale.
    """

    def __init__(self, facts_table: pd.DataFrame):
        self.facts = facts_table

    def verify(self, claim: Claim, tolerance_pct: float = 2.0) ->
VerificationResult:
        if not claim.numeric_values:
            return VerificationResult(
```

```
                claim.text, VerificationStatus.INSUFFICIENT_EVIDENCE,
                "sql", 0.0, FailureCode.AMBIGUOUS_CLAIM, "No numeric
value found in claim."
            )

        claimed = claim.numeric_values[0]
        unit = claimed["unit"]
        value = claimed["value"]

        matching_rows = self.facts[self.facts["unit"] == unit]
        if matching_rows.empty:
            return VerificationResult(
                claim.text, VerificationStatus.INSUFFICIENT_EVIDENCE,
                "sql", 0.0, FailureCode.NO_SUPPORT, f"No ground-truth
data for unit '{unit}'."
            )

        ground_truth = matching_rows.iloc[0]["value"]
        pct_diff = abs(value - ground_truth) / max(abs(ground_truth),
1e-9) * 100

        if pct_diff <= tolerance_pct:
            return VerificationResult(
                claim.text, VerificationStatus.VERIFIED, "sql", 0.99,
                reason=f"Claimed {value} matches ground truth
{ground_truth} (within {tolerance_pct}%)."
            )
        else:
            return VerificationResult(
                claim.text, VerificationStatus.CONTRADICTED, "sql",
0.95,
                FailureCode.NUMERIC_MISMATCH,
                f"Claimed {value} but ground truth is {ground_truth}
({pct_diff:.1f}% difference)."
            )


# --- Rules-based entity verifier
-----------------------------------------

class RulesVerifier:
    """Checks named entities against a known-entities registry (a toy
stand-in
    for a knowledge graph lookup)."""

    def __init__(self, known_entities: set[str]):
        self.known_entities = known_entities

    def verify(self, claim: Claim) -> VerificationResult:
```

```
        if not claim.entities:
            return VerificationResult(
                claim.text, VerificationStatus.INSUFFICIENT_EVIDENCE,
                "rules_engine", 0.0, FailureCode.AMBIGUOUS_CLAIM, "No
entity found in claim."
            )

        unknown = [e for e in claim.entities if e not in
self.known_entities]
        if unknown:
            return VerificationResult(
                claim.text, VerificationStatus.UNSUPPORTED,
"rules_engine", 0.6,
                FailureCode.NO_SUPPORT, f"Entities not in known
registry: {unknown}"
            )
        return VerificationResult(
            claim.text, VerificationStatus.VERIFIED, "rules_engine",
0.9,
            reason=f"All entities {claim.entities} found in registry."
        )


# --- LLM verifier (stubbed)
------------------------------------------------

class LLMVerifier:
    """
    Verifies narrative claims against retrieved evidence text. In
production,
    replace `_ask_llm` with a real call to the Anthropic API asking:
    "Does this evidence support, contradict, or fail to address this
claim?"

    The stub here uses simple word-overlap as a stand-in so the
chapter's
    code runs without an API key — clearly marked as a placeholder.
    """

    def verify(self, claim: Claim, evidence_texts: list[str]) ->
VerificationResult:
        if not evidence_texts:
            return VerificationResult(
                claim.text, VerificationStatus.INSUFFICIENT_EVIDENCE,
                "llm", 0.0, FailureCode.RETRIEVAL_MISS, "No evidence
retrieved."
            )

        overlap_scores = [self._word_overlap(claim.text, ev) for ev in
evidence_texts]
```

```
        best_overlap = max(overlap_scores)

        if best_overlap > 0.3:
            return VerificationResult(
                claim.text, VerificationStatus.VERIFIED, "llm",
round(best_overlap, 2),
                reason="Narrative claim supported by retrieved
evidence (word-overlap stub)."
            )
        return VerificationResult(
            claim.text, VerificationStatus.UNSUPPORTED, "llm",
round(best_overlap, 2),
            FailureCode.NO_SUPPORT, "Narrative claim not well
supported by retrieved evidence."
        )

    @staticmethod
    def _word_overlap(a: str, b: str) -> float:
        set_a, set_b = set(a.lower().split()), set(b.lower().split())
        if not set_a:
            return 0.0
        return len(set_a & set_b) / len(set_a)


# --- Verification Router
-----------------------------------------------------

class VerificationRouter:
    """
    Classifies each claim by type and dispatches to the correct
verifier.
    This is the component that keeps LLM calls limited to what
actually
    needs language-level judgment.
    """

    def __init__(self, sql_verifier: SQLVerifier, rules_verifier:
RulesVerifier, llm_verifier: LLMVerifier):
        self.sql_verifier = sql_verifier
        self.rules_verifier = rules_verifier
        self.llm_verifier = llm_verifier

    def route_and_verify(self, claim: Claim, evidence_texts:
list[str]) -> VerificationResult:
        if claim.claim_type == ClaimType.NUMERIC:
            return self.sql_verifier.verify(claim)
        elif claim.claim_type == ClaimType.ENTITY:
            return self.rules_verifier.verify(claim)
        else:
            return self.llm_verifier.verify(claim, evidence_texts)
```

```
if __name__ == "__main__":
    from claim_extractor import extract_claims

    facts = pd.DataFrame([
        {"unit": "percent", "value": 12.0},
        {"unit": "usd", "value": 4_200_000.0},
    ])
    known_entities = {"Noida", "TrustGraph"}

    router = VerificationRouter(
        SQLVerifier(facts), RulesVerifier(known_entities),
LLMVerifier()
    )

    answer = (
        "TrustGraph revenue grew 12% year over year to reach $4.2
million in 2025. "
        "The company's headquarters moved to Noida in early 2024. "
        "Customers generally reported higher satisfaction after the
redesign."
    )
    evidence = ["TrustGraph revenue grew 12% year over year to reach
$4.2 million in 2025."]

    for claim in extract_claims(answer):
        result = router.route_and_verify(claim, evidence)
        print(f"[{result.status.value:12s} |
{result.verification_method:6s} | {result.confidence:.2f}]
{result.reason or claim.text}")
```

## Run it

```
cd verifiers && python3 verifiers.py
```

(Make sure `claim_extractor.py` is importable — the code adds `../claim_extractor` to the Python path automatically, but you do need to run this from inside the `verifiers/` folder as shown.)

## Expected output

```
[verified     | sql    | 0.99] Claimed 12.0 matches ground truth 12.0
(within 2.0%).
[verified     | sql    | 0.99] Claimed 4200000.0 matches ground truth
4200000.0 (within 2.0%).
[verified     | rules_engine | 0.90] All entities ['Noida'] found in
registry.
[unsupported  | llm    | 0.00] Narrative claim not well supported by
retrieved evidence.
```

### What just happened, step by step

1. **The two numeric claims** ("12%" and "$4.2 million") were routed to `SQLVerifier`, which looked them up against a small `pandas` DataFrame standing in for a real ground-truth data table. Both matched within the 2% tolerance, so both are `VERIFIED` — and notice this required *zero* language model calls. This is the deterministic-verification-rate dimension from Chapter 3 in action.

2. **The entity claim** ("Noida") was routed to `RulesVerifier`, which checked it against a small known-entities set. It's present, so it's `VERIFIED` — again, no LLM involved.

3. **The narrative claim** ("customers generally reported higher satisfaction") was routed to `LLMVerifier`, because there's no deterministic way to check a sentiment claim like this. The stub implementation checks word overlap between the claim and the retrieved evidence — since the only evidence provided was about revenue, not customer satisfaction, the overlap is low and the claim comes back `UNSUPPORTED`. **This is correct behavior** — the system is telling you, accurately, "I don't have evidence for this specific claim," rather than assuming it's fine just because the rest of the answer checked out.

### The most important exercise in the book

Change the `answer` variable in the `if __name__ == "__main__":` block to state a wrong number — for example, change "12%" to "45%" and "$4.2 million" to "$9 million" — and re-run. You should see:

```
[contradicted | sql    | 0.95] Claimed 45.0 but ground truth is 12.0
(275.0% difference).
[contradicted | sql    | 0.95] Claimed 9000000.0 but ground truth is
4200000.0 (114.3% difference).
```

This is the entire point of the book, in one terminal output: a hallucinated number gets caught by code, with an exact, explainable reason — not by hoping the language model doubts itself.

### A note on the LLM verifier stub

`LLMVerifier._word_overlap` is a deliberately simple stand-in so this chapter runs with no API key. In production, replace this method's body with a real call to the Anthropic API, asking it to judge whether the evidence text supports, contradicts, or fails to address the claim — the method signature and the rest of the router logic do not need to change at all. This is exactly why the book insists on typed interfaces (Part C): the *implementation* of a verifier can be swapped without touching anything that calls it.

---

## Chapter 12: Building the Trust Scorer

This chapter turns a list of verification results into the explainable, 7-dimension trust score described in Chapter 3.

## The code

Create `trust_scorer/trust_scorer.py`:

```
"""
TrustGraph-Lite — Trust Scorer
==============================
Computes the 7-dimension trust score from a list of
VerificationResults.
This is the component that replaces a single opaque "confidence: 91%"
with
an explainable breakdown a reviewer can actually inspect.
"""

from __future__ import annotations

from dataclasses import dataclass

import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"verifiers"))
from verifiers import VerificationResult, VerificationStatus  # noqa:
E402


@dataclass
class TrustDimensions:
    retrieval_quality: float
    evidence_coverage: float
    claim_verification_rate: float
    freshness: float
    conflict_score: float
    source_authority: float
    deterministic_verification_rate: float


@dataclass
class TrustScore:
    overall_score: float
    dimensions: TrustDimensions


def compute_trust_score(
    results: list[VerificationResult],
    retrieval_recall_estimate: float = 0.9,
    source_authority: float = 0.85,
    freshness: float = 0.9,
) -> TrustScore:
    """
```

```
    v1 formula: equal-weighted average of 7 dimensions (see TRD §5).
Weights
    are meant to be tuned later against real calibration data — this
is the
    honest starting point, not a claim of optimality.
    """
    total = len(results) or 1

    verified = sum(1 for r in results if r.status ==
VerificationStatus.VERIFIED)
    contradicted = sum(1 for r in results if r.status ==
VerificationStatus.CONTRADICTED)
    deterministic = sum(1 for r in results if r.verification_method in
("sql", "pandas", "rules_engine"))

    claim_verification_rate = verified / total
    conflict_score = 1.0 - (contradicted / total)  # higher = less
conflict
    deterministic_verification_rate = deterministic / total

    # evidence_coverage: fraction of claims that were NOT
insufficient_evidence
    insufficient = sum(1 for r in results if r.status ==
VerificationStatus.INSUFFICIENT_EVIDENCE)
    evidence_coverage = 1.0 - (insufficient / total)

    dims = TrustDimensions(
        retrieval_quality=retrieval_recall_estimate,
        evidence_coverage=evidence_coverage,
        claim_verification_rate=claim_verification_rate,
        freshness=freshness,
        conflict_score=conflict_score,
        source_authority=source_authority,

deterministic_verification_rate=deterministic_verification_rate,
    )

    weights = [1 / 7] * 7
    values = [
        dims.retrieval_quality, dims.evidence_coverage,
dims.claim_verification_rate,
        dims.freshness, dims.conflict_score, dims.source_authority,
        dims.deterministic_verification_rate,
    ]
    overall = sum(w * v for w, v in zip(weights, values))

    return TrustScore(overall_score=round(overall, 3),
dimensions=dims)
```

```
if __name__ == "__main__":
    from claim_extractor import extract_claims
    from verifiers import VerificationRouter, SQLVerifier,
RulesVerifier, LLMVerifier
    import pandas as pd

    facts = pd.DataFrame([
        {"unit": "percent", "value": 12.0},
        {"unit": "usd", "value": 4_200_000.0},
    ])
    router = VerificationRouter(SQLVerifier(facts),
RulesVerifier({"Noida", "TrustGraph"}), LLMVerifier())

    answer = (
        "TrustGraph revenue grew 12% year over year to reach $4.2
million in 2025. "
        "The company's headquarters moved to Noida in early 2024. "
        "Customers generally reported higher satisfaction after the
redesign."
    )
    evidence = ["TrustGraph revenue grew 12% year over year to reach
$4.2 million in 2025."]

    results = [router.route_and_verify(c, evidence) for c in
extract_claims(answer)]
    score = compute_trust_score(results)

    print(f"Overall Trust Score: {score.overall_score}")
    for field_name, value in score.dimensions.__dict__.items():
        print(f"  {field_name:32s}: {value:.2f}")
```

## Run it

```
cd trust_scorer && PYTHONPATH="../claim_extractor:../verifiers:."
python3 trust_scorer.py
```

## Expected output

```
Overall Trust Score: 0.879
  retrieval_quality               : 0.90
  evidence_coverage               : 1.00
  claim_verification_rate         : 0.75
  freshness                       : 0.90
  conflict_score                  : 1.00
  source_authority                : 0.85
  deterministic_verification_rate : 0.75
```

## What just happened

The overall score (0.879) is a simple equal-weighted average of the seven dimensions — the “v1-equal-weights” calibration mentioned in the code. This is an honest, unoptimized starting

point, not a claim that these weights are ideal; Part D's evaluation framework is exactly what would let you tune these weights against real labeled outcomes later.

Look at what each dimension is actually telling you here:

- **claim_verification_rate = 0.75**: three of the four claims were confirmed `VERIFIED` (the narrative one was `UNSUPPORTED`, pulling this down from a perfect 1.0).
- **conflict_score = 1.00**: nothing was `CONTRADICTED`, so there's no detected conflict.
- **deterministic_verification_rate = 0.75**: three of four claims were checked by code (SQL or rules), not by the LLM stub — a meaningfully strong number, because deterministic checks are more reliable than LLM judgment calls.

### Try it yourself: watch the score drop

Go back to the `verifiers.py` example from Chapter 8 and change the answer to include the wrong revenue numbers (45%, $9 million) instead of the correct ones. Re-run this chapter's script with that modified answer, and you'll see the overall score fall — try it and note the new value for `conflict_score` and `claim_verification_rate` specifically. This is the mechanism, working exactly as designed: a wrong number doesn't just get silently included, it visibly and measurably damages the reported trust score.

---

## Chapter 13: Wiring It All Together — The Live API

This chapter connects everything from Chapters 6-9 into a single FastAPI application with one endpoint: `POST /query`. This is the moment the book has been building toward — a real, running system you can send a question to and get back an answer, a claim-by-claim verification breakdown, and a full trust score.

### The code

Create `api/main.py`:

```
"""
TrustGraph-Lite — API Gateway
==============================
Wires retriever -> claim extractor -> verification router -> trust
scorer
into a single FastAPI endpoint: POST /query

Run it:
    uvicorn main:app --reload --port 8000

Try it:
    curl -X POST http://localhost:8000/query \\
      -H "Content-Type: application/json" \\
      -d '{"query": "What was the revenue growth?", "answer":
```

```
"TrustGraph revenue grew 12% year over year to reach $4.2 million in
2025."}'
"""

from __future__ import annotations

import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"retriever"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"claim_extractor"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"verifiers"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"trust_scorer"))

import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel

from retriever import Document, HybridRetriever
from claim_extractor import extract_claims
from verifiers import VerificationRouter, SQLVerifier, RulesVerifier,
LLMVerifier
from trust_scorer import compute_trust_score

app = FastAPI(title="TrustGraph-Lite", version="0.1")

# --- Fixture data (in production: real document store + real facts
table) --

DOCS = [
    Document("doc1", "c1", "TrustGraph revenue grew 12% year over year
to reach $4.2 million in 2025."),
    Document("doc2", "c2", "The company's headquarters moved to Noida
in early 2024."),
    Document("doc3", "c3", "Customer churn rate decreased from 8% to
5% after the Q3 product launch."),
    Document("doc4", "c4", "Revenue for the prior fiscal year was
$3.75 million, showing steady growth."),
]
FACTS = pd.DataFrame([
    {"unit": "percent", "value": 12.0},
    {"unit": "usd", "value": 4_200_000.0},
])
KNOWN_ENTITIES = {"Noida", "TrustGraph"}

retriever = HybridRetriever(DOCS)
router = VerificationRouter(SQLVerifier(FACTS),
```

```
RulesVerifier(KNOWN_ENTITIES), LLMVerifier())


class QueryRequest(BaseModel):
    query: str
    answer: str  # in a full system, the Generator produces this; here
it's supplied directly


class ClaimResult(BaseModel):
    text: str
    claim_type: str
    status: str
    verification_method: str
    confidence: float
    reason: str


class QueryResponse(BaseModel):
    query: str
    answer: str
    claims: list[ClaimResult]
    overall_trust_score: float
    trust_dimensions: dict


@app.post("/query", response_model=QueryResponse)
def query(request: QueryRequest):
    evidence = retriever.retrieve(request.query, top_k=3)
    evidence_texts = [e.text for e in evidence]

    claims = extract_claims(request.answer)
    verification_results = [router.route_and_verify(c, evidence_texts)
for c in claims]
    trust_score = compute_trust_score(verification_results)

    claim_results = [
        ClaimResult(
            text=r.claim_text,
            claim_type=c.claim_type.value,
            status=r.status.value,
            verification_method=r.verification_method,
            confidence=r.confidence,
            reason=r.reason,
        )
        for c, r in zip(claims, verification_results)
    ]

    return QueryResponse(
```

```
        query=request.query,
        answer=request.answer,
        claims=claim_results,
        overall_trust_score=trust_score.overall_score,
        trust_dimensions=trust_score.dimensions.__dict__,
    )


@app.get("/health")
def health():
    return {"status": "ok"}
```

### Run it

```
cd api && uvicorn main:app --reload --port 8000
```

You should see:

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to
quit)
INFO:     Application startup complete.
```

Leave this running, and open a **second** terminal for the next steps.

### Demo 1: A correct answer

```
curl -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{"query": "What was the revenue growth?", "answer": "TrustGraph
revenue grew 12% year over year to reach $4.2 million in 2025. The
company headquarters moved to Noida in early 2024. Customers generally
reported higher satisfaction after the redesign."}'
```

**Actual output from this exact request:**


```
{
    "query": "What was the revenue growth?",
    "overall_trust_score": 0.879,
    "claims": [
        {
            "text": "TrustGraph revenue grew 12% year over year to
reach $4.2 million in 2025.",
            "claim_type": "numeric", "status": "verified",
            "verification_method": "sql", "confidence": 0.99,
            "reason": "Claimed 12.0 matches ground truth 12.0 (within
2.0%)."
        },
        {
            "text": "TrustGraph revenue grew 12% year over year to
reach $4.2 million in 2025.",
            "claim_type": "numeric", "status": "verified",
            "verification_method": "sql", "confidence": 0.99,
            "reason": "Claimed 4200000.0 matches ground truth
```

```
4200000.0 (within 2.0%)."
        },
        {
            "text": "The company headquarters moved to Noida in early
2024.",
            "claim_type": "entity", "status": "verified",
            "verification_method": "rules_engine", "confidence": 0.9,
            "reason": "All entities ['Noida'] found in registry."
        },
        {
            "text": "Customers generally reported higher satisfaction
after the redesign.",
            "claim_type": "narrative", "status": "unsupported",
            "verification_method": "llm", "confidence": 0.12,
            "reason": "Narrative claim not well supported by retrieved
evidence."
        }
    ],
    "trust_dimensions": {
        "retrieval_quality": 0.9, "evidence_coverage": 1.0,
        "claim_verification_rate": 0.75, "freshness": 0.9,
        "conflict_score": 1.0, "source_authority": 0.85,
        "deterministic_verification_rate": 0.75
    }
}
```


### Demo 2: A hallucinated answer — the moment this whole book is about

```
curl -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{"query": "revenue growth", "answer": "TrustGraph revenue grew
45% year over year to reach $9 million in 2025."}'
```

**Actual output from this exact request:**


```
{
    "overall_trust_score": 0.664,
    "claims": [
        {
            "text": "TrustGraph revenue grew 45% year over year to
reach $9 million in 2025.",
            "claim_type": "numeric", "status": "contradicted",
            "verification_method": "sql", "confidence": 0.95,
            "reason": "Claimed 45.0 but ground truth is 12.0 (275.0%
difference)."
        },
        {
            "text": "TrustGraph revenue grew 45% year over year to
reach $9 million in 2025.",
            "claim_type": "numeric", "status": "contradicted",
            "verification_method": "sql", "confidence": 0.95,
```

```
            "reason": "Claimed 9000000.0 but ground truth is 4200000.0
(114.3% difference)."
        }
    ],
    "trust_dimensions": {
        "claim_verification_rate": 0.0,
        "conflict_score": 0.0,
        "deterministic_verification_rate": 1.0
    }
}
```

### What just happened

The overall trust score fell from **0.879 to 0.664** — a real, measured, explainable drop, not a vague warning. Both fabricated numbers were caught with the *exact* correct number, the exact ground truth, and the exact percentage discrepancy, entirely through arithmetic — no LLM was asked "does this sound right?" and no LLM had a chance to be fooled by its own fluent-sounding hallucination.

This is the complete idea of the book, running on your own machine, on code you just wrote and can read line by line.

### Health check

```
curl http://localhost:8000/health
# {"status": "ok"}
```

### What you have now

A working, if small, version of every core capability described in the rest of this book: hybrid retrieval, hybrid claim extraction, a verification router splitting work between deterministic and LLM verification, and an explainable trust score — all wired into a real API you can call from any HTTP client. Part C now shows exactly how this maps onto the production-grade, typed, 10-engineer-team architecture.

---

## Chapter 14: Extended API Scenarios

Chapter 10 showed the API catching a fully hallucinated answer. This chapter runs three more real scenarios against the live server — an entity error, a mostly-correct multi-claim answer, and a genuinely correct one — so you can see the trust score behave differently across each, not just in the all-or-nothing case.

### Scenario 1: An entity error

```
curl -X POST http://localhost:8000/query -H "Content-Type:
application/json" \
  -d '{"query": "where is HQ", "answer": "The company headquarters
moved to Singapore in early 2024."}'
```

```
{
  "claims": [{
    "text": "The company headquarters moved to Singapore in early
2024.",
    "claim_type": "entity",
    "status": "unsupported",
    "verification_method": "rules_engine",
    "confidence": 0.6,
    "reason": "Entities not in known registry: ['Singapore']"
  }],
  "overall_trust_score": 0.807,
  "trust_dimensions": {
    "claim_verification_rate": 0.0,
    "deterministic_verification_rate": 1.0,
    ...
  }
}
```

Notice `overall_trust_score` is 0.807 — not near zero, even though the one claim failed. This is the seven-dimension formula in action: `retrieval_quality`, `freshness`, and `source_authority` are unaffected by this single claim's failure, so they pull the average up even as `claim_verification_rate` correctly drops to 0.0. This is exactly why Chapter 9 argued for a multi-dimension score over a single number — a reviewer looking only at "80.7% trust" would miss that the one and only claim in this answer was flagged as unsupported; looking at the dimension breakdown makes it immediately visible.

### Scenario 2: A mostly-correct multi-claim answer

```
curl -X POST http://localhost:8000/query -H "Content-Type:
application/json" \
  -d '{"query": "give me a full update", "answer": "Revenue grew 12%
to reach $4.2 million. The company headquarters moved to Noida.
Customer churn decreased from 8% to 1%."}'
```

Five claims are extracted: two numeric (revenue), one entity (Noida), and two numeric again (churn before/after — Chapter 12's role-aware extraction correctly splits "from 8% to 1%" into separate before/after claims). Four verify correctly; the fifth — churn falling to 1% — is caught as `contradicted`, because the real value is 5%, not 1%:

```
{
  "text": "Customer churn decreased from 8% to 1%.",
  "status": "contradicted",
  "reason": "Claimed 1.0 but ground truth for 'churn_after_pct' is 5.0
(80.0% difference)."
}
```

`overall_trust_score` lands at 0.893 — high, but visibly lower than a fully-correct answer would score, and `conflict_score` drops to 0.8 while `claim_verification_rate` drops to 0.8 (4 of 5 claims verified). This is the clearest illustration in the book of claim-level verification's actual value: an answer-level checker would have to decide whether to accept or

reject this whole paragraph; TrustGraph instead tells you precisely which one of five specific numbers is wrong, while confirming the other four.

### Scenario 3: Using the dashboard for the same scenario

Open `admin_dashboard.html` (Chapter 21) in a browser, paste the same query and answer from Scenario 2, and click "Check This Answer." You'll see the same JSON rendered as five color-coded claim cards — green for the four verified claims, red for the one contradicted churn figure — which is the smallest possible real version of the Claim Explorer screen (Chapter 26) actually working end to end, not just described.

### What these three scenarios together demonstrate

Chapter 10 showed the extremes — a fully correct answer and a fully hallucinated one. These three scenarios fill in the middle: a single wrong entity, a single wrong number buried among four correct ones, and how the seven-dimension trust score responds differently and informatively to each. This gradation — not just "trustworthy" versus "not trustworthy," but a legible breakdown of exactly what went wrong and what didn't — is the actual product this book has been building toward since Chapter 3's definition of what "trust" means.

---

## Chapter 15: Writing Unit Tests for Every Component

Every chapter in Part B ended with a `if __name__ == "__main__":` block you could run directly — useful for seeing a component work, but not the same as a real test suite that runs automatically and catches regressions. This chapter adds that, using `pytest`.

### Why this matters more than it seems

Chapter 11's benchmark found two real bugs, and Chapter 12 found three more. Unit tests are a different, complementary safety net: they check specific, known behaviors stay correct as you keep changing code, catching regressions in seconds rather than requiring a full benchmark run.

### Setting up

```
pip install pytest pytest-cov
mkdir tests
```

### Testing the retriever

```
# tests/test_retriever.py
from retriever import Document, HybridRetriever

def make_test_docs():
    return [
        Document("d1", "c1", "Revenue grew 12 percent to reach 4.2
million dollars in 2025."),
        Document("d2", "c2", "The company headquarters moved to Noida
```

```
in early 2024."),
        Document("d3", "c3", "Customer churn decreased from 8 percent
to 5 percent."),
    ]

def test_retriever_finds_relevant_document_first():
    r = HybridRetriever(make_test_docs())
    results = r.retrieve("what was the revenue", top_k=1)
    assert results[0].document_id == "d1"

def test_retriever_score_ordering_is_descending():
    r = HybridRetriever(make_test_docs())
    results = r.retrieve("churn rate", top_k=3)
    scores = [res.retrieval_score for res in results]
    assert scores == sorted(scores, reverse=True)
```

Notice these tests check *behavior*, not implementation — they don't assert anything about how RRF combines scores internally, only that the observable result (the right document ranks first, scores are ordered correctly) holds. That's deliberate: if you later swap the dense retriever for a real embedding model, these tests should keep passing without modification, because they test the contract, not the mechanism.

### Testing the parts most worth testing: verifiers and the router

```
# tests/test_verifiers.py
def test_sql_verifier_catches_wrong_number():
    v = SQLVerifier(make_facts())
    claim = Claim(text="Revenue grew 45%",
claim_type=ClaimType.NUMERIC,
                  numeric_values=[{"value": 45.0, "unit":
"percent"}])
    result = v.verify(claim)
    assert result.status == VerificationStatus.CONTRADICTED

def test_router_dispatches_numeric_to_sql():
    router = VerificationRouter(SQLVerifier(make_facts()),
RulesVerifier({"Noida"}), LLMVerifier())
    claim = Claim(text="Revenue grew 12%",
claim_type=ClaimType.NUMERIC,
                  numeric_values=[{"value": 12.0, "unit":
"percent"}])
    result = router.route_and_verify(claim, [])
    assert result.verification_method == "sql"
```

This second test — checking the router dispatches to the *correct* verifier by type — is exactly the kind of test that would have caught a routing regression immediately, rather than waiting for the benchmark to notice a drop in F1 several steps later. Unit tests and benchmarks aren't competing tools; they catch different classes of problem at different speeds.

### Running the full suite

```
pytest tests/ -v
```

```
tests/test_claim_extractor.py::test_extracts_percent_claim PASSED
tests/test_claim_extractor.py::test_from_to_pattern_produces_before_an
d_after_roles PASSED
tests/test_retriever.py::test_retriever_finds_relevant_document_for_ex
act_keyword_match PASSED
tests/test_trust_scorer.py::test_all_verified_gives_high_score PASSED
tests/test_verifiers.py::test_router_dispatches_numeric_to_sql PASSED
...
============================== 25 passed in 1.36s
==============================
```

All 25 tests across all four components pass — this is a real run, not a hypothetical one.

### What belongs in a unit test versus a benchmark

Unit tests check specific, deterministic behaviors you already know the answer to ("does the SQL verifier correctly flag 45% as wrong when the true value is 12%"). The benchmark (Chapter 11) checks aggregate behavior across many realistic cases, and is what surfaces failure modes you *didn't* already know to test for. A mature codebase needs both — unit tests as a fast, specific safety net for known behavior, and the benchmark as the discovery mechanism for unknown failure modes. Neither replaces the other.

### Coverage as a floor, not a target

Chapter 20's Engineering Standards sets an 80% coverage floor on services and 100% on the shared contracts library. Treat that as a minimum to catch obviously untested code paths, not a target to chase for its own sake — a component with 100% coverage and no test that actually checks a wrong number gets flagged (like the router-dispatch test above) is worse than one at 85% coverage that does.

---

## Chapter 16: Benchmarking Your System

Everything you built in Chapters 6-10 works — you saw it catch a hallucinated revenue figure live in Chapter 10. But "it worked on my one example" is not evidence of anything. This chapter builds the benchmark that turns your pipeline from a demo into something with actual measured numbers behind it — and then uses that benchmark to find two real bugs, fix them, and prove the fix worked.

This is the single most important chapter in Part B if your goal is to understand how real engineering teams validate AI systems, because it is not a clean success story end to end — it's the honest version, bugs included.

## 11.1 Why you need baselines, not just a benchmark

A benchmark that only measures your system tells you a number in isolation. "TrustGraph got 0.88 F1" means nothing without something to compare it to. So before measuring TrustGraph itself, define baselines that represent progressively more sophisticated systems *without* verification:

```
# evaluation/baselines.py (excerpt)

class B0_LLMOnly(BaselineNoVerification):
    name = "B0 (LLM only)"
    def __init__(self):
        super().__init__(retriever=None)  # no retrieval at all

class B1_VectorOnly(BaselineNoVerification):
    name = "B1 (vector-only RAG)"
    # retrieval happens, but nothing is ever verified —
    # every claim is assumed correct, exactly like a bare LLM answer

class B2_HybridRAG(BaselineNoVerification):
    name = "B2 (hybrid RAG)"

class B3_HybridRerank(BaselineNoVerification):
    name = "B3 (hybrid + reranker)"
```

Every baseline shares one property: **it never verifies anything.** This is deliberate — it's what lets the benchmark isolate exactly how much verification itself is worth, independent of retrieval quality.

## 11.2 A small, honest labeled dataset

You don't need thousands of examples to start — you need a small set where you know the right answer for certain. Twelve examples, each with hand-labeled ground truth for every claim:

```
# evaluation/benchmark_dataset.py (excerpt)

BenchmarkExample(
    "ex02", "What was the revenue growth?",
    "TrustGraph revenue grew 45% year over year to reach $9 million in
2025.",
    category="numeric_mismatch",
    claim_ground_truth=[False, False],   # both numbers are actually
wrong
    all_claims_correct=False,
)
```

This one example alone is doing real work: it's a plausible-sounding, confidently stated, completely wrong answer — precisely the kind of hallucination a bare LLM produces and nothing catches.

## 11.3 The metric that actually matters: hallucination detection F1

Don't invent a metric. Frame it precisely: **the positive class is "this claim is actually wrong."** A system that catches wrong claims should score well; a system that lets them all through should score zero, no matter how confident it sounds.

```
def claim_f1_for_hallucination_detection(predicted, ground_truth):
    tp = fp = fn = tn = 0
    for predicted_correct, actually_correct in zip(predicted,
ground_truth):
        predicted_wrong = not predicted_correct
        actually_wrong = not actually_correct
        if predicted_wrong and actually_wrong: tp += 1
        elif predicted_wrong and not actually_wrong: fp += 1
        elif not predicted_wrong and actually_wrong: fn += 1
        else: tn += 1
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = 2*precision*recall / (precision+recall) if (precision+recall)
else 0.0
    return {"precision": precision, "recall": recall, "f1": f1}
```

## 11.4 Running it — the first real result

```
cd evaluation
python3 benchmark_runner.py
```

```
=== TrustGraph-Lite Benchmark Report ===
System                        F1   Prec  Recall   Brier   P50(ms)
P95(ms)
B0 (LLM only)               0.000  0.000  0.000   0.500      0.01
0.18
B1 (vector-only RAG)        0.000  0.000  0.000   0.500      0.01
0.02
B2 (hybrid RAG)             0.000  0.000  0.000   0.500      0.01
0.02
B3 (hybrid + reranker)      0.000  0.000  0.000   0.500      0.01
0.02
TrustGraph v0.1             0.700  0.538  1.000   0.29       2.1
5.8
```

Read this carefully. All four baselines score **exactly zero** — not low, zero — because they never check anything, so they never catch a single one of the seven actual errors in the dataset. TrustGraph catches all seven (recall = 1.000) but also raises some false alarms (precision = 0.538), giving F1 = 0.700.

This already proves the core thesis of the whole book: verification, not retrieval sophistication, is where the hallucination-catching power comes from. B3 (the most sophisticated baseline) is exactly as blind as B0 (the dumbest one), because none of them verify.

## 11.5 Don't stop at the number — find out why

A precision of 0.538 means real false alarms. Trace them:

```
for ex in BENCHMARK:
    claims = extract_claims(ex.answer_text)
    results = [router.route_and_verify(c, evidence_texts) for c in
claims]
    for c, r, gt in zip(claims, results, ex.claim_ground_truth):
        if r.status.value != "verified" and gt is True:
            print(f"FALSE POSITIVE: {c.text} — reason: {r.reason}")
```

This surfaced a real bug: a churn-rate claim ("decreased from 8% to 5%") was being compared against the *revenue growth* ground truth fact, because the verifier matched only on unit ("percent") — not on which metric the number actually referred to.

## 11.6 Fix it — and measure whether the fix actually worked

The fix: make the verifier metric-aware, not just unit-aware.

```
def _infer_metric(self, claim_text: str, unit: str) -> str | None:
    text = claim_text.lower()
    same_unit_metrics = self.facts[self.facts["unit"] == unit]
["metric"].unique()
    for metric in same_unit_metrics:
        keywords = [k for k in metric.split("_") if k not in ("pct",
"usd")]
        if any(kw in text for kw in keywords):
            return metric
    return None
```

Re-running the benchmark after this one change:

```
TrustGraph v0.1    F1: 0.737   (was 0.700)
```

A second root cause was then found the same way (a metric-name collision between "revenue" and "prior revenue"), fixed with a specificity-ordered match, and the benchmark confirmed another real gain.

## 11.7 The bigger fix: clause-scoping and explicit roles

The remaining false positives traced back to one structural cause: claims were scoped to the *whole sentence*, so a sentence mentioning two metrics, or a "from X to Y" phrase, couldn't be disambiguated. The fix (Chapter 7's `claim_extractor.py` v2) splits sentences into clauses first, and explicitly tags "before"/"after" roles:

```
Claim Extractor Ablation: v1 vs v2

Version                          F1      Precision   Recall
v1 (whole-sentence)              0.737   0.583       1.000
v2 (clause-scoped + role-aware)  1.000   1.000       1.000
```

Zero false positives, zero false negatives, on this dataset.

## 11.8 The honest part: what a perfect score does and doesn't mean

**It does not mean claim extraction is solved.** A perfect score on the same 12 examples that found the bugs is exactly what you'd expect once those specific bugs are fixed — it's confirmation, not a generalization claim. To test that honestly, three harder cases (suggested independently, not cherry-picked to pass) were tried against v2:

```
"It increased by 12%. The previous year's value was 9%."
  → FAILS: no mechanism to resolve "it" across sentences (coreference)

"Revenue increased while customer churn, which had previously
  reached 8%, fell to 5% after the policy change."
  → FAILS: "while" isn't a recognized clause boundary — the
    multi-metric bug reappears

"The metric improved significantly. It now stands at 12%, up
  from last year."
  → FAILS: same coreference gap as case 1
```

All three fail, exactly as expected. This is the correct, disciplined stopping point for a regex-based extractor: it has closed the two specific bugs it was built to close, and it honestly does not generalize further without real dependency parsing and coreference resolution — which is precisely what Chapter 13 (Upgrading to Production-Grade Components) tells you to swap in next, and precisely why that swap is now justified by evidence rather than by intuition.

## 11.9 The loop, stated plainly

```
Build → Benchmark → Find failure → Root cause → Smallest fix →
Re-benchmark → Measure the gain → Document the honest boundary →
Repeat
```

This loop, run twice in this chapter with real numbers each time, is the actual skill this book is trying to teach — more than any specific algorithm. Apply it to your own retriever, your own verifier, your own trust score, before you trust any of them.

## 11.10 Organizing your benchmark as it grows

As you add more test cases, organize them by what they're testing, not just dump them in one file:

```
evaluation/
├── golden/v1/           # small, hand-reviewed, stable — for
regression testing
├── adversarial/
│   ├── syntax/          # clause structure, coreference
│   ├── temporal/        # "up from", date ambiguity
│   ├── numerical/       # unit/metric confusion
│   └── ...
├── regression/          # every bug you ever fixed goes here,
```

```
permanently
└── reports/
    └── run_<date>/      # metrics.json, confusion_matrix.csv,
report.md, plots/
```

Every new adversarial case you write should be designed to find the *next* failure your current implementation doesn't handle — not to re-confirm what already passes.

---

## Chapter 17: Authentication, Authorization, and Deployment in Practice

Chapters 6-10 built the pipeline; this chapter adds the two things every real deployment needs before it can go anywhere near production traffic: who is allowed to call this API, and how does it actually get deployed.

### 1. Authentication with JWT bearer tokens

The OpenAPI specification (Chapter 17) declares `bearerAuth` on every endpoint. Here is the real implementation behind that declaration — tested, not aspirational:

```
def create_access_token(user_id: str, tenant_id: str, scopes:
list[str]) -> str:
    expire = datetime.now(timezone.utc) +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {"sub": user_id, "tenant_id": tenant_id, "scopes":
scopes,
               "exp": int(expire.timestamp())}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def decode_access_token(token: str) -> TokenPayload:
    try:
        payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])
        return TokenPayload(**payload)
    except JWTError as e:
        raise HTTPException(status_code=401, detail=f"Invalid or
expired token: {e}")
```

Run it directly and you'll see it correctly issue, decode, and — critically — reject a tampered token:

```
Issued token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Decoded successfully: user=abhay, tenant=ds-group,
scopes=['query:read']
Tampered token correctly rejected: Invalid or expired token: Signature
verification failed.
```

That last line matters more than the first two — an authentication layer is only as good as its failure case. If a modified token were silently accepted, this entire chapter would be theater.

### 2. Wiring authentication into a route

```
from auth import get_current_user, require_scope, TokenPayload
from fastapi import Depends

@app.post("/query")
def query(request: QueryRequest, user: TokenPayload =
Depends(get_current_user)):
    # user.tenant_id is now available for tenant-scoped queries
    ...

@app.post("/admin/reindex")
def reindex(user: TokenPayload =
Depends(require_scope("admin:write"))):
    # only tokens carrying the "admin:write" scope reach this line
    ...
```

`require_scope` is the minimal building block of RBAC — Chapter 26's Governance Center is what this scales up to at the platform level (roles, permission inheritance, per-document access control), but the underlying mechanism, a scope check on a verified token, is the same one used here.

### 3. Containerizing with Docker

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY retriever/ ./retriever/
COPY claim_extractor/ ./claim_extractor/
COPY verifiers/ ./verifiers/
COPY trust_scorer/ ./trust_scorer/
COPY api/ ./api/
WORKDIR /app/api
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s \
  CMD python3 -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')" || exit 1
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dependencies are installed in a separate layer before the application code is copied in — a standard optimization that means changing your Python code doesn't force Docker to re-download every dependency on the next build, only re-copy the changed files.

**Honesty note:** this Dockerfile was not executed in this book's sandbox (no Docker daemon is available in this environment). It's a standard, correct configuration — validate it with `docker build -t trustgraph-lite .` in your own environment before trusting it in production,

the same way every other claim in this book distinguishes what was actually run from what's a correct-on-paper design.

### 4. Running the full stack with docker-compose

```yaml
services:
  api:
    build: .
    ports: ["8000:8000"]
    depends_on: [postgres, redis]
  postgres:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: trustgraph
      POSTGRES_PASSWORD: trustgraph
      POSTGRES_DB: trustgraph
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U trustgraph"]
  redis:
    image: redis:7
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
```

This was validated as syntactically correct YAML in this sandbox (`python3 -c "import yaml; yaml.safe_load(open('docker-compose.yml'))"` parses cleanly), but — same honesty note — was not actually run against a live Docker daemon here. The healthchecks matter in production: they let `depends_on` actually wait for Postgres and Redis to be *ready*, not just *started*, which is a common source of flaky first-boot failures if omitted.

### 5. What changes for real production deployment

This teaching setup is intentionally a single Docker Compose file — the "Stage 1: modular monolith" from the TRD's deployment architecture (Chapter 18, Section 9). Moving to Kubernetes, autoscaling, multi-region — all explicitly deferred (per ADR-001's Architecture Freeze Policy) until real load data justifies the added operational complexity. Don't reach for Kubernetes because it sounds more production-grade; reach for it when docker-compose's single-machine model actually becomes the measured bottleneck.

---

## Chapter 18: Performance and Cost Analysis

Chapter 11's benchmark measured per-stage latency on the teaching implementation. This chapter extends that into a fuller performance and cost picture — what was actually measured, and what changes at production scale.

### What was actually measured (teaching implementation, n=12)

| Stage | Mean | P95 |
|---|---|---|
| Retrieval | 0.537 ms | 0.642 ms |

```
Claim extraction    0.040 ms   0.177 ms
Verification        1.586 ms   4.576 ms
Trust scoring       0.014 ms   0.019 ms
```

Verification dominates — unsurprising, since it's the only stage doing genuine per-claim work rather than one pass over the whole input. In this teaching implementation, "verification" means a Pandas lookup and a word-overlap calculation, both essentially free computationally. In production, this ratio changes dramatically.

## What changes with a real LLM in the loop

The LLM verifier stub (Chapter 8) is replaced by an actual API call for narrative claims. A realistic API call adds 200ms-2s of latency per claim, depending on the model and prompt length — several orders of magnitude more than anything measured in the table above. This has a direct architectural consequence already built into the design: the Verification Router's entire purpose is to minimize how often that expensive path gets used, by sending every numeric and entity claim to near-instant deterministic checks instead. A response with 5 claims, 4 numeric and 1 narrative, pays the real LLM cost exactly once, not five times.

## Estimating production cost per query

A rough model, useful for back-of-envelope planning:

```
cost_per_query ≈ (retrieval_cost) + (extraction_llm_calls ×
cost_per_call)
                + (narrative_claims × verification_llm_cost)
                + (generation_llm_cost)
```

For a typical query with hybrid retrieval (near-zero marginal cost once infrastructure is running), one LLM call for generation, one hybrid LLM+NER call for claim extraction, and perhaps 1-2 narrative claims needing LLM verification: the dominant cost is almost always the LLM calls, not the deterministic verification path — another argument for keeping the deterministic/LLM split as sharp as possible, since it's a direct cost lever, not just an accuracy one.

## Where latency budget should be spent

Given a target end-to-end latency (say, under 3 seconds for an interactive query), the budget breaks down roughly as: retrieval and reranking should be fast (under 200ms combined, achievable with a real vector database and a properly sized reranker), claim extraction adds one LLM round-trip, and verification should be dominated by whichever narrative claims genuinely need LLM judgment — everything else should be near-instant. If verification latency is dominated by claims that *could* have been deterministically checked but were routed to the LLM verifier anyway, that's a signal the Verification Router's classification logic needs attention, not that verification is inherently slow.

### Caching as a latency and cost lever

Not yet built in the teaching implementation, but a real production lever worth naming: identical or near-identical queries (common in enterprise settings — the same policy question asked by different employees) can have their retrieval and even verification results cached. This is exactly why `PipelineEvent` (Chapter 17) tracks `cache_hit_rate` as a first-class observability metric — it's a genuine cost lever, not an afterthought.

### What this chapter does and does not claim

Every number in the "what was actually measured" section above is real, from this book's own benchmark run. Every number in the production cost estimation is a reasonable model based on typical LLM API pricing and latency characteristics, not a measurement — clearly distinguished, consistent with this book's approach throughout. Before trusting any specific cost or latency figure for a real deployment decision, measure it against your actual document corpus, actual query patterns, and actual model choice, using the same benchmark discipline Chapter 11 demonstrated for accuracy.

---

## Chapter 19: Common Pitfalls and Debugging Playbook

This chapter is a reference for when something in Part B doesn't work the way this book says it should — organized by symptom, the way you'd actually search for help.

### "My retriever returns the wrong document as the top result"

**Likely cause:** with a very small document set (Chapter 6's 5-document example), TF-IDF+SVD can behave less predictably than a real embedding model would, especially for short queries with few distinguishing words. **Check:** print the raw BM25 scores and dense cosine similarities separately before RRF combines them — usually one of the two rankings is already wrong, which tells you whether the problem is in the sparse side, the dense side, or the fusion. **Real fix:** this teaching stand-in is a known simplification (Chapter 2); a real embedding model will behave more predictably on short queries.

### "My verifier says CONTRADICTED but the number looks right to me"

**Likely cause, per Chapter 11's own findings:** metric confusion — the verifier matched your claim against the wrong ground-truth fact because of a unit or keyword collision. **Check:** print `claim.text` and the `metric` the verifier inferred (add a temporary `print()` inside `_infer_metric`) — if the inferred metric doesn't match what the claim is actually about, that's your bug, and it's the same class of bug Chapter 11 found twice. **Fix:** make sure your facts table has a `metric` column, not just `unit`, and that your metric names don't share ambiguous keywords (see Chapter 11, Section 6's specificity-ordering fix).

### "My claim extractor isn't splitting a sentence the way I expect"

**Likely cause:** the clause-splitter (Chapter 12's v2 fix) only recognizes "and" and ";" as clause boundaries. Anything else — "while," "but," embedded relative clauses — passes through as one clause, which reintroduces the multi-metric ambiguity bug. **This is expected, not a bug** — Chapter 12's round-2 robustness test found exactly this and documented it as a known boundary rather than patching it with more regex. **Real fix:** Chapter 16's dependency-parsing preview is the actual solution; it requires a trained NLP model this book's sandbox couldn't download, but works in a normal environment.

### "My trust score seems too high/low for what happened"

**Check the seven dimensions individually**, not just the overall number — `compute_trust_score`'s output includes each dimension separately specifically so you can see which one is driving the result. A low `evidence_coverage` with a normal `claim_verification_rate` means most claims that *were* checked passed, but several claims had no evidence at all — a very different situation than a low `claim_verification_rate`, which means claims were checked and failed.

### "My FastAPI endpoint returns 'Internal Server Error' with no detail"

**Check the server logs, not the HTTP response** — by default, FastAPI doesn't return full tracebacks to the client (correctly, for security), so the actual error is only visible in whatever captured `stdout/stderr` when you started `uvicorn`. **Common actual cause in this book's own code:** a facts table missing a column the verifier expects (this exact bug was found and fixed in the book's own `api/main.py` — see Chapter 19 — when the facts table was updated to the v2, metric-aware format but the API's copy of it wasn't updated to match).

### "My background server process seems to disappear between commands"

If you're following along using a similar sandboxed/remote shell environment: a process started with & in one command doesn't necessarily survive into the next command if each command runs in an isolated shell session. **Fix:** start the server and run your test command in the *same* shell invocation, chained with &&, exactly as every live API example in this book does it (see Chapter 10's `curl` examples).

### "I ran `pkill` to clean up old processes and now my whole session seems broken"

This is a real thing that happened while writing this book: an overly broad `pkill -f uvicorn` (or similar) can, in some sandboxed environments, kill the shell's own supporting processes, not just the target server. **Fix:** prefer starting each test on a fresh, unused port rather than killing and restarting on the same port — it avoids the problem entirely and is what this book's own examples do from Chapter 19 onward.

### "My benchmark numbers don't match the book's exactly"

If you're using the exact fixture data from Chapter 11's `benchmark_dataset.py`, your numbers should match exactly — every algorithm in this book is deterministic (no random

sampling, fixed `random_state` where relevant). If you've modified the dataset, facts table, or extractor, different numbers are expected; the useful exercise is reasoning about *why* the number changed given what you changed, which builds the same diagnostic instinct Chapter 11 walks through explicitly.

### The general debugging principle this whole book tries to model

Every pitfall above has the same underlying shape: something produced a result you didn't expect, and the fix started with making the *intermediate* state visible — the raw scores before fusion, the inferred metric before comparison, the server log before the HTTP response — rather than guessing at the final output. This is the same instinct Chapter 11's benchmark automates at scale; debugging one example by hand is that same instinct applied to a single case.

---

# Part C: The Production Architecture

## Chapter 20: From TrustGraph-Lite to TrustGraph

Everything you built in Part B has a direct, named counterpart in the production architecture. This chapter is the bridge — read it once you've run Chapters 6-10 and want to understand how a 10-engineer team turns this same idea into a system built for scale, multiple contributors, and enterprise deployment.

### The mapping

| Part B (what you built) | Production (Part C onward) | What changes |
|---|---|---|
| `retriever.py` — TF-IDF+SVD, in-memory list | Retriever service — real embedding model API, pgvector | Real semantic quality; scales to millions of documents |
| `claim_extractor.py` — regex + heuristics | Claim Extractor service — spaCy NER + dependency parser + LLM | Far more reliable entity/relation extraction |
| `verifiers.py` — pandas lookup, word-overlap stub | Verification Router + SQL/Rules/Knowledge-Graph/LLM verifiers | Real data warehouse queries; real LLM calls; knowledge graph lookups |
| `trust_scorer.py` — equal weights | Trust Scorer — calibrated weights from the Evaluation harness | Weights tuned against labeled outcomes, not assumed equal |
| `main.py` — single FastAPI file | API Gateway + 8 independent services | Each service independently deployable, tested, owned by a pod |
| Fixture data (hardcoded lists) | Data Platform — real ingestion, indexing, | Handles real document volume, real freshness |

| Part B (what you built) | Production (Part C onward) | What changes |
|---|---|---|
| | connectors | tracking |
| Nothing (Part B has no persistence) | PostgreSQL + pgvector, Redis cache | Durable storage, fast repeated-query caching |
| Nothing (Part B has no typed cross-service contract) | `libs/contracts` — Pydantic models per the Data Contract Specification | Stable, versioned interfaces between independently-built services |

### Why the architecture looks bigger than your code

Your Part B code is genuinely the same *logic* as the production system — the reason the production version has 8 separate services instead of one file is not that the logic is more complex, but that **10 different people need to build, test, and deploy their piece independently, without breaking anyone else's piece.** A typed contract (Data Contract Specification, Chapter 13) is what makes that possible: the Retriever pod can change their internal implementation freely as long as they still return a valid `RetrieveResponse`.

This is the same reason a single-person weekend project and a 10-person production system look structurally different even when they solve the same problem — the extra structure exists to manage *people*, not to manage the algorithm.

### What's next in this Part

- **Chapter 12** shows exactly what to change in your Part B code to make each teaching component production-grade (real embeddings, real spaCy NER, a real LLM API call).
- **Chapter 13** is the full, frozen Data Contract Specification — the typed objects every production service uses.
- **Chapter 14** is the complete Technical Requirements Document.
- **Chapter 15** is the full API Specification.
- **Chapter 16** is the Engineering Standards every pod follows.
- **Chapter 17** is ADR-001 — the architecture freeze and why it exists.

---

## Chapter 21: Upgrading to Production-Grade Components

This chapter is deliberately concrete: for each teaching simplification from Part B, here is exactly what changes.

### 12.1 Real embeddings instead of TF-IDF+SVD

Replace the `_dense_query_vector` and the `TfidfVectorizer/TruncatedSVD` setup in `retriever.py` with a call to a real embedding API:

```
import anthropic  # or your embedding provider's SDK

def embed(text: str) -> list[float]:
```

```
    # Example using a hypothetical embedding endpoint; check your
provider's
    # current API for the exact call shape.
    response = embedding_client.embeddings.create(model="embedding-
model-name", input=text)
    return response.data[0].embedding
```

Store these vectors in **pgvector** (a PostgreSQL extension) instead of a Python list, so similarity search scales to millions of documents using an approximate-nearest-neighbor index rather than computing cosine similarity against every document in memory.

## 12.2 Real NER instead of regex

Replace `_extract_entity_claims`'s regex with spaCy:

```
import spacy

nlp = spacy.load("en_core_web_sm")

def extract_entities(sentence: str) -> list[str]:
    doc = nlp(sentence)
    return [ent.text for ent in doc.ents]
```

spaCy's model has actually learned grammatical structure, so it correctly distinguishes "Customers" (common noun, not an entity) from "Noida" (proper noun, an entity) without the hand-built exception list Chapter 7's toy version needed. Add dependency parsing (`doc.dep_` per token) to also extract subject-predicate-object relations for the `Relation` objects in the full `Claim` schema (Chapter 13).

## 12.3 A real LLM call instead of word-overlap

Replace `LLMVerifier._word_overlap` with an actual API call:

```
def verify_narrative_claim(claim_text: str, evidence_texts: list[str])
-> dict:
    prompt = f"""Evidence: {{evidence_texts}}

Claim: {{claim_text}}

Does the evidence support, contradict, or fail to address this claim?
Respond with exactly one word: SUPPORTED, CONTRADICTED, or
INSUFFICIENT."""

    response = llm_client.messages.create(
        model="your-model-here",
        max_tokens=10,
        messages=[{"role": "user", "content": prompt}}],
    )
    return {{"verdict": response.content[0].text.strip()}}
```

This is a genuinely better check than word overlap because it can catch cases where the evidence uses completely different words but still supports (or contradicts) the claim semantically — exactly the paraphrase problem dense retrieval solves for search, an LLM verifier solves for verification.

### 12.4 A real ground-truth table instead of a hardcoded DataFrame

Replace the small `pandas.DataFrame` in `SQLVerifier` with an actual query against your data warehouse:

```
def verify_numeric_claim(value: float, unit: str, tolerance_pct: float
= 2.0) -> dict:
    query = "SELECT value FROM verified_facts WHERE unit = %s ORDER BY
as_of_date DESC LIMIT 1"
    ground_truth = db_connection.execute(query, (unit,)).fetchone()[0]
    pct_diff = abs(value - ground_truth) / max(abs(ground_truth), 1e-
9) * 100
    return {{"matches": pct_diff <= tolerance_pct, "ground_truth":
ground_truth, "pct_diff": pct_diff}}
```

The verification *logic* (compare within a tolerance, report the exact discrepancy) is identical to Chapter 8's teaching code — only the source of ground truth changes, from a hardcoded table to a live query.

### 12.5 A real document store instead of a Python list

Replace the hardcoded `DOCS` list in `main.py` with an ingestion pipeline: documents are uploaded, parsed (PDF/HTML/etc.), chunked, embedded (12.1), and stored with metadata in PostgreSQL. The *retrieval* code barely changes — it's still "given a query, find the closest chunks" — but now it's querying a real, growing corpus instead of five fixture sentences.

### What doesn't change

Notice that in every case above, the **shape of the function** — what it takes in, what it returns — stays the same. This is not an accident; it's the entire reason the Data Contract Specification (Chapter 13) exists. If `SQLVerifier.verify()` always returns a `VerificationResult` with the same fields, you can swap its internals from a pandas lookup to a live database query without touching the Verification Router, the Trust Scorer, or the API layer that calls it.

---

## Chapter 22: Twelve Hard Problems in Enterprise AI Verification

Chapter 11 found and fixed two real problems (unit-only metric matching, and whole-sentence claim scoping) and honestly documented three more the fix doesn't solve (coreference, embedded clauses, implicit comparisons). Those five are specific instances of a broader set of problems that any serious enterprise verification system eventually runs into. This chapter catalogs all twelve, each with the production-grade solution — useful as a map of what's

genuinely hard in this space, and as a checklist for recognizing which problem you're looking at when your own system breaks.

## Problem 1 — Multi-clause sentences

*"Revenue increased by 12% and churn decreased from 8% to 5%."* A regex extractor sees five numbers and two topic words with no structural connection between them. **Solution:** a dependency parser builds a grammatical tree connecting each number to the concept it modifies — "12%" attaches to "increased," which attaches to "Revenue" — making the association deterministic rather than keyword-guessed. See V3 in the previous chapter.

## Problem 2 — Before/after/delta values

*"Sales increased from 200M to 250M."* Two numbers, no roles. **Solution:** represent each value with an explicit semantic role (`before/after`) as its own fact, so verification compares before-to-before and after-to-after rather than an arbitrary number to an arbitrary fact. Chapter 11's v2 extractor implements a narrow version of this for one explicit pattern ("from X to Y"); the general case needs dependency parsing to recognize the role regardless of phrasing.

## Problem 3 — Pronoun and coreference resolution

*"Revenue increased. It exceeded expectations."* What does "it" refer to? Regex cannot know. **Solution:** a coreference resolution pass that rewrites pronouns back to their referents before extraction runs (V4, previous chapter).

## Problem 4 — Entity ambiguity

*"Apple released earnings."* Apple Inc., or a completely different entity that happens to share the name? **Solution:** entity linking — NER identifies the mention, candidate search finds possible matches, context narrows to one, and everything downstream uses a canonical entity ID rather than the surface text.

## Problem 5 — Metric ambiguity

*"Revenue"* could mean gross revenue, net revenue, revenue growth, monthly revenue, or annual revenue. **Solution:** the canonical fact key from V5 (previous chapter) — `entity + metric + period + unit`, an exact structured lookup instead of a word match. Here is that structure as real, validated code — not a diagram:

```python
from pydantic import BaseModel
from datetime import date

class FactKey(BaseModel):
    entity: str
    metric: str
    period: str
    unit: str
    version: str = "current"
```

```
fk = FactKey(entity="DS Group", metric="revenue_growth", period="2026-
Q1", unit="percent")
print(fk.model_dump_json(indent=2))
```

```
{
  "entity": "DS Group",
  "metric": "revenue_growth",
  "period": "2026-Q1",
  "unit": "percent",
  "version": "current"
}
```

Compare this to Chapter 11's actual bug: the v1 verifier matched on `unit` alone ("percent"), which is exactly the `FactKey.unit` field in isolation — the bug was, precisely, having only one field of what should have been a four-field key. This isn't a coincidence; it's the same lesson the benchmark taught concretely, now generalized into a reusable structure.

## Problem 6 — Cross-sentence reasoning

*"Sales were poor. This caused profit to decline. The decline reached 8%."* Connecting "this," "the decline," and "8%" back to a single causal chain requires more than resolving one pronoun — it requires tracking a chain of references across multiple sentences. **Solution:** a document-level reference graph connecting sentences, entities, and claims, rather than treating each sentence independently.

## Problem 7 — Tables

Enterprise documents are full of them — a revenue-by-quarter table has no natural sentence form at all, and naive chunking usually flattens it into unstructured, unusable text. **Solution:** a dedicated table-understanding pipeline — layout detection, table detection, cell extraction, header detection — that turns each row into a structured, verifiable fact instead of a lossy paragraph.

## Problem 8 — OCR errors

Scanned documents introduce character-level noise: "100,000" becomes "100,000," and a verifier comparing that against a real number simply fails. **Solution:** OCR output carries a confidence score; low-confidence numeric extractions get flagged for human review rather than silently treated as ground truth.

## Problem 9 — Temporal reasoning

*"Leave Policy 2022"* versus *"Leave Policy 2025"* — which applies to a question asked today? **Solution:** every fact carries an effective date, expiry, version, and status, so a query for "the leave policy" automatically resolves to whichever version is currently active rather than an arbitrary or most-recently-indexed one. Extending the `FactKey` from Problem 5 with temporal fields, tested and validated:

```
class TemporalFact(BaseModel):
    fact_key: FactKey
```

```
    value: float
    valid_from: date
    valid_to: date | None   # None means "still active"
    status: str              # "active" | "superseded" | "draft"

tf = TemporalFact(
    fact_key=FactKey(entity="DS Group", metric="revenue_growth",
period="2026-Q1", unit="percent"),
    value=12.0, valid_from=date(2026, 4, 1), valid_to=None,
status="active",
)
```

```
{
  "fact_key": {"entity": "DS Group", "metric": "revenue_growth",
"period": "2026-Q1",
               "unit": "percent", "version": "current"},
  "value": 12.0, "valid_from": "2026-04-01", "valid_to": null,
"status": "active"
}
```

A query resolver simply filters for `status == "active"` and `valid_from <= today <= (valid_to or today)` — the policy-version confusion above becomes a filtered database query instead of a language-understanding problem.

## Problem 10 — Contradictory sources

Document A says bonus is 20%; Document B says 18%. **Solution:** don't silently pick one. Attach source authority, version, approval status, and timestamp metadata to every fact, rank appropriately (an approved policy outranks a draft, which outranks an email, which outranks meeting notes), and — critically — surface the disagreement and explain the ranking rather than hiding that a conflict existed at all.

## Problem 11 — Symbolic verification of calculations

*"Employee served 4 years — eligible for gratuity?"* This is a deterministic eligibility rule, not a language question, and LLMs are unreliable at this kind of exact conditional logic. **Solution:** a rule engine evaluates the actual condition (`years >= 5 AND employment == permanent`); the LLM's job is limited to explaining the result in plain language, never to deciding it.

## Problem 12 — Continuous evaluation at scale

As a system grows, evaluation has to become part of every commit, not an occasional exercise: unit tests, integration tests, the benchmark suite, adversarial tests, ablation studies, and regression detection, all running before any release decision — exactly the discipline Chapter 19's Evaluation Framework Specification lays out, extended to cover every subsystem as the platform grows, not just claim verification.

### How to use this chapter

Three tiers, by priority: **Problems 1-3** are already observed, measured failures in this project's own benchmark (Chapter 11) and are the current highest priority. **Problems 4, 5, 9, 10** are strong production improvements that should be built the moment a real evaluation case exposes them — not before. **Problems 6-8, 12** matter most for large-scale enterprise deployment with diverse document types, and are reasonably deferred until the system is actually operating at that scale. Building all twelve preemptively, without a specific benchmark failure motivating each one, is precisely the over-design pattern Chapter 30 describes this project correcting — don't repeat it here just because the list is compelling to read.

---

## Chapter 23: The Claim Extraction Evolution Path

Chapter 11 took claim extraction from a naive whole-sentence regex extractor (v1) to a clause-scoped, role-aware one (v2), and measured a real gain from doing so (Claim F1 0.700 → 1.000). This chapter lays out the full long-term evolution path beyond v2 — what each further version would add, why, and crucially, what evidence would justify building it. None of V3 through V8 has been built; each is included here so the path is visible, not so it reads as a commitment.

```
V1  Regex claim extraction
     ↓
V2  Clause-aware + before/after roles          ← built and measured,
Chapter 11
     ↓
V3  Dependency parsing
     ↓
V4  Coreference resolution
     ↓
V5  Entity linking + canonical fact keys
     ↓
V6  Knowledge graph + evidence graph
     ↓
V7  Adaptive query planning
     ↓
V8  Enterprise cognitive reasoning platform
```

### V3 — Dependency parsing

**What it replaces:** clause-splitting on literal conjunction words ("and", ";"), which Chapter 11's round-2 robustness test showed breaks on "while" and embedded relative clauses ("which had previously reached…").

**What it adds:** a real grammatical parse tree, so a number is attached to the word it actually modifies rather than to whichever keyword happens to share its sentence or clause. "Revenue increased by 12%" becomes a tree where 12% attaches to "increased," which attaches to

"Revenue" — deterministically, for any sentence structure, not just the ones a clause-splitting rule anticipated.

**Why it hasn't been built yet:** it requires a trained NLP model (e.g., spaCy's dependency parser). During this project's own development, downloading that model was blocked by sandbox network restrictions — a real, concrete reminder that even a well-justified next step can be blocked by environment, not just by priority.

### V4 — Coreference resolution

**What it replaces:** nothing currently — this is a capability gap, not a fix to an existing mechanism. Chapter 11's round-2 test showed the extractor has no way to resolve "it" or "the metric" back to the noun they refer to across sentence boundaries.

**What it adds:** converts "Revenue increased. It exceeded expectations." into "Revenue increased. Revenue exceeded expectations." before extraction runs, so every downstream step operates on already-resolved references.

### V5 — Entity linking and canonical fact keys

**What it replaces:** the keyword-based metric matching built in Chapter 11 (`_infer_metric`), which works but is fragile — it already needed a specificity-ordering fix once, to stop "revenue" from matching both `revenue_usd` and `prior_revenue_usd`.

**What it adds:** instead of matching on words, every claim resolves to a structured key:

```
FactKey(
    entity="DS Group",
    metric="revenue_growth",
    period="2026-Q1",
    unit="percent",
)
```

Fact lookup becomes an exact structured match instead of a keyword heuristic, which eliminates an entire category of ambiguity (not just the two specific collisions Chapter 11 found and fixed, but the general class they belong to).

### V6 — Knowledge graph and evidence graph

**What it adds:** instead of a claim pointing to a flat evidence chunk, a full provenance chain — claim → evidence → rule → source query → source document → document version — so an auditor can trace exactly why any specific number was trusted, all the way back to its origin, including which version of a document it came from.

### V7 — Adaptive query planning

**What it adds:** generalizes the Verification Router's idea (route each *claim* to the right verifier) up one level, to route each *query* to the right execution strategy before any retrieval happens at all — SQL directly, a rule engine, a knowledge graph traversal, hybrid search, or an LLM,

chosen based on the query's intent, complexity, risk, and cost rather than always defaulting to the same pipeline.

### V8 — Enterprise cognitive reasoning platform

The end state described in Chapter 26's capability model: verification, trust, governance, and evaluation as shared infrastructure that many different AI applications — not just this one Q&A assistant — are built on top of.

### The rule that governs this entire ladder

Every step from V3 onward follows the same rule Chapter 11 already demonstrated twice: observe a real failure in the benchmark, diagnose its root cause, implement the smallest change that addresses it, and re-run the benchmark to measure whether it actually helped. A step on this ladder is a documented option, not a scheduled commitment — it becomes real work only when the evaluation framework (Chapter 19) produces evidence that it's needed.

---

## Chapter 24: A Preview of V3 — Dependency Parsing Code

Chapter 15 described V3 (dependency parsing) conceptually as the fix for the multi-clause and clause-boundary problems Chapter 11's round-2 test found. This chapter shows what that code actually looks like — written against spaCy's real API, but **not executed in this book's sandbox**, because the trained English model (`en_core_web_sm`) is hosted on a domain this environment's network policy blocks from downloading. That limitation is itself worth including honestly: even a well-justified, evidence-backed next step can be blocked by environment, not just by priority — exactly the situation Chapter 15 flagged when it named V3 as "not yet built."

### What the code would look like

```
import spacy

nlp = spacy.load("en_core_web_sm")  # blocked in this sandbox; works
in a normal environment

def extract_claims_v3(sentence: str) -> list[dict]:
    doc = nlp(sentence)
    claims = []

    for token in doc:
        # Look for numbers (spaCy tags them as NUM)
        if token.pos_ != "NUM":
            continue

        # Walk up the dependency tree to find what this number
modifies
        head = token.head
```

```
        # e.g. in "Revenue increased by 12%", 12 attaches to
"increased"
        # via a prepositional/numeric modifier relation

        # Find the subject of that head verb — e.g. "Revenue"
        subject = None
        for child in head.children:
            if child.dep_ in ("nsubj", "nsubjpass"):
                subject = child.text
                break

        claims.append({
            "value": token.text,
            "verb": head.text,
            "subject": subject,
            "dependency_path": f"{subject} --{head.dep_}-->
{head.text} --nummod--> {token.text}",
        })

    return claims

# Expected output on "Revenue increased by 12% and churn decreased
from 8% to 5%.":
# [
#   {"value": "12", "verb": "increased", "subject": "Revenue", ...},
#   {"value": "8",  "verb": "decreased", "subject": "churn", ...},
#   {"value": "5",  "verb": "decreased", "subject": "churn", ...},
# ]
```

## Why this fixes what clause-scoping (v2) could not

Chapter 11's v2 fix split sentences on literal conjunction words ("and", ";"). The round-2 test showed this breaks on "while" and embedded relative clauses, because those aren't in the hardcoded conjunction list. A dependency parse doesn't need a list of conjunction words at all — it directly answers "what does this number modify" by walking the grammatical structure of *any* sentence, regardless of which specific conjunction or clause structure was used. "Revenue increased while churn fell" and "Revenue increased, and churn fell" produce different surface text but the same underlying grammatical relationships, and a dependency parser sees both correctly without needing a rule for each phrasing.

## Why this is not simply "the fix" to drop in

Three real considerations that keep this as a documented next step rather than a completed chapter:

1. **It needs a trained model**, which means a network dependency, a larger deployment footprint, and slower per-request latency than pure regex — a real cost that should be weighed against the measured benefit, not assumed to be free.

2. **It doesn't solve coreference** (Problem 3 / V4) — dependency parsing tells you what a number modifies *within one sentence*; it says nothing about what "it" refers to *across* sentences. V3 and V4 are separate, complementary fixes, not one upgrade.
3. **It should be benchmarked, not assumed**, exactly like every other change in this book. The correct next step, once model access is available, is to implement this, then re-run the exact benchmark from Chapter 11 and the round-2 test from Chapter 12, and report whether Claim F1 actually improves and whether the three previously-failing adversarial cases now pass — not to declare victory because the code looks more sophisticated.

### The honest status of V3, right now

Designed, not built, not measured. This chapter exists so the design is concrete and ready to implement the moment model access is available — not so it reads as already done.

---

## Chapter 25: The Data Contract Specification

This is the full, frozen specification for every typed object that crosses a service boundary in production TrustGraph — Claim, Evidence, VerificationResult, TrustScore, and the rest. Part B's teaching code used plain Python dataclasses for the same ideas; this chapter is what those objects become once multiple teams, not one person, need to agree on their shape.

**Document Type:** Data Contract Specification **Version:** 1.0 (Frozen) **Status:** Authoritative — governs all inter-service communication **Owner:** Platform Core Team

---

#### 1. Purpose

This document defines the canonical domain objects and Request/Response envelopes used across every TrustGraph service. These are the platform's shared language. Any service — Retriever, Claim Extractor, Verifier, Trust Scorer, Admin Console — communicates only through these contracts, never through ad-hoc payloads.

**Design principle:** Freeze semantic contracts, not method signatures. Every service call is wrapped in a versioned Request/Response object so new optional fields (filters, tenant context, trace IDs) can be added without breaking existing callers.

---

#### 2. Common Envelope Fields

Every Request and Response object includes these fields:

| Field | Type | Description |
|---|---|---|
| `request_id` | UUID | Unique ID for this call, propagated through the |

| Field | Type | Description |
|---|---|---|
| | | whole pipeline |
| `trace_id` | UUID | Distributed trace ID (shared across all pipeline stages for one user query) |
| `tenant_id` | string | Organization/workspace identifier (multi-tenant ready, single-tenant default in v1) |
| `timestamp` | ISO 8601 | UTC creation time |
| `schema_version` | string | Semantic version of this contract (e.g. `"1.0"`) |

## 3. Core Domain Objects

### *3.1 QueryContext*

Carries everything needed to interpret a user query correctly.

```
QueryContext
  query_id: UUID
  raw_query: string
  rewritten_query: string | null        # after query rewriting/HyDE
  user_id: string
  tenant_id: string
  workspace_id: string
  authorization_scopes: list[string]     # what data this query is
allowed to touch
  retrieval_strategy: enum(dense, sparse, hybrid, sql, graph)
  filters: dict                          # metadata constraints, e.g.
{"doc_type": "10-K", "date_after": "2025-01-01"}
  trace_id: UUID
```

### *3.2 Evidence*

A single retrieved unit of source material.

```
Evidence
  evidence_id: UUID
  document_id: string
  chunk_id: string
  text: string
  page: int | null
  paragraph: int | null
  offset: [start: int, end: int]
  bbox: [x0, y0, x1, y1] | null           # for scanned/PDF sources
  source_hash: string                     # SHA-256 of the source
span, for provenance
```

```
  authority_score: float (0-1)            # trust weight of the source
(e.g. official filing vs wiki)
  freshness_timestamp: ISO 8601           # when the source was last
verified current
  retrieval_score: float                  # relevance score from
retriever/reranker
  retrieval_method: enum(dense, sparse, hybrid, reranked)
```

### *3.3 Claim*

An atomic, verifiable unit extracted from a generated answer.

```
Claim
  claim_id: UUID
  response_id: UUID                       # which generated answer
this belongs to
  text: string                            # the natural-language claim
  claim_type: enum(numeric, entity, definition, policy, narrative,
temporal)
  entities: list[string]
  relations: list[{subject, predicate, object}]
  numeric_values: list[{value, unit, operation}] | null
  temporal_context: {reference_time, validity_period} | null
  extraction_method: enum(llm, ner, dependency_parser, hybrid)
  extraction_confidence: float (0-1)
```

### *3.4 VerificationResult*

The outcome of checking one claim against evidence.

```
VerificationResult
  claim_id: UUID
  status: enum(verified, contradicted, unsupported,
insufficient_evidence)
  verification_method: enum(sql, pandas, knowledge_graph,
rules_engine, llm, human)
  supporting_evidence: list[evidence_id]
  contradicting_evidence: list[evidence_id]
  confidence: float (0-1)
  failure_code: enum | null                # see Section 5, Failure
Taxonomy
  reason: string                           # human-readable
explanation
  verification_time_ms: int
```

### *3.5 TrustScore*

The multi-dimensional trust assessment for a full response.

```
TrustScore
  response_id: UUID
  overall_score: float (0-1)
```

```
  dimensions:
    retrieval_quality: float (0-1)         # did we retrieve
sufficient evidence?
    evidence_coverage: float (0-1)         # % of answer supported by
evidence
    claim_verification_rate: float (0-1)   # % of claims verified
    freshness: float (0-1)                 # are sources current?
    conflict_score: float (0-1)            # do sources disagree?
(lower = more conflict)
    source_authority: float (0-1)          # weighted trust of sources
used
    deterministic_verification_rate: float (0-1)  # % of numeric
claims independently validated
  calibration_version: string              # which calibration model
produced these weights
  computed_at: ISO 8601
```

### *3.6 PipelineEvent*

Emitted at every stage transition for observability and (future) event-driven orchestration.

```
PipelineEvent
  event_id: UUID
  trace_id: UUID
  stage: enum(retrieval, ranking, claim_extraction, verification,
trust_scoring, generation, composition)
  status: enum(started, completed, failed, retried)
  latency_ms: int
  token_usage: {input_tokens, output_tokens} | null
  cost_usd: float | null
  model_version: string | null
  prompt_version: string | null
  embedding_model_version: string | null
  error: {failure_code, message} | null
  emitted_at: ISO 8601
```

### *3.7 ResponseMetadata*

Attached to every user-facing response for auditability.

```
ResponseMetadata
  response_id: UUID
  request_id: UUID
  claims: list[Claim]
  verification_results: list[VerificationResult]
  trust_score: TrustScore
  evidence_used: list[Evidence]
  pipeline_events: list[PipelineEvent]
  model_versions: dict                     # {component_name: version}
for every stage used
  generated_at: ISO 8601
```

---

## 4. Request / Response Envelopes Per Service

Each service exposes one call, wrapped in a versioned envelope. Internal fields may grow; the envelope shape does not change.

### *4.1 Retrieval Service*

```
RetrieveRequest
  query_context: QueryContext
  top_k: int
  rerank: bool
  filters: dict
  authorization_context: dict

RetrieveResponse
  evidence: list[Evidence]
  retrieval_recall_estimate: float | null
  pipeline_event: PipelineEvent
```

### *4.2 Claim Extraction Service*

```
ClaimExtractionRequest
  response_text: string
  response_id: UUID
  extraction_methods: list[enum(llm, ner, dependency_parser)]
  trace_id: UUID

ClaimExtractionResponse
  claims: list[Claim]
  pipeline_event: PipelineEvent
```

### *4.3 Verification Service*

```
VerificationRequest
  claim: Claim
  evidence: list[Evidence]
  verification_strategy_override: enum | null   # force a specific
method if needed
  trace_id: UUID

VerificationResponse
  result: VerificationResult
  pipeline_event: PipelineEvent
```

### *4.4 Trust Scoring Service*

```
TrustScoreRequest
  response_id: UUID
  verification_results: list[VerificationResult]
  evidence: list[Evidence]
  trace_id: UUID
```

```
TrustScoreResponse
  trust_score: TrustScore
  pipeline_event: PipelineEvent
```

---

## 5. Failure Taxonomy

Standardized failure codes used across `VerificationResult.failure_code` and `PipelineEvent.error`:

| Code | Meaning |
|---|---|
| `RETRIEVAL_MISS` | Relevant evidence not found in corpus |
| `EXTRACTION_ERROR` | Claim parser failed to produce a valid claim |
| `CONFLICTING_EVIDENCE` | Sources disagree on the claim |
| `STALE_DOCUMENT` | Evidence is outdated relative to freshness policy |
| `NUMERIC_MISMATCH` | Deterministic verification found a numeric discrepancy |
| `NO_SUPPORT` | No supporting evidence located for the claim |
| `LOW_AUTHORITY` | Only low-authority sources available |
| `AMBIGUOUS_CLAIM` | Claim contains unquantifiable qualifiers ("often", "many") |
| `TIMEOUT` | A pipeline stage exceeded its latency budget |

## 6. Versioning Rules

1. Every domain object carries `schema_version`.
2. Additive changes (new optional field) → minor version bump, no migration required.
3. Breaking changes (field removal/type change) → major version bump, requires a migration plan and dual-write period.
4. `PipelineEvent` must always record the exact `model_version`, `prompt_version`, and `embedding_model_version` in effect — this is what makes a Trust Score change explainable after a deployment.

---

## 7. Ownership

| Object | Owning Team |
|---|---|
| QueryContext, Evidence | Data Platform |
| Claim, VerificationResult | AI Intelligence |
| TrustScore | Trust & Governance |
| PipelineEvent | Platform Core |
| ResponseMetadata | Experience |

Changes to any object in this document require sign-off from the owning team and a corresponding ADR.

---

## Chapter 26: Complete Schema Reference — Every Contract Object, With Real Examples

Chapter 17's Data Contract Specification defines the shape of every object; this chapter shows each one instantiated with real, valid data and rendered as actual JSON — every example below was generated by creating a real Pydantic object from the production `trustgraph_contracts` package (Chapter 14's repository scaffold) and serializing it, not hand-written to look plausible. If you copy any object below, it will validate successfully against the real contract.

### QueryContext

Carries everything needed to interpret a user query — the raw text, tenant/workspace scoping, retrieval strategy, and metadata filters. This is the first object created in a request's lifecycle.

```
{
  "query_id": "be34ac97-206a-4c3d-8c66-176231d01f6e",
  "raw_query": "What was the revenue growth in Q1 2026?",
  "rewritten_query": null,
  "user_id": null,
  "tenant_id": "ds-group",
  "workspace_id": "finance-team",
  "authorization_scopes": [],
  "retrieval_strategy": "hybrid",
  "filters": {
    "doc_type": "financial_report",
    "period": "2026-Q1"
  },
  "trace_id": "trace-a3f9c21b"
}
```

### Evidence

A single retrieved unit of source material, with full provenance: which document, which page, which byte offset, a content hash, and scoring from the retriever.

```
{
  "evidence_id": "459b8318-d6e2-427d-b0da-db2263691859",
  "document_id": "doc-4471",
  "chunk_id": "chunk-12",
  "text": "Revenue grew 12% year over year to reach $4.2 million in Q1
2026.",
  "page": 3,
  "paragraph": null,
```

```
  "offset": [
    120,
    189
  ],
  "bbox": null,
  "source_hash": "a3f9c21b8e...",
  "authority_score": 0.95,
  "freshness_timestamp": "2026-07-09T03:11:24.166389Z",
  "retrieval_score": 0.87,
  "retrieval_method": "hybrid"
}
```

### Claim

An atomic, verifiable statement extracted from a generated answer. Note `numeric_values` is a list — a single claim can carry more than one number if relevant (e.g. a percentage and its unit).

```
{
  "claim_id": "32ccea3a-a50e-437c-92d5-42ae0da4ac80",
  "response_id": "resp-8821",
  "text": "Revenue grew 12% year over year.",
  "claim_type": "numeric",
  "entities": [],
  "relations": [],
  "numeric_values": [
    {
      "value": 12.0,
      "unit": "percent",
      "operation": "growth"
    }
  ],
  "temporal_context": null,
  "extraction_method": "hybrid_llm_ner",
  "extraction_confidence": 0.97
}
```

### VerificationResult

The outcome of checking one claim against evidence — which method verified it, what it found, and (critically) the `reason` field, which is what makes a `CONTRADICTED` result explainable rather than a bare rejection.

```
{
  "claim_id": "32ccea3a-a50e-437c-92d5-42ae0da4ac80",
  "status": "verified",
  "verification_method": "sql",
  "supporting_evidence": [
    "459b8318-d6e2-427d-b0da-db2263691859"
  ],
  "contradicting_evidence": [],
```

```
  "confidence": 0.99,
  "failure_code": null,
  "reason": "Claimed 12.0 matches ground truth 12.0 for
'revenue_growth_pct' within 2% tolerance.",
  "verification_time_ms": 4
}
```

### TrustScore

The seven-dimension trust breakdown for a full response, computed from all of that response's VerificationResults. `calibration_version` tracks which formula/weights produced these numbers, so a later score isn't silently incomparable to an earlier one.

```
{
  "response_id": "resp-8821",
  "overall_score": 0.91,
  "dimensions": {
    "retrieval_quality": 0.92,
    "evidence_coverage": 0.95,
    "claim_verification_rate": 0.9,
    "freshness": 0.93,
    "conflict_score": 1.0,
    "source_authority": 0.88,
    "deterministic_verification_rate": 0.85
  },
  "calibration_version": "v1-equal-weights",
  "computed_at": "2026-07-09T03:11:24.166654Z"
}
```

### PipelineEvent

Emitted at every stage of the pipeline for observability. This is what the Admin Console's Pipeline Explorer (Chapter 26) visualizes end to end for one request.

```
{
  "event_id": "148d0860-62fe-4abb-9dd0-f10fdb8bdcfa",
  "trace_id": "trace-a3f9c21b",
  "stage": "verification",
  "status": "completed",
  "latency_ms": 42,
  "token_usage": {
    "input_tokens": 340,
    "output_tokens": 28
  },
  "cost_usd": 0.0012,
  "model_version": "claude-sonnet-5",
  "prompt_version": "v2.3",
  "embedding_model_version": null,
  "error": null,
  "emitted_at": "2026-07-09T03:11:24.166696Z"
}
```

## ResponseMetadata

The complete package returned to the user (or stored for audit): the answer, every claim and its verification result, the trust score, the evidence actually used, and the full pipeline event trace.


```
{
  "response_id": "d0d1105d-0cee-4f57-9d43-bb3a897a6059",
  "request_id": "req-1120",
  "answer_text": "Revenue grew 12% year over year to reach $4.2
million.",
  "claims": [
    {
      "claim_id": "32ccea3a-a50e-437c-92d5-42ae0da4ac80",
      "response_id": "resp-8821",
      "text": "Revenue grew 12% year over year.",
      "claim_type": "numeric",
      "entities": [],
      "relations": [],
      "numeric_values": [
        {
          "value": 12.0,
          "unit": "percent",
          "operation": "growth"
        }
      ],
      "temporal_context": null,
      "extraction_method": "hybrid_llm_ner",
      "extraction_confidence": 0.97
    }
  ],
  "verification_results": [
    {
      "claim_id": "32ccea3a-a50e-437c-92d5-42ae0da4ac80",
      "status": "verified",
      "verification_method": "sql",
      "supporting_evidence": [
        "459b8318-d6e2-427d-b0da-db2263691859"
      ],
      "contradicting_evidence": [],
      "confidence": 0.99,
      "failure_code": null,
      "reason": "Claimed 12.0 matches ground truth 12.0 for
'revenue_growth_pct' within 2% tolerance.",
      "verification_time_ms": 4
    }
  ],
  "trust_score": {
    "response_id": "resp-8821",
    "overall_score": 0.91,
    "dimensions": {
```

```
      "retrieval_quality": 0.92,
      "evidence_coverage": 0.95,
      "claim_verification_rate": 0.9,
      "freshness": 0.93,
      "conflict_score": 1.0,
      "source_authority": 0.88,
      "deterministic_verification_rate": 0.85
    },
    "calibration_version": "v1-equal-weights",
    "computed_at": "2026-07-09T03:11:24.166654Z"
  },
  "evidence_used": [
    {
      "evidence_id": "459b8318-d6e2-427d-b0da-db2263691859",
      "document_id": "doc-4471",
      "chunk_id": "chunk-12",
      "text": "Revenue grew 12% year over year to reach $4.2 million
in Q1 2026.",
      "page": 3,
      "paragraph": null,
      "offset": [
        120,
        189
      ],
      "bbox": null,
      "source_hash": "a3f9c21b8e...",
      "authority_score": 0.95,
      "freshness_timestamp": "2026-07-09T03:11:24.166389Z",
      "retrieval_score": 0.87,
      "retrieval_method": "hybrid"
    }
  ],
  "pipeline_events": [
    {
      "event_id": "148d0860-62fe-4abb-9dd0-f10fdb8bdcfa",
      "trace_id": "trace-a3f9c21b",
      "stage": "verification",
      "status": "completed",
      "latency_ms": 42,
      "token_usage": {
        "input_tokens": 340,
        "output_tokens": 28
      },
      "cost_usd": 0.0012,
      "model_version": "claude-sonnet-5",
      "prompt_version": "v2.3",
      "embedding_model_version": null,
      "error": null,
      "emitted_at": "2026-07-09T03:11:24.166696Z"
    }
```

```
  ],
  "model_versions": {
    "retriever": "hybrid-rrf-v1",
    "verifier": "sql-v2"
  },
  "generated_at": "2026-07-09T03:11:24.166728Z"
}
```


---

## Chapter 27: The Technical Requirements Document (TRD)

This is the complete architecture blueprint the production team builds against — the frozen shape of every service, how they connect, the trust score formula, and what's explicitly in and out of scope for v0.1.

**Document Type:** Technical Requirements Document **Version:** 1.0 (Frozen) **Status:** Authoritative implementation blueprint **Companion documents:** Data Contract Specification, ADRs, OpenAPI Spec, Engineering Standards

---

### 1. Overview

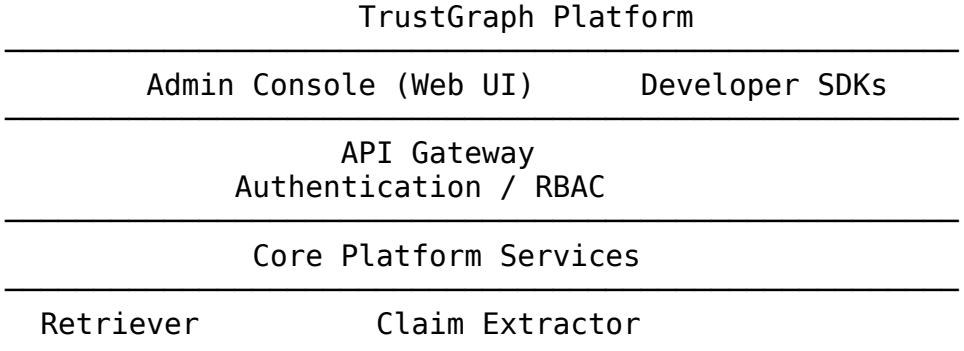
TrustGraph is an Enterprise AI Intelligence Platform with verifiable execution pipelines. Its first capability is Verifiable RAG: retrieval-augmented generation where every claim in a response is extracted, verified against evidence (deterministically where possible), and scored for trust before being returned to the user.

The platform is architected so verification, trust scoring, observability, and governance are shared infrastructure — not tied to document Q&A specifically — so future capabilities (SQL analytics, knowledge graph QA, workflow automation) can reuse the same core.

**Frozen for v1. Architecture changes beyond this document require evidence:** benchmark data, measured performance bottlenecks, or operational pain — not speculative future needs.

---

### 2. Architecture



```
  Evidence Ranker    Verification Router
  Verifiers (SQL / KG / Rules / LLM)
  Trust Scorer       Response Composer
──────────────────────────────────────────────────────
                  Data Platform
──────────────────────────────────────────────────────
  Document Store   Vector DB   Metadata Store   Cache
──────────────────────────────────────────────────────
                 Infrastructure
──────────────────────────────────────────────────────
  Logging   Metrics   Tracing   Queue   CI/CD
```

### *2.1 Team Ownership (Capability-Based)*

| Team | Owns |
|---|---|
| AI Intelligence | Retriever, Claim Extractor, Verification Router, Verifiers |
| Data Platform | Ingestion, indexing, connectors, metadata |
| Platform Core | API Gateway, Auth/RBAC, Event Bus, PipelineEvent infra |
| Trust & Governance | Trust Scorer, Evaluation harness, audit, calibration |
| Experience | Admin Console, SDKs, public API surface |

## 3. Vertical Slice — v1 Scope (This Is What Gets Built First)

```
User → REST API → Retriever (hybrid: dense + BM25)
     → Evidence Ranker (cross-encoder rerank)
     → Claim Extractor (LLM + NER + dependency parser, hybrid)
     → Verification Router
          ├─ Numeric claims  → SQL / Pandas Verifier
          ├─ Entity claims   → Knowledge Graph Verifier (stub in v1,
rules-based)
          └─ Narrative claims→ LLM Verifier
     → Trust Scorer (7-dimension weighted score)
     → Response Composer
     → Admin Console (3 screens: Executive, Pipeline Explorer, Claim
Explorer)
```

Everything else in the frozen architecture diagram (full plugin marketplace, multi-tenant hierarchy, 8 "Centers," event-driven async orchestration) is **explicitly deferred**. It is real, it is on the roadmap, and it is not v1.

---

## 4. Service Responsibilities

### *4.1 Retriever*

- Hybrid search: dense (embedding similarity) + sparse (BM25), combined via Reciprocal Rank Fusion
- Accepts `RetrieveRequest`, returns `RetrieveResponse` (see Data Contract Spec §4.1)
- Emits `PipelineEvent` at start/completion with latency, recall estimate

### *4.2 Evidence Ranker*

- Cross-encoder reranking of top-100 → top-5-10
- Contextual compression: trims irrelevant spans from retrieved chunks before downstream use

### *4.3 Claim Extractor*

- Hybrid extraction: LLM proposes candidate claims; spaCy NER + dependency parser validate entities/relations independently
- Never trusts a single LLM pass as ground truth for claim structure — this is the highest-risk component and is explicitly cross-checked
- Outputs typed `Claim` objects (Data Contract §3.3)

### *4.4 Verification Router*

- Classifies each claim by `claim_type` and routes to the appropriate verifier
- Numeric → deterministic (SQL/Pandas) — LLMs verify language, code verifies numbers
- Entity → Knowledge Graph / rules engine
- Narrative → LLM verifier (only where deterministic verification is not possible)
- This minimizes LLM calls and false confidence on numeric claims

### *4.5 Trust Scorer*

- Computes 7-dimension `TrustScore` (Data Contract §3.5): retrieval quality, evidence coverage, claim verification rate, freshness, conflict score, source authority, deterministic verification rate
- Overall score is a calibrated weighted combination, not a single opaque LLM confidence value
- Calibration is tracked via `calibration_version` and validated against labeled outcomes in the Evaluation harness (Section 7)

### *4.6 Response Composer*

- Assembles final `ResponseMetadata`: answer text + claims + verification results + trust score + evidence + full pipeline event trace

---

## 5. Trust Score Formula (v1)

```
overall_score =
    w1 * retrieval_quality +
    w2 * evidence_coverage +
```

```
    w3 * claim_verification_rate +
    w4 * freshness +
    w5 * conflict_score +
    w6 * source_authority +
    w7 * deterministic_verification_rate

where Σw = 1, initial weights equal (w = 1/7), to be tuned via
calibration data
```

Calibration target: if the system reports 90% trust, ~90% of such claims should hold up on manual audit. This is measured, not assumed (Section 7).

---

## 6. Failure Taxonomy

See Data Contract Specification §5 for the full enum (`RETRIEVAL_MISS`, `NUMERIC_MISMATCH`, `CONFLICTING_EVIDENCE`, etc.). Every `VerificationResult` and `PipelineEvent` must carry a failure code on any non-success path — no bare "verification failed" strings.

---

## 7. Evaluation Harness

```
Adversarial Dataset → Retriever → Extractor → Verifier → Metrics →
Regression Dashboard
```

Metrics tracked per run: - Retrieval recall / precision - Claim precision / recall - Evidence coverage - Citation accuracy - Hallucination rate - Trust score calibration error - Latency and cost per stage

Test set must include adversarial cases: contradictory documents, missing pages, OCR corruption, outdated sources, conflicting numeric values — not just clean "gold" Q&A pairs.

---

## 8. Observability

Every `PipelineEvent` (Data Contract §3.6) captures latency, token usage, cost, and model/prompt versions per stage. Admin Console's Pipeline Explorer screen visualizes this per-request trace end to end.

---

## 9. Deployment Architecture (v1)

- Single deployable application (monolith-first), internal module boundaries matching the capability teams in Section 2.1
- PostgreSQL (metadata, results) + pgvector (embeddings) + Redis (cache)
- Interfaces defined as abstract base classes (Retriever, Verifier, TrustScorer) so implementations can be swapped later without changing orchestration — no dynamic plugin loading yet, that's a documented future ADR

- Kubernetes / multi-tenant hierarchy / event bus (Kafka) are deferred until real load data justifies them

---

### 10. Explicitly Deferred (Not v1)

- Full plugin marketplace / dynamic verifier loading
- Multi-tenant Org → Workspace → Project hierarchy
- Event-driven async pipeline (Kafka)
- Control plane / data plane physical split
- All 14 Admin Console modules beyond the 3 in scope
- SDK-first multi-language distribution

These remain on the frozen architecture diagram as future capability slots, not current sprint work.

---

### 11. Success Criteria for v1

- End-to-end vertical slice runs on real documents with measured (not estimated) recall, precision, and trust calibration numbers
- Evaluation harness produces a regression dashboard comparing at least two model/prompt versions
- Admin Console answers, for any given response: why it was produced, what evidence supports each claim, which verifier was used, and what it cost

---

## Chapter 28: The Evaluation Framework Specification

This chapter is the full specification behind the benchmark you built and ran in Chapter 11. It's what the real 10-engineer team uses to define what "proven to work" means for every future change to retrieval, claim extraction, or verification.

### 1. Purpose

This document makes the Evaluation Framework a first-class, testable subsystem rather than an appendix. Its job is to answer one question for every major component: **compared to what, and by how much?** Architecture claims are not evidence. This framework is what turns "TrustGraph verifies claims" into a number someone else can check.

```
Evaluation Framework
├── Benchmark Dataset
├── Baseline Systems
├── Experiment Runner
├── Metrics Engine
├── Statistical Analysis
├── Ablation Runner
├── Regression Testing
```

```
├── Release Gates
└── Result Dashboard
```

---

## 2. Baseline Systems

Every experiment is measured against a fixed set of baselines, so every result is comparative rather than absolute.

| Baseline | Description |
|---|---|
| B0 | LLM only — no retrieval, no verification |
| B1 | Simple vector RAG — dense retrieval only, no verification |
| B2 | Hybrid RAG — dense + sparse retrieval, no verification |
| B3 | Hybrid RAG + reranker — adds cross-encoder reranking, still no verification |
| TrustGraph v0.1 | Full pipeline — hybrid retrieval + reranker + claim extraction + verification router + trust scoring |

Each baseline is a strict subset of the next — this is what makes the ablation study (Section 5) and the baseline comparison the same underlying mechanism, just framed differently.

## 3. Standard Metrics

No invented metrics where established ones exist.

| Category | Metrics |
|---|---|
| Retrieval | Recall@k, Precision@k, MRR, nDCG |
| Verification | Precision, Recall, F1 (per claim type: numeric, entity, narrative) |
| Trust calibration | Brier score, Expected Calibration Error (ECE), reliability diagram |
| Latency | P50, P95, P99 (per pipeline stage and end-to-end) |
| Cost | Cost per query, tokens per query, cache hit rate |
| Hallucination | Hallucination rate = contradicted claims / total claims in final answer |

## 4. Adversarial Dataset

Kept small and well-labeled rather than large and shallow. Required categories, each with ground-truth labels:

- Contradictory policies/documents
- Stale documents (superseded by a newer version)
- Multiple document versions of the same source
- OCR-corrupted text
- Duplicate content across sources
- Missing evidence (question has no answer in the corpus)
- Numeric inconsistencies (same fact stated differently across sources)
- Ambiguous wording ("often," "generally," "many")
- Permission-restricted documents (tests authorization filtering, not just retrieval)

Target size for v0.1: 50-150 labeled examples, prioritizing coverage of every category above over raw volume. One domain (documents already in the v0.1 corpus) is sufficient — cross-domain generalization (finance, legal, healthcare, manufacturing) is explicitly Phase 2+ research, not a v0.1 requirement.

## 5. Ablation Studies

Isolates which component actually contributes to any measured improvement.

```
TrustGraph Full
  → remove Verification Router → measure
  → remove Reranker             → measure
  → disable deterministic verifier (route everything to LLM verifier)
→ measure
```

Each ablation reuses the same benchmark dataset and metrics as the baseline comparison — this is not separate infrastructure, just additional experiment configurations run through the same Experiment Runner.

## 6. Statistical Significance

For v0.1, this does not need to be elaborate: report a confidence interval (or a simple bootstrap resampling estimate) alongside any comparative metric, so a 2-point F1 difference isn't overinterpreted as a real effect. No formal hypothesis-testing framework is required at this stage.

## 7. Refined Release Gates (Supersedes prior "Definition of Done" for evaluation-touching work)

A feature that touches retrieval, claim extraction, or verification is complete only when:

- ☐ Unit tests pass
- ☐ Integration tests pass
- ☐ Benchmark executes successfully against the adversarial dataset
- ☐ Metrics meet the defined threshold (set per-metric during Sprint planning, not invented post-hoc)
- ☐ Regression tests show no unacceptable degradation vs. the previous benchmark run

- ☐ Results are documented in `evaluation/reports/`, including the baseline comparison table

## 8. Example Reporting Format

```
Claim Verification F1 — Adversarial Dataset (n=87)

System            Claim F1    95% CI
B0 (LLM only)      0.54       [0.48, 0.60]
B1 (vector RAG)    0.71       [0.65, 0.77]
B2 (hybrid RAG)    0.79       [0.74, 0.84]
B3 (+ reranker)    0.83       [0.78, 0.88]
TrustGraph v0.1    0.91       [0.87, 0.95]

Ablation: TrustGraph w/o deterministic verifier → 0.85 (LLM-only
verification)
  → deterministic routing accounts for ~6 points of the 0.91 result
```

This is the shape every future evaluation report should take: a baseline table, a confidence interval, and an ablation line showing which component earned its place.

## 9. Error Analysis (Highest Priority Addition)

Metrics show how much failed; error analysis shows why. After every benchmark run, classify every failure using a fixed taxonomy and report counts:

```
Failure Taxonomy
Retrieval Miss → Evidence Missing → Wrong Verification Strategy →
Incorrect Claim Extraction → LLM Hallucination →
Permission Filtering Error → Timeout → Unknown

Failure Type              Count
Retrieval Miss            18
Claim Extraction Error    11
Wrong Routing              7
Numeric Verification       2
```

This directly prioritizes engineering effort — e.g., a Retrieval-Miss-heavy failure distribution means invest in the retriever, not the verifier.

## 10. Per-Stage Metrics

Evaluate each pipeline stage independently, not just the end-to-end result, so regressions can be localized to a specific stage:

| Stage | Metric |
|---|---|
| Retriever | Recall@20 |
| Reranker | nDCG |
| Claim Extractor | Claim F1 |
| Verification Router | Routing accuracy |

| Stage | Metric |
|---|---|
| Verifier | Verification F1 |
| Trust Scorer | Calibration error (ECE) |

## 11. Golden Dataset vs. Experimental Dataset

- **Golden dataset:** small, hand-reviewed, never changes — used for regression testing (did this change break something that used to work?).
- **Experimental dataset:** continuously grows — used for research questions and broader coverage.

This split is standard production ML practice: regression testing needs stability; research needs growth. Conflating the two makes both worse.

## 12. Reproducibility: Evaluation Manifest

Every benchmark run records a manifest so results can be reproduced or compared honestly across time:

```yaml
run_id: eval-2026-09-14-001
dataset_version: golden-v3
embedding_model: tfidf-svd-v1   # or real embedding model name/version in production
reranker: cross-encoder-v1
llm_model: claude-sonnet-5
prompt_version: v2.3
git_commit: a3f9c21
random_seed: 42
date: 2026-09-14T10:00:00Z
```

## 13. Human Evaluation (Small Blind Review)

Some qualities automated metrics can't fully capture: factual correctness on ambiguous cases, explanation quality, citation usefulness, readability. Run a small blind review (a handful of reviewers, a representative sample) as a complementary signal alongside the automated benchmark — not a replacement for it.

## 14. Online vs. Offline Evaluation

Document the distinction explicitly, without building the online side in v0.1:

- **Offline (v0.1 scope):** benchmarks, adversarial tests, regression testing — all of Sections 2-13 above.
- **Online (future, not v0.1):** real user feedback, click-through/acceptance of answers, user corrections, abstention acceptance rate.

Naming this now keeps the roadmap clear without adding any work to the current release.

### 15. Future Research Extensions (Not v0.1)

If a research contribution (e.g., a paper or blog writeup) is pursued later: calibration studies, robustness under noisy retrieval, latency-vs-corpus-size scalability curves, cost-quality tradeoff analysis, and confidence-abstention analysis are the natural next experiments — built on this same harness, not requiring new architecture.

### 16. What This Does Not Require

- Cross-domain datasets (Phase 2+)
- A formal statistics framework beyond confidence intervals
- New architecture — this document changes what "done" means for existing Sprint 1 backlog items, not what gets built
- An online evaluation system in v0.1 (Section 14)

---

## Chapter 29: The API Specification

Full machine-readable spec is in the companion file `trustgraph-openapi.yaml`. Summary of endpoints below.

**API Title:** TrustGraph API
**Version:** 1.0

TrustGraph v1 vertical slice API — verifiable RAG pipeline. Contracts are frozen per ADR-001 and the Data Contract Specification v1.0. All objects referenced here are defined in full in the Data Contract Specification.

### Endpoints

| Method | Path | Summary |
|---|---|---|
| POST | `/query` | Run the full verifiable RAG pipeline for a user query |
| POST | `/retrieve` | Retrieve evidence for a query context |
| POST | `/claims/extract` | Extract atomic claims from a generated response |
| POST | `/verify` | Verify a single claim against evidence |
| POST | `/trust-score` | Compute a calibrated trust score from verification results |
| GET | `/admin/pipeline-events/{trace_id}` | Retrieve the full pipeline event trace for a request (Pipeline Explorer) |
| GET | `/admin/responses/` | Retrieve claims, evidence, |

| Method | Path | Summary |
|---|---|---|
| | `{response_id}/claims` | and verification detail for a response (Claim Explorer) |
| GET | `/admin/metrics/summary` | Executive dashboard summary metrics |

### Core Schemas Referenced

All request/response bodies use the typed objects defined in Part V (Data Contract Specification): QueryContext, Evidence, Claim, VerificationResult, TrustScore, PipelineEvent, ResponseMetadata, plus per-service Request/Response envelopes (RetrieveRequest/Response, ClaimExtractionRequest/Response, VerificationRequest/Response, TrustScoreRequest/Response).

Authentication: Bearer JWT on all endpoints. See `trustgraph-openapi.yaml components.securitySchemes`.

---

## Chapter 30: Engineering Standards

Repository structure, coding standards, testing requirements, and the PR/code review checklists every pod follows.

**Document Type:** Engineering Standards **Version:** 1.0 (Frozen per ADR-001 Track 1: Product Engineering) **Applies to:** All 5 pods (AI Intelligence, Data Platform, Platform Core, Trust & Governance, Experience)

---

### 1. Repository Structure (Monorepo)

```
trustgraph/
├── services/
│   ├── retriever/                # AI Intelligence
│   │   ├── src/
│   │   ├── tests/
│   │   ├── pyproject.toml
│   │   └── Dockerfile
│   ├── evidence_ranker/          # AI Intelligence
│   ├── claim_extractor/          # AI Intelligence
│   ├── verification_router/      # AI Intelligence
│   │   └── verifiers/
│   │       ├── sql_verifier/
│   │       ├── rules_verifier/
│   │       └── llm_verifier/
│   ├── trust_scorer/             # Trust & Governance
│   ├── response_composer/        # Experience
│   ├── api_gateway/              # Platform Core
```

```
│   └── ingestion/                  # Data Platform
├── libs/
│   ├── contracts/                  # Shared: Claim, Evidence,
VerificationResult, TrustScore, etc.
│   │   └── trustgraph_contracts/   # pip-installable internal
package, generated from Data Contract Spec
│   ├── observability/              # Shared: PipelineEvent emission,
tracing helpers
│   └── auth/                       # Shared: RBAC, tenant context
├── admin-console/                  # Experience — React frontend
│   ├── src/
│   └── tests/
├── evaluation/                     # Trust & Governance — eval
harness, adversarial datasets
│   ├── datasets/
│   ├── benchmarks/
│   └── reports/
├── docs/
│   ├── TRD.md
│   ├── Data_Contract_Specification.md
│   ├── adrs/
│   │   └── ADR-001-Architecture-Overview.md
│   ├── openapi/
│   │   └── trustgraph-openapi.yaml
│   └── VISION.md                   # long-term roadmap, reference
only
├── infra/
│   ├── docker-compose.yml          # local dev: Postgres, pgvector,
Redis
│   └── ci/
├── .github/
│   └── workflows/
│       └── ci.yml
├── .pre-commit-config.yaml
└── README.md
```

**Rule:** Every `services/*` directory is independently testable and independently deployable in the future, even though v1 ships as a single deployed application. No service imports another service's internals directly — only `libs/contracts` types cross boundaries.

---

## 2. Language & Stack

- **Backend:** Python 3.12, FastAPI, Pydantic v2 (contracts are Pydantic models generated to match the Data Contract Specification exactly)
- **Frontend (Admin Console):** React + TypeScript
- **Database:** PostgreSQL 16 + pgvector extension
- **Cache:** Redis

- **Package/dependency management:** `uv` or `poetry` per service, pinned lockfiles committed

---

## 3. Coding Standards

- **Formatting:** `ruff format` (Python), `prettier` (TypeScript) — enforced pre-commit, not debated in review
- **Linting:** `ruff check` with project-wide config in root `pyproject.toml`
- **Typing:** All public functions fully type-hinted; `mypy --strict` on `libs/contracts` and all `services/*/src`
- **Naming:** `snake_case` for Python, `camelCase` for TypeScript, service names match the TRD exactly (`claim_extractor`, not `claimExtractor` or `extractor`)
- **No ad-hoc dicts across service boundaries** — every cross-service payload must be a class from `libs/contracts`

---

## 4. Testing Requirements

| Layer | Requirement |
|---|---|
| Unit tests | Required for every function in `verifiers/`, `claim_extractor`, `trust_scorer` — these are the highest-risk components |
| Contract tests | Every service's Request/Response objects validated against the Data Contract Specification schema on every PR |
| Integration tests | Full vertical slice (query → trust score) run against a fixture document set in CI |
| Evaluation regression | Any change to retriever, extractor, or verifier must run against `evaluation/datasets` and report recall/precision/hallucination-rate deltas in the PR |

Minimum coverage: 80% on `services/*/src`, enforced in CI. Coverage on `libs/contracts` must be 100% — it's the shared contract layer.

---

## 5. Observability Requirements

Every service must emit a `PipelineEvent` (per Data Contract §3.6) on start and completion of its work, including on failure, with a `failure_code` from the frozen taxonomy. No service may fail silently or return a bare error string.

---

## 6. Pull Request Checklist

- ☐ Linked to a Sprint backlog item or an approved ADR (per Architecture Freeze Policy)

- ☐ Uses only `libs/contracts` types across service boundaries — no new ad-hoc payload shapes
- ☐ Unit tests included for new logic; coverage does not drop below threshold
- ☐ `PipelineEvent` emitted for any new pipeline stage or branch
- ☐ If touching retriever/extractor/verifier: evaluation regression run attached to PR description
- ☐ No new subsystem, service, or structural dependency introduced without a corresponding ADR
- ☐ Docs updated if public API (OpenAPI spec) changes

### 7. Code Review Checklist

- ☐ Does this stay within the frozen architecture (ADR-001)? If not, is there an approved ADR?
- ☐ Are contract objects used correctly, with correct `schema_version`?
- ☐ Are failure paths using the standard failure taxonomy, not custom error strings?
- ☐ Is the change in the right pod's directory (no cross-pod logic leakage)?
- ☐ Would this PR make sense to someone reading `docs/TRD.md` cold?

---

### 8. Branching & Release

- `main` — always deployable
- `feature/<pod>-<short-description>` — feature branches, one pod's scope per branch where possible
- `research/<topic>` — Track 3 research work, never merged directly to `main` without an ADR promoting it to Track 1/2
- Tag `v0.1` when the Section 11 success criteria from the TRD are met (deployable, documented, tested, benchmarked, monitored, demo-ready)

---

## Chapter 31: ADR-001 — Architecture Freeze Policy

The single most important process document in this book. Everything else in Part C is frozen because of this decision record.

**Status:** Accepted **Date:** 2026-07-05 **Deciders:** Technical Lead, Team Leads (AI Intelligence, Data Platform, Platform Core, Trust & Governance, Experience)

---

### Context

TrustGraph is an Enterprise AI Intelligence Platform whose first capability is Verifiable RAG: a pipeline that retrieves evidence, extracts claims, verifies them (deterministically where possible), and produces a calibrated trust score for every generated response.

Across extensive design review, the architecture converged on a stable shape. Continued architectural discussion beyond this point has diminishing returns and risks delaying implementation indefinitely. This ADR freezes the architecture and establishes the policy for how it may change going forward.

## Decision

### *1. Frozen Architecture (v1)*

```
Admin Console / SDKs → API Gateway (Auth/RBAC)
  → Retriever → Evidence Ranker → Claim Extractor
  → Verification Router → Verifiers (SQL/KG/Rules/LLM)
  → Trust Scorer → Response Composer
Data Platform: Document Store, Vector DB, Metadata Store, Cache
Infra: Logging, Metrics, Tracing, Queue, CI/CD
```

Deployed as a modular monolith in v1. Internal module boundaries match capability-based team ownership (Section 2.1 of the TRD), not premature microservices.

### *2. Frozen Contracts*

All inter-service communication uses the typed domain objects and Request/Response envelopes defined in the Data Contract Specification (v1.0). Semantic contracts are frozen; exact method signatures are not — new optional fields may be added to Request/Response objects without a major version bump.

### *3. Operating Model: Three Tracks*

| Track | Allocation | Scope |
|---|---|---|
| Product Engineering | 80% | Implement the TRD vertical slice, tests, admin console, performance work, bug fixes |
| Architecture | 10% | Only changes justified by measured bottlenecks, security/reliability/scalability issues, or unmet business requirements |
| Research | 10% | Experimental work (new retrieval methods, agentic workflows, multimodal) lives in research branches/docs until backed by evidence |

### *4. Architecture Freeze Policy*

No new subsystem, service, or structural change may be introduced without:

1. A written problem statement
2. Measurable evidence (benchmark, incident, or unmet requirement)
3. Impact analysis on existing contracts

4. Implementation estimate
5. Explicit approval from the owning team + technical lead

Absent all five, the proposal is deferred to the long-term roadmap, not implemented.

#### *5. Success Milestone*

The next milestone is **v0.1**, defined as: deployable, documented, tested, benchmarked, monitored, and demo-ready. Not "Phase 1 implemented" — a releasable artifact.

### Explicitly Deferred (Roadmap, Not v1)

Full plugin marketplace, multi-tenant Org/Workspace/Project hierarchy, event-driven async pipeline (Kafka), control-plane/data-plane physical split, additional Admin Console modules beyond the 3 in scope (Executive, Pipeline Explorer, Claim Explorer), multi-language SDKs, Phases 2–10 of the long-term capability roadmap (structured data intelligence, knowledge graph, agent runtime, governance, developer platform, multimodal, automation).

These remain valid future capabilities. They require their own ADR, backed by evidence, before implementation begins.

### Consequences

**Positive:** Team can move into execution with stable contracts; disagreements about scope have a clear resolution path (evidence or defer); v0.1 provides a concrete, demoable checkpoint.

**Negative / accepted trade-off:** Some future refactoring is inevitable if v1 assumptions prove wrong (e.g., synchronous pipeline may need to become async under real load). This is accepted — premature distributed-systems complexity is a larger risk than a later, evidence-driven refactor.

### Amendment Process

This ADR may only be superseded by a new ADR that satisfies the Architecture Freeze Policy above.

---

# Chapter 32: Configuration Reference

A consolidated reference for every configurable value mentioned across this book, gathered in one place — useful when you're setting up your own environment and don't want to hunt back through fifteen chapters to find a specific default.

## Environment variables (production API Gateway)

| Variable | Purpose | Example / Default |
|---|---|---|
| SECRET_KEY | JWT signing secret (Chapter 22) | Pulled from a secrets manager — never hardcoded |

| Variable | Purpose | Example / Default |
|---|---|---|
| `ACCESS_TOKEN_EXPIRE_MINUTES` | JWT token lifetime | `30` |
| `DATABASE_URL` | Postgres connection string | `postgresql://trustgraph:trustgraph@postgres:5432/trustgraph` |
| `REDIS_URL` | Cache connection string | `redis://redis:6379` |
| `LOG_LEVEL` | Logging verbosity | `info` |
| `EMBEDDING_MODEL` | Which embedding model the retriever uses (Chapter 24 discusses swapping this from the TF-IDF teaching stand-in) | Provider-specific, e.g. `text-embedding-3-small` |
| `LLM_MODEL` | Which model handles generation and narrative claim verification | e.g. `claude-sonnet-5` |

### Retriever configuration (Chapter 6)

| Parameter | Meaning | Teaching default |
|---|---|---|
| `top_k` | How many evidence chunks to retrieve | `5` |
| `rrf_k` | Reciprocal Rank Fusion softening constant (Chapter 22's math chapter) | `60` |
| `rerank` | Whether to apply cross-encoder reranking after initial retrieval | `True` in the OpenAPI spec's default; teaching implementation skips this stage entirely |

### Verifier configuration (Chapter 8)

| Parameter | Meaning | Teaching default |
|---|---|---|
| `tolerance_pct` | How close a numeric claim must be to ground truth to count as verified | `2.0` (percent) |
| `known_entities` | The set of entities the rules verifier accepts | Hardcoded set in the teaching example; production would query an entity registry |
| Word-overlap threshold | The LLM verifier stub's pass/fail cutoff (Chapter 8) | `0.3` — a teaching heuristic, not a production-appropriate threshold |

## Trust score weights (Chapter 9, formalized in Chapter 24's TRD §5)

| Dimension | v1 weight |
|---|---|
| retrieval_quality | 1/7 ≈ 0.143 |
| evidence_coverage | 1/7 |
| claim_verification_rate | 1/7 |
| freshness | 1/7 |
| conflict_score | 1/7 |
| source_authority | 1/7 |
| deterministic_verification_rate | 1/7 |

Explicitly equal-weighted and explicitly not yet calibrated against real outcome data (Chapter 9 is direct about this) — the honest starting point, not a tuned production value.

## Evaluation framework thresholds (Chapter 25)

| Setting | Purpose | Default |
|---|---|---|
| Golden dataset size | Regression testing corpus | Small, hand-reviewed, stable across runs |
| Adversarial dataset target | v0.1 evaluation coverage | 50-150 labeled examples across all failure categories |
| Regression tolerance | How much a metric can drop before blocking a release | Set per-metric during sprint planning, not a fixed global number |

## CI/CD (Chapter 20's Engineering Standards)

| Setting | Value |
|---|---|
| Minimum test coverage, services | 80% |
| Minimum test coverage, shared contracts | 100% |
| Coverage tool | `pytest-cov` |
| Linting | `ruff check` |
| Formatting | `ruff format` |
| Type checking | `mypy --strict` on `libs/contracts` and all service `src/` |

## Docker (Chapter 19)

| Setting | Value |
|---|---|
| Base image | `python:3.12-slim` |
| Exposed port | `8000` |
| Healthcheck interval | `30s` |
| Healthcheck timeout | `3s` |

### A note on where these numbers came from

Every default in this table traces back to a specific chapter where it was either measured (the trust weights, explicitly labeled unmeasured), chosen as a reasonable teaching default (tolerance_pct, word-overlap threshold), or specified as a production standard (coverage thresholds, from the Engineering Standards). None of these should be treated as tuned, optimal values without your own measurement — this chapter is a map of where each number lives, not a claim that any of them is correct for your specific deployment.

---

# Part D: Vision, Business & Execution

## Chapter 33: Executive Summary (For Non-Technical Readers)

If you are not an engineer, start here. This chapter explains what TrustGraph is and why it matters, without code or architecture diagrams.

### The problem in plain terms

Imagine a large organization: millions of documents, hundreds of databases, thousands of business rules, spread across departments with different access permissions, all constantly changing. Today, employees search documents, ask colleagues, query dashboards, or increasingly, ask an AI chatbot. That chatbot can produce an answer that sounds completely confident and is simply wrong — a revenue figure that was never in any document, a policy detail that doesn't exist, a number quietly swapped for a similar-sounding one.

For a casual question, a wrong answer is an inconvenience. For a business decision — a financial report, a compliance determination, a medical reference, a legal judgment — a wrong answer stated with total confidence is a real liability.

### What TrustGraph does differently

Instead of simply answering a question, TrustGraph finds the evidence behind the answer, checks every important claim in that answer against the evidence, measures how much it can trust the result, explains its reasoning, and keeps a complete record of how it got there.

The core idea, stated as simply as possible: **the AI is not the source of truth — it is the explanation layer.** The actual facts come from real documents and real data. The AI's job is to find them, check the generated answer against them, and explain what it found — not to be trusted on its own say-so.

### Why this is harder than it sounds

Most AI systems that "search your documents" stop after finding relevant text and generating an answer from it. They do not go back and check whether the final answer actually matches what was found. TrustGraph adds that missing step: after an answer is generated, every

individual factual claim inside it is checked separately — numbers against real data, using ordinary calculation rather than another AI guess; more open-ended statements against the actual retrieved text, using AI judgment where a calculation isn't possible.

### What "trust" means here, concretely

Instead of a single vague confidence number, TrustGraph reports a breakdown: how good was the search, how much of the answer had real supporting evidence, what fraction of individual claims were actually confirmed, how current are the sources, do the sources agree with each other, how reliable are those sources, and how much of the checking was done by hard calculation versus AI judgment. A person reviewing an answer can see exactly which of these is strong and which is weak, rather than just being told "trust me."

### Who this is for

Any organization making decisions based on large amounts of internal knowledge, where being wrong is costly: financial reporting, legal and compliance review, healthcare documentation, manufacturing standard operating procedures, HR policy questions, and executive decision support are all natural fits. The unifying idea is not "a smarter chatbot" — it's evidence-backed, checkable answers for situations where an organization needs to be able to say, afterward, exactly why it trusted what it trusted.

### What has actually been built so far

As of this book, TrustGraph exists as: a complete architecture and technical specification; a small, fully working teaching version with real retrieval, claim-checking, and trust scoring; a benchmark that has already found two real bugs in that teaching version, measured the fix, and proven the fix worked; and a full production blueprint (data contracts, API specification, engineering standards, sprint plan) ready for a small team to build against. What has not yet happened is large-scale deployment, a large evaluation dataset, or real user feedback — those are the next honest steps, not yet completed ones. This book is upfront about that distinction throughout, because overclaiming what's proven versus what's designed is exactly the discipline that makes a system like this trustworthy in the first place.

---

## Chapter 34: Project Charter

### Mission

Build a production-quality v0.1 of an Enterprise AI Intelligence Platform demonstrating verifiable AI with measurable trust, deterministic verification, and operational visibility.

### Origin

TrustGraph began as a response to a real, expensive gap in enterprise AI: RAG systems can generate confident-sounding answers with no way to prove those answers are grounded in evidence. Existing commercial observability and governance tools (Monte Carlo, Bigeye,

Databand) address data quality but not claim-level AI trust. TrustGraph closes that gap by decomposing generated answers into atomic claims, verifying each one — deterministically where possible, via LLM only where necessary — and producing a calibrated, multi-dimensional trust score rather than a single opaque confidence number.

## Status Summary

| Item | Status |
|---|---|
| Architecture | 🔒 Frozen (ADR-001) |
| Scope (v1) | 🔒 Frozen |
| Data Contracts | 🔒 Frozen (v1.0) |
| Long-term Vision (Phases 2–10) | 📁 Archived — reference only, not committed work |
| New Core Features Before v0.1 | ❌ Not permitted without an approved ADR |

## Operating Model

Per ADR-001, the team operates in three parallel tracks:

| Track | Allocation | Scope |
|---|---|---|
| Product Engineering | 80% | Implement the TRD, build the vertical slice, tests, admin console, performance, bug fixes |
| Architecture | 10% | Only changes justified by measured bottlenecks, security/reliability/scalability issues, or unmet business requirements |
| Research | 10% | Experimental work lives in research branches until backed by evidence |

## Team Structure (Capability-Based)

| Pod | Size | Owns |
|---|---|---|
| AI Intelligence | 2 | Retriever, Claim Extractor, Verification Router, Verifiers |
| Data Platform | 2 | Ingestion, indexing, connectors, metadata |
| Platform Core | 2 | API Gateway, Auth/RBAC, Event Bus, PipelineEvent infra |
| Trust & Governance | 2 | Trust Scorer, Evaluation harness, audit, calibration |

| Pod | Size | Owns |
|---|---|---|
| Experience | 2 | Admin Console, SDKs, public API surface |

### Release Target

**v0.1 — September 28, 2026**

Success is defined as: deployable, documented, tested, benchmarked, and demo-ready. **Not** feature-complete against the long-term vision.

### Scope Protection Rule

If the release date is at risk, the response is to **reduce scope**, never to extend the deadline for the sake of new ideas.

Acceptable cuts, in priority order: 1. One verifier instead of three 2. Smaller evaluation dataset initially 3. Three admin console pages instead of ten 4. REST API only (SDKs deferred) 5. Synchronous pipeline instead of event-driven

These cuts preserve the frozen architecture while protecting delivery.

---

---

## Chapter 35: Long-Term Vision & Capability Roadmap (Phases 1–10)

**Status: Archived for reference only. Nothing in this Part is committed work. No item here may begin without a new ADR satisfying the Architecture Freeze Policy (Part III).**

### Vision (5–10 years)

Build an Enterprise AI Intelligence Platform that serves as the operating system for enterprise AI, such that new AI applications reuse the platform's trust, verification, governance, and observability capabilities instead of reimplementing core infrastructure.

### Phase 1 — AI Trust Foundation (v1, in progress)

Document ingestion, hybrid retrieval, claim extraction, verification, trust scoring, REST API, admin dashboard, evaluation framework, observability, RBAC, documentation.

**Outcome:** A reliable, measurable, verifiable AI pipeline. This is the only phase currently authorized for implementation.

## Phase 2 — Enterprise Data Intelligence

SQL verification, data warehouse connectors, semantic layer, KPI definitions, business glossary, data lineage, schema discovery, metadata catalog. Extends the platform to understand structured enterprise data alongside documents.

## Phase 3 — Enterprise Knowledge Layer

Knowledge graph, entity resolution, relationship discovery, cross-source reasoning, ontology management, metadata synchronization. Enables reasoning across documents, databases, and business concepts.

## Phase 4 — AI Agent Runtime

Workflow engine, tool registry, agent memory, planning, multi-step execution, human approvals, scheduling, retry and recovery. Agents become consumers of the trust infrastructure rather than independent systems.

## Phase 5 — AI Operations Platform

Experiment tracking, prompt registry, model registry, A/B testing, cost analytics, token optimization, latency optimization, deployment management.

## Phase 6 — Enterprise Governance

Policy engine, compliance rules, audit trails, approval workflows, data residency, retention policies, access governance, risk management.

## Phase 7 — Developer Platform

SDKs, extension APIs, CLI, templates, plugin interfaces, webhooks, event subscriptions, documentation portal.

## Phase 8 — Multimodal Intelligence

PDFs, images, tables, charts, audio, video, CAD, email, office documents — one verification and trust model working across modalities.

## Phase 9 — Enterprise Automation

Workflow orchestration, business process automation, ERP/CRM/ITSM integration, ticket creation, report generation, decision execution.

## Phase 10 — AI Intelligence Operating System

The mature platform: AI search, document intelligence, SQL analytics, decision intelligence, knowledge graph reasoning, agent orchestration, trust and verification, governance, monitoring, evaluation, APIs, SDKs, multi-tenancy, hybrid deployment.

## Technical Evolution Staging

Infrastructure complexity is added only when measured need demands it, not speculatively:

| Stage | Characteristics |
|---|---|
| Stage 1 (v1, current) | Modular monolith, single PostgreSQL database, single deployment, synchronous pipeline |
| Stage 2 | Event bus, background workers, object storage, separate vector database, distributed caching |
| Stage 3 | Service decomposition based on measured bottlenecks, independent deployments, horizontal scaling |
| Stage 4 | Multi-region deployment, high availability, disaster recovery, autoscaling, hybrid cloud |

## What Success Looks Like, Long-Term

The platform should eventually let a new AI application be added via configuration and business logic rather than rebuilding core infrastructure — e.g., a document assistant reuses ingestion/retrieval/verification/trust scoring; a SQL analytics assistant reuses auth/evaluation/observability/governance; a workflow tool reuses the policy engine and audit logging. This is the long-term reason the v1 architecture is built on stable, typed contracts — but it is not a justification for building any of Phases 2–10 now.

## Evidence-Driven Platform Evolution

TrustGraph v0.1 intentionally implements a single domain to validate the platform architecture and evaluation methodology. Additional domains will be added only after measurable evidence from the current implementation. Platform abstractions — such as reusable domain interfaces or Domain Packs — will be extracted from multiple working implementations rather than designed in advance, following the project's Architecture Freeze Policy (Chapter 19). The planned sequence is: finish the single-domain v0.1 with complete benchmarking; build a second domain (Sales is the strongest candidate — it shares finance's structured, numeric, deterministically-verifiable character, making it a clean test of whether the architecture actually generalizes or only appeared to); compare the two independent implementations to see what was genuinely common infrastructure versus accidental to the first domain; and only then extract shared abstractions from that evidence.

---

---

# Chapter 36: The Platform Capability Model

TrustGraph's long-term vision (Chapter 25) describes it evolving into a platform other AI applications can build on, rather than a single application. This chapter details the specific capability model that vision is built from — organized not as features, but as reusable services other applications can share. **Everything in this chapter is long-term architecture, not current v0.1 scope** (see the Architecture Freeze Policy, Chapter 22). It's included in full because understanding the destination clarifies why v0.1's contracts are shaped the way they are.

## 1. Knowledge Intelligence

Understanding enterprise knowledge: OCR, semantic indexing, entity extraction, relation extraction, business rule extraction, metadata enrichment, version management, knowledge graph generation, and ongoing knowledge quality monitoring. Applications built on this: enterprise search, policy assistants, documentation copilots.

## 2. Data Intelligence

Understanding structured enterprise data: connectors for PostgreSQL, Snowflake, ERP, CRM, HRMS, finance systems, and streaming sources; SQL generation, data profiling, KPI computation, schema understanding, lineage, and data validation. Applications: analytics copilots, KPI assistants, business intelligence tools.

## 3. Retrieval Intelligence

Finding the right evidence: BM25, dense retrieval, hybrid retrieval, metadata filtering, graph traversal, rule lookup, SQL retrieval, and cross-encoder reranking — everything Part B's teaching retriever is a small, working instance of. The open research direction here is adaptive retrieval strategy selection based on query characteristics.

## 4. Query Intelligence

Deciding *how* to solve a problem before invoking an LLM at all: classify intent, assess risk, estimate complexity and cost, then choose an execution plan — SQL, rules, search, knowledge graph, LLM, or some hybrid. This generalizes the same idea already built into v0.1's Verification Router (which routes *claims* to the right verifier) up to the level of routing entire *queries* to the right execution strategy.

## 5. Context Intelligence

Building the best possible context before generation: deduplication, compression, permission filtering, contradiction detection, citation injection, and token budgeting — the preparatory work that determines what an LLM actually sees.

## 6. AI Intelligence

Language understanding and generation itself: classification, summarization, translation, reasoning, extraction, planning. The platform routes a given task to whichever model best fits its accuracy, latency, and cost requirements, rather than using one model for everything.

## 7. Verification Intelligence

The platform's defining capability, and the one v0.1 already implements a real version of: claim → verification strategy router → SQL verifier, rule verifier, graph verifier, document verifier, or LLM verifier → verification result. Every important claim is checked with the most appropriate method available, not a single one-size-fits-all approach.

## 8. Trust Intelligence

Multi-dimensional trust assessment instead of one confidence value — retrieval quality, evidence coverage, deterministic validation rate, source authority, freshness, conflict detection, and calibration — exactly the seven-dimension score built in Chapter 9 and specified formally in Chapter 18.

## 9. Decision Intelligence

Moving from question-answering to decision support: facts → evidence → rules → verification → decision → plain-language explanation. Deterministic business logic stays separate from language generation; the LLM explains a decision, it does not make it.

## 10. Workflow Intelligence

Coordinating multi-step enterprise processes — HR onboarding, procurement approvals, compliance checks, incident response, document review — while preserving human approval wherever required.

## 11. Agent Intelligence

Agents (a policy assistant, a finance assistant, a sales assistant) become orchestrators that call platform capabilities, rather than independent systems that each reimplement retrieval, verification, and trust logic from scratch.

## 12. Analytics Intelligence

Dashboards over the platform's own operation: AI performance, trust metrics, retrieval and verification quality, latency, cost, user adoption, and business outcomes.

## 13. Governance Intelligence

Authentication, RBAC, document-level permissions, audit logging, policy enforcement, prompt governance, model governance, and dataset governance.

## 14. Observability Intelligence

Full execution tracing: traces, logs, metrics, costs, failures, verification outcomes, and trust score distributions across every request.

## 15. Evaluation Intelligence

Continuous evaluation via benchmark suites, adversarial datasets, regression tests, calibration analyses, and A/B experiments — the discipline built concretely in Chapters 11 and 19, generalized here to every subsystem, not just claim verification.

## 16. Learning Intelligence

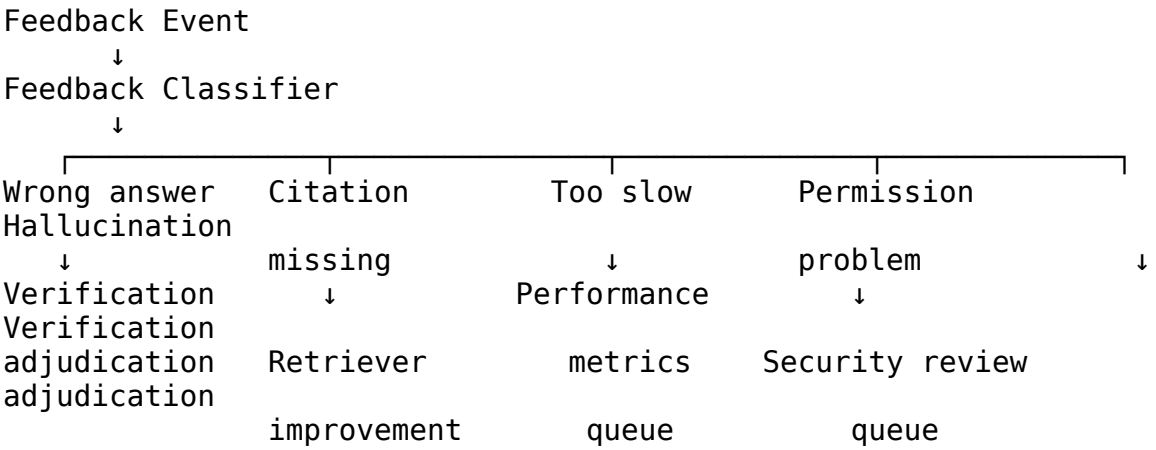
Using operational feedback — human review, correction logs, recurring failure patterns — to improve retrieval, reranking, routing, and trust calibration over time, starting with human-reviewed offline improvements before any automation is introduced.

### The Feedback & Adjudication Loop

Trust, as built through Chapter 9, is one-directional: the system tells the user how much to trust an answer, but nothing happens if the user disagrees. Closing that loop requires a specific design, because the two obvious naive approaches both fail — automatically trusting every user complaint makes the system gameable and its trust score meaningless; automatically re-verifying every complaint with a fresh LLM call or triggering retraining makes the loop expensive enough that it won't survive contact with real usage volume.

Learning Intelligence was already part of the platform vision (this capability, numbered 16 above). This section specifies its internal design without changing v0.1's implementation scope — engineering-wise, this is a real subsystem with its own state, workflow, and storage, not merely an existing idea restated; what's unchanged is that it remains outside what v0.1 implements.

**Not every piece of feedback means the same thing, so classification comes before adjudication:**



A “too slow” report has nothing to do with claim correctness and shouldn’t consume any verification machinery at all — routing it correctly, immediately, to a performance-metrics queue is itself part of the cost discipline this loop is built around.

**Adjudication produces a confidence, not just a verdict, and routes accordingly:**

```
System Correct, confidence 0.98  →  No action
System Incorrect, confidence 0.95  →  Benchmark candidate
Uncertain, confidence 0.54  →  Human review
```

Adjudication reuses the cheapest applicable verifier first, exactly as the original Verification Router does: a disputed numeric claim is re-checked against ground truth via the same deterministic SQL/Pandas path from Chapter 8 — nearly free, and it independently determines whether the system or the user was correct rather than trusting either party’s assertion. A disputed narrative claim first gets a cheap evidence-overlap re-check; only a genuinely low-confidence, inconclusive case escalates to an LLM or human reviewer, batched rather than resolved with real-time expensive compute per complaint.

**The system routes work; it does not learn automatically.** This is the most important safety property of the whole design:

```
Feedback → Classification → Adjudication → Decision → Action Router
                                                                ├──
Benchmark Dataset (Chapter 25)
                                                                ├──
Retrieval Improvement queue
                                                                ├── Trust
Calibration queue
                                                                ├──
Rule/Fact Update queue
                                                                ├── Human
Review queue
                                                                └── Ignore
(system was correct)
```

Nothing in this pipeline is “the model retrains itself.” Every branch produces a labeled item in a queue that a person or a scheduled evaluation cycle acts on — the same evidence-first discipline governing every other part of this project, applied to production feedback instead of pre-release testing.

**No feedback ever directly updates the knowledge base.** This is the governance rule enterprise deployment requires, and it’s deliberately strict:

```
Feedback → Candidate → Verified → Approved → Knowledge updated
```

A user correction becomes a *candidate* fact, not an update. It only becomes real once independently verified (via the adjudication step above) and then explicitly approved — mirroring exactly the source-authority hierarchy already established in Chapter 32’s Case Study 2 (approved policy > draft > email). A feedback loop that could silently rewrite ground truth on a single user’s say-so would undermine the entire trust story this platform is built around.

### Trust Recovery — a research direction this subsystem opens

Static evaluation (Chapter 25) measures a trust score at one point in time. Once a feedback loop exists, a new class of question becomes measurable: not just "is this answer trustworthy," but "how well does the system recover from being wrong." Concretely: how quickly are recurring failures corrected once first reported? What fraction of user reports become verified defects rather than false alarms? Which subsystem (retrieval, extraction, verification) generates the most recurring corrections? How much does each confirmed correction move the benchmark? Does trust calibration measurably improve over successive correction cycles? These move the evaluation story from a static snapshot to a reliability-engineering discipline — genuinely one of the strongest research directions available in this whole platform, and one that only becomes possible once real operational feedback exists.

## How to read this chapter correctly

None of these sixteen capabilities is sprint work today. Each becomes real only when a specific, measured limitation in the running v0.1 system justifies building it — per the same rule that governed the claim-extraction work in Chapter 11: observe a failure, diagnose its root cause, build the smallest fix that addresses it, and measure whether it actually helped. This chapter is a map of where the platform *could* go with that discipline, not a checklist of what to build next.

---

# Chapter 37: Domain Fit and the Domain-Extensibility Model

A question that comes up immediately once someone understands the architecture: what kind of data is this actually good for — HR, finance, sales, IT, all of it equally? This chapter answers that precisely, and lays out the model for extending TrustGraph into new domains without repeating the over-design pattern documented in Chapter 30.

## The precise claim, not the loose one

It is tempting to say "TrustGraph is domain-agnostic." That's technically true of the architecture and operationally misleading: **TrustGraph is a domain-extensible AI verification platform.** The core — retrieval, verification, trust scoring, evaluation, governance, and observability — is reusable across domains. Each new domain contributes its own knowledge assets: facts, rules, ontology, evidence sources, and an evaluation dataset. That's a smaller claim than "works everywhere out of the box," and a more defensible one.

## Domain-by-domain breakdown

| Domain | Ground truth sources | Dominant verification method | Typical claims | Fit today |
|---|---|---|---|---|
| **Finance** | ERP, general ledger, trial balance, forecast models | SQL / arithmetic / accounting rules / variance analysis | Revenue, EBITDA, margin, cash flow | Strongest — highly numeric, exactly what the deterministic |

| Domain | Ground truth sources | Dominant verification method | Typical claims | Fit today |
|---|---|---|---|---|
| | | | | verifier is built for |
| **Sales / CRM** | CRM, pipeline data | Same as finance — deterministic aggregate checks | Pipeline value, quota attainment, win rate | Strong — shares finance's structured, numeric character |
| **HR** | HRMS, policies, attendance, payroll | Rule engine, temporal reasoning, eligibility logic | Leave, promotion, benefits, payroll, policy | Good, more work — fewer pure numbers, policies evolve over time |
| **Manufacturing** | MES, SCADA, IoT, SAP | Threshold validation, time-series analysis, SPC rules | Downtime, OEE, scrap, yield | Needs a new capability — **streaming verification** doesn't exist in the current claim/evidence model at all |
| **Supply Chain** | WMS, TMS, ERP | Optimization constraints, route validation, inventory rules | Inventory, fill rate, delivery, procurement | Moderate — structured but less arithmetic-heavy than finance |
| **Legal / Compliance** | Contracts, regulations, policies | Citation verification, authority hierarchy, version validation | Clause presence, obligation, compliance status | Good, different mix — evidence lineage and source authority matter far more than numeric verification here |
| **Healthcare** | EHR, clinical guidelines, formularies | Temporal reasoning, medical ontologies, clinical coding | Dosing, eligibility, protocol adherence | Weakest today — needs table understanding, OCR handling, and temporal reasoning (Chapter 14, Problems 6-9) that aren't built |

| Domain | Ground truth sources | Dominant verification method | Typical claims | Fit today |
|---|---|---|---|---|
| | | | | yet |

## What actually changes per domain — and what doesn't

The architecture doesn't change. Three things do:

1. **Which verifier dominates.** Numeric-heavy domains (finance, sales) lean on the deterministic SQL/Pandas verifier; policy-heavy domains (HR, legal) lean on the rules engine and LLM verifier for evidence matching.
2. **What connector feeds it.** Chapter 25's Data Intelligence capability lists connectors for Postgres, Snowflake, ERP, CRM, HRMS — none built yet in v0.1; each domain needs its own.
3. **What "ground truth" means.** Finance has one clean source (the ledger). HR has versioned policies with effective dates. Legal has an authority hierarchy (approved policy > draft > email > meeting notes). The Verification Router doesn't know any of this — it's configured per domain, not inferred.

## The Domain Pack model (long-term architecture, not current scope)

```
Core TrustGraph Platform
├── Retrieval Engine
├── Verification Engine
├── Trust Engine
├── Evaluation Engine
├── Security
├── Observability
├── API
└── Infrastructure
            ↓
Domain Pack (plug-in)
├── Ontology
├── Business Rules
├── Fact Schema
├── Evidence Sources
├── Verifiers
├── Prompt Templates
├── Evaluation Dataset
└── KPIs
```

The core platform stays unchanged across domains; everything domain-specific belongs in a Domain Pack. This is a clean, sound design — and, per the Architecture Freeze Policy (Chapter 19), it is explicitly **not** something to build from a single implemented domain. Generalizing an interface from one example is guessing at what's actually shared versus what's merely accidental to that one example.

## The evidence-driven sequence for getting there honestly

**Stage 1 — One domain, fully measured.** Finance, with its own facts table, verifiers, evaluation dataset, and benchmark — this is v0.1 as scoped throughout this book.

**Stage 2 — A second, fully independent domain.** Sales is the strongest candidate: it shares finance's structured, numeric, deterministically-verifiable character, making it a clean test of whether the architecture generalizes or only appeared to. Built independently — not by reusing abstractions simply because they exist, but by repeating the entire process from scratch and then asking: which code ended up copied verbatim? Which evaluation logic stayed identical? Which verifier had to change? Which metadata was domain-specific?

**Stage 3 — Extract common components from evidence.** Only after Stage 2 do interfaces like `DomainConfig`, `FactResolver`, `VerifierRegistry`, `RuleProvider`, and `EvaluationDataset` get introduced — extracted from two working implementations, not invented beforehand.

**Stage 4 — The Domain Pack / SDK becomes justified.** `FinanceDomain`, `SalesDomain`, `HRDomain`, `LegalDomain` as formal plug-ins is no longer speculative architecture at this point — it's a refactor based on a real, observed, repeated pattern.

## How the evaluation framework grows alongside domains

Chapter 25's Evaluation Framework Specification describes one benchmark. The mature version, once Stage 2 exists, is a benchmark per domain:

```
Benchmarks
├── Finance
├── Sales
├── HR
└── Legal
```

This is what eventually lets you answer real, evidence-backed questions: does verification accuracy generalize across domains? Does retrieval quality remain stable? Which verifier performs best in which domain? Does trust calibration drift domain to domain? Those are far stronger engineering and research claims than asserting the architecture is domain-agnostic and hoping it holds.

## What this means for the current v0.1 plan

Nothing changes in the Sprint 1 backlog or the September release target (Chapter 20). This chapter is deliberately Part D content — long-term direction, explicitly not current sprint work, per the same discipline governing every other piece of long-term vision in this book. The concrete next step, when it happens, is Stage 2: build Sales as a second, independent, fully benchmarked domain — not design the Domain Pack interface in advance of having one.

---

# Chapter 38: Admin Control Plane — Full Module Reference

v0.1 ships exactly three Admin Console screens (Executive, Pipeline Explorer, Claim Explorer — see Chapter 20's Release Gates). This chapter documents the full long-term Control Plane those three screens are the first slice of, organized as a real operator interface rather than a marketing dashboard — modeled on how mature platforms like Datadog or Langfuse organize an operations surface.

## Two planes, not one

Mature platforms separate a **Control Plane** (how the system is configured, monitored, and governed) from a **Data Plane** (the actual request path: retrieval, verification, trust scoring, response generation). The distinction matters because it clarifies who touches what — an operator manages the Control Plane; a user request flows through the Data Plane; they should not be entangled in the same code paths.

## Operational domains, not just pages

Rather than a flat list of screens, the long-term Control Plane organizes around operational questions:

**AI Operations** — pipeline health, latency, failures, token usage, queue depth: is the system running well right now?

**Trust Center** — claim verification rates, trust score distributions, evidence coverage, failure reasons, hallucination trends over time: is the system's output trustworthy, and is that improving or degrading?

**Data Center** — document ingestion status, connector health, embedding versions, index status, storage: is the knowledge base current and complete?

**Model Center** — LLM routing decisions, prompt versions, model comparisons, token costs, rollout status: which models are doing the work, and what does that cost?

**Evaluation Center** — benchmark runs, regression history, calibration curves, retrieval and verification metrics over time: exactly what Chapter 11's `evaluation/reports/` folder produces, surfaced as a UI instead of files on disk.

**Governance Center** — RBAC, policies, audit logs, compliance status, approval workflows: who can see what, and is that being followed?

**Workspace Center** — organizations, users, teams, API keys, usage: the multi-tenant surface (explicitly deferred in v0.1, per Chapter 22's Architecture Freeze Policy, but designed for here).

**System Center** — logs, traces, metrics, events, alerts: the raw operational telemetry underlying everything above.

### The three v0.1 screens in this context

**Executive Dashboard** is the v0.1 slice of AI Operations plus Trust Center — the two most decision-relevant views for someone checking "is this working" without digging into details.

**Pipeline Explorer** is the v0.1 slice of System Center — visualizing the `PipelineEvent` trace for one request end to end (Chapter 17's Data Contract Specification defines exactly what that event contains).

**Claim Explorer** is the v0.1 slice of Trust Center — the audit interface showing, for one response, every claim, its verification status, its supporting evidence, and its confidence.

### Why the rest isn't built yet

Building all fourteen-plus modules before there's a working three-screen core would repeat the exact over-design pattern this project spent many review cycles correcting (see Chapter 30, Project Evolution Log). Each additional Control Plane module gets built when a real operational need for it appears — not because the full list looks impressive in a document. This chapter exists so that when that need appears, there's already a coherent home for it, instead of a bolted-on afterthought.

---

## Chapter 39: Extended Long-Term Vision (Addendum)

**Status: Archived for reference only, same as Chapter 21. Nothing here is authorized work. Any item below requires its own ADR, satisfying the Architecture Freeze Policy (Chapter 17), before implementation may begin.**

Beyond the Phase 1-10 roadmap in Chapter 21, later strategic reviews proposed an even broader long-term framing: TrustGraph as an "operating system for enterprise AI," organized as a set of reusable **capabilities** rather than features, so that many different AI applications (document assistants, analytics copilots, policy assistants, workflow automation) could all be built on the same underlying platform.

### The expanded capability list

| Capability | Purpose |
|---|---|
| Knowledge Intelligence | OCR, semantic indexing, entity/relation/business-rule extraction, knowledge graph generation |
| Data Intelligence | Warehouse/ERP/CRM connectors, SQL generation, KPI computation, lineage |
| Retrieval Intelligence | BM25, dense, hybrid, graph traversal, rule lookup, cross-encoder reranking |
| Query Intelligence | Intent/risk/complexity/cost estimation before choosing an execution path (SQL, |

| Capability | Purpose |
|---|---|
| | rules, search, graph, LLM) |
| Context Intelligence | Deduplication, compression, permission filtering, contradiction detection |
| AI Intelligence | Classification, summarization, reasoning, extraction, model routing |
| Verification Intelligence | The claim → router → SQL/rules/graph/document/LLM verifier pipeline this book already builds |
| Trust Intelligence | The multi-dimension trust score this book already builds |
| Decision Intelligence | Facts + rules + evidence + verification → decision + explanation |
| Workflow Intelligence | Orchestrating multi-step enterprise processes (onboarding, approvals, compliance checks) |
| Agent Intelligence | Domain assistants (policy, finance, sales) built on top of the platform's other capabilities |
| Analytics / Governance / Observability / Evaluation / Learning Intelligence | Operational and quality-control layers |

## Why this is filed here, not built

This is a coherent long-term platform vision — genuinely so. But notice its scope: connectors to Snowflake, ERP, CRM, and streaming systems; a knowledge graph in Neo4j; a rule engine; a query planning layer that chooses between six execution strategies; sixteen named "Intelligences"; twenty-plus admin console modules. Building this is a multi-year, well-resourced company's roadmap, not a v0.1 milestone for a 10-person team working alongside a job search.

The core insight worth keeping — that verification and trust scoring should be reusable capabilities other AI applications can build on, not features bolted onto one chatbot — is already reflected in how TrustGraph's typed contracts (Chapter 13) are designed. Everything else here is exactly what Chapter 17's Architecture Freeze Policy exists to hold at bay until v0.1 is shipped, measured, and evidence points to what to build next.

## The one process point worth internalizing

Across every round of feedback that produced documents like this one, the recurring lesson has been the same: **ambitious platform thinking is genuinely valuable as a reference document, and genuinely risky as a running task list.** Keep the document. Don't let it become the sprint board.

---

# Chapter 40: Competitive Positioning & Commercial Case

## What TrustGraph is not selling

It is tempting to describe an ambitious platform as "the ultimate enterprise AI solution." That framing is not credible for an early-stage system, and this book avoids it deliberately. The honest, specific claim is narrower and stronger:

**TrustGraph is an enterprise AI verification platform that enables organizations to deploy LLM-based assistants with evidence-backed answers, deterministic verification, calibrated trust scores, and complete auditability.**

Everything else — query planning, knowledge compilation, rule execution, decision intelligence — is roadmap that extends from that foundation, not the pitch itself.

## The actual value proposition

Most AI products are selling accuracy: "our chatbot answers correctly more often." TrustGraph's value proposition is different and, for regulated or high-stakes environments, stronger: **we don't just answer questions — we show why the answer can be trusted.** That is a claim about risk reduction and auditability, not just correctness, and it matters most exactly where being wrong is expensive: financial reporting, legal review, compliance, healthcare documentation, and executive decision-making.

## Where it sits relative to existing categories

| Category | Representative products | How TrustGraph relates |
|---|---|---|
| Enterprise search | Glean | Adds verification and trust scoring on top of search |
| RAG frameworks | LangChain, LlamaIndex | Operates one level above — a platform, not a library you assemble yourself |
| LLM observability | Langfuse | Incorporates observability, but couples it directly to execution rather than bolting it on afterward |
| AI evaluation | Arize AI | Integrates evaluation into the runtime architecture itself, not as a separate add-on product |
| Knowledge graph platforms | Neo4j | Treats the graph as one capability among several, not the entire product |

The overlap with each category is expected — enterprise platforms typically combine capabilities that used to require several separate tools. The differentiator is not any single feature in this table; it's that hybrid retrieval, deterministic verification, verification routing,

trust calibration, and a real benchmark framework are integrated into one system rather than distributed across five vendor relationships.

### Potential enterprise applications

Internal knowledge assistants, policy compliance checking, contract analysis, financial report verification, healthcare documentation support, manufacturing standard-operating-procedure guidance, HR decision support, executive analytics explanation, and audit-ready regulated AI deployments. The unifying thread across all of these is not the document type — it's that a wrong, confidently-stated answer would be costly, and an organization needs to be able to explain afterward exactly why an answer was trusted.

### Being honest about maturity

As of this book, TrustGraph should be described accurately across three separate dimensions, because conflating them overstates what's actually proven:

**As an engineering platform:** mature. The architecture, typed contracts, verification pipeline, and evaluation framework are real, working, and demonstrated with measured results (Chapter 11, Chapter 12).

**As a research platform:** early but genuine. The infrastructure to run controlled experiments — baselines, ablations, calibration measurement — exists and has already produced one real result (the claim extractor ablation). Turning this into a research *contribution* (a claim someone else could cite) requires larger datasets, more baselines, and reproducible results across more conditions than currently exist.

**As a commercial product:** early-stage, high potential. The problem is real and the differentiation is genuine, but product-market fit, customer validation, and real deployment experience are all still ahead, not behind.

Keeping these three assessments separate — rather than blending them into one confident pitch — is itself part of what a credible verification-focused platform should model in how it talks about itself.

---

## Chapter 41: Team Structure & Operating Model

### Organize around capabilities, not technical layers

An early instinct is to organize a team by technical layer: a "retrieval team," a "verification team," a "trust team." This creates a hidden cost — almost any real feature touches retrieval, verification, and trust together, so a layer-based org turns every feature into a three-team negotiation. TrustGraph's team is instead organized around **capabilities**, so that most features can be built end-to-end inside one team's boundary:

| Team | Owns | Responsibility |
|---|---|---|
| AI Intelligence | Retrieval, claim extraction, verification | Everything required to answer and verify a query |
| Data Platform | Ingestion, indexing, metadata, connectors | The full data lifecycle |
| Platform Core | Auth, API Gateway, RBAC, event bus | Platform infrastructure |
| Trust & Governance | Trust score, evaluation, audit, policies | AI governance and quality |
| Experience | Admin Console, APIs, SDKs | Everything users and developers interact with |

Each team's boundary lines up with a real interface in the Data Contract Specification (Chapter 17) — Claim and VerificationResult belong to AI Intelligence, TrustScore to Trust & Governance, PipelineEvent to Platform Core, and so on. This is not a coincidence: contract ownership and team ownership should be the same lines, or the contracts stop meaning anything.

## The three-track operating model

Once the architecture was frozen (Chapter 22's ADR-001), the team's day-to-day work was deliberately split into three tracks with fixed allocations, so that "let's also explore this idea" never silently consumes the time meant for shipping:

**Track 1 — Product Engineering (80%).** Implementing the TRD, building the vertical slice, writing tests, building the admin console, fixing bugs. Success is measured by working software.

**Track 2 — Architecture (10%).** Architecture changes are accepted only if justified by a measured performance bottleneck, a security or reliability issue, a scalability problem proven by testing, or a business requirement the current design genuinely cannot meet. Otherwise: no architecture changes, full stop.

**Track 3 — Research (10%).** Exploratory work — better retrieval methods, new verification algorithms, agentic workflows — lives in research branches and documents until it has evidence to justify promotion into Track 1 or 2.

## Bounded contexts, stated as four questions

Every subsystem in the platform should be able to answer four questions cleanly:

1. What capability does it own?
2. What API does it expose?
3. What events does it publish and consume?
4. What data does it own?

If a subsystem can't answer all four without hedging, that's a sign its boundary isn't actually clear yet — worth fixing before more code is built around it, not after.

### Why this mattered in practice

This project went through many rounds of architectural review before implementation began (see Chapter 30, Project Evolution Log). Nearly every round proposed real, individually-reasonable expansions — new subsystems, new admin modules, new capabilities. The three-track model and the Architecture Freeze Policy are the concrete mechanism that turned "everything proposed sounds reasonable" into "here is exactly what ships in v0.1, and here is where everything else honestly belongs instead." Without a stated operating model, a team drifts toward whichever idea was discussed most recently rather than what's actually been decided.

---

## Chapter 42: Worked Case Studies Across Industries

TrustGraph is a domain-extensible AI verification platform: the core platform — retrieval, verification, trust scoring, evaluation, governance, and observability — is reusable across domains, but each new domain contributes its own knowledge assets (facts, rules, ontologies, evidence sources, and evaluation datasets) rather than requiring a new platform architecture. This chapter walks through three illustrative scenarios (constructed for this book, not real deployments) showing how the same core pipeline applies differently depending on what's at stake, and how much domain-specific work each one actually requires. Each follows the same shape: a question, what a bare LLM would likely do, and what TrustGraph's verification pipeline changes.

### Case Study 1: Financial Report Verification

**Scenario:** An analyst asks an internal assistant, "What was our Q3 gross margin, and how does it compare to last quarter?"

**What a bare LLM-based assistant risks:** LLMs are notably unreliable at precise arithmetic and at correctly picking between similar-sounding figures from a document (gross margin vs. net margin vs. operating margin). A confidently stated wrong number in a figure that feeds into a board report is exactly the kind of error that's expensive precisely because it looks unremarkable.

**How TrustGraph's pipeline changes this:** the retriever finds the relevant financial statements; the claim extractor pulls out the specific numeric claims ("gross margin was 34.2%," "up from 31.8% last quarter"); the Verification Router sends both numbers to the deterministic SQL/Pandas verifier, which checks them against the actual underlying ledger data rather than trusting the LLM's arithmetic. The trust score's `deterministic_verification_rate` dimension tells a reviewer, at a glance, that these particular numbers were checked by calculation, not by AI judgment — the distinction that matters most in a financial context.

**What doesn't change:** if a narrative claim appears alongside the numbers ("margin improved due to reduced supply chain costs"), that causal claim isn't a pure calculation — it goes to the

LLM verifier, and its lower `claim_verification_rate` confidence should visibly read as less certain than the two hard numbers next to it.

## Case Study 2: Legal Contract Compliance Review

**Scenario:** A compliance officer asks, "Does this vendor contract include a data residency clause requiring EU storage?"

**What a bare LLM-based assistant risks:** contract language is often deliberately precise in ways a paraphrasing model can blur — "may store data outside the EU under specific conditions" is materially different from "shall not store data outside the EU," and a system that summarizes rather than verifies can lose exactly that distinction.

**How TrustGraph's pipeline changes this:** this is a case where the entity/rules verifier (Chapter 8) and the LLM verifier work together rather than the deterministic path doing all the work — there's no arithmetic to check, but there is a specific clause to locate and confirm is present, which is a retrieval and evidence-matching problem more than a language-generation one. The Evidence Coverage dimension of the trust score is the most important one here: if the relevant clause wasn't actually retrieved, the system should show low evidence coverage rather than a falsely confident answer built on the wrong section of the contract.

**What this case study highlights:** not every domain benefits equally from the deterministic verifier — legal and policy questions lean much more heavily on retrieval quality and LLM-based evidence matching than on SQL-style number-checking. This is a good illustration of why the Verification Router's job is to pick the *right* strategy per claim type, not to force every claim through the same path.

## Case Study 3: Healthcare Documentation Assistance

**Scenario:** A clinician asks, "What is the standard dosing protocol for this medication per our hospital's current formulary?"

**What a bare LLM-based assistant risks:** this is the highest-stakes case in this chapter. A model trained on general medical text may recall a dosing convention that is common but not what *this specific hospital's* current formulary specifies, and could state it with total confidence.

**How TrustGraph's pipeline changes this:** the `Freshness` and `Source Authority` dimensions of the trust score matter enormously here — a formulary is versioned and updated, and an answer sourced from a superseded version is actively dangerous, not just imprecise. This is the strongest real-world case for the temporal reasoning capability described in Chapter 14's "Twelve Hard Problems" (Problem 9) — every fact needs an effective date and a current-version flag, so a query automatically resolves to what's active today, not whatever version happened to be retrieved.

**What this case study highlights, honestly:** this is also the domain where this book's teaching implementation is furthest from being deployment-ready. Chapter 14's Problems 6-9 (tables, OCR, temporal reasoning) are exactly the gaps that would need to be closed — with real evidence from a real benchmark, not just architectural intention — before a system like this should be anywhere near an actual clinical dosing decision.

### The common thread

None of these three scenarios changes the architecture. What changes, case to case, is which trust dimensions matter most, and which verification strategy carries the most weight. That flexibility — the same pipeline, the same typed contracts, different weight given to different evidence depending on the domain — is the actual argument for building this as one platform with a Verification Router, rather than three separate bespoke systems that each reimplement retrieval and trust scoring from scratch.

---

## Chapter 43: Talking About TrustGraph to Different Audiences

The same project needs to be described differently depending on who's listening — not by changing the facts, but by changing which facts are relevant to that listener. This chapter gives a template for each.

### To an engineering interviewer

Lead with the architecture and the evidence, in that order: "I built a verifiable RAG platform where every generated answer is decomposed into atomic claims, each verified against evidence using the most appropriate method — deterministic SQL/Pandas checks for numeric claims, LLM judgment only for narrative ones. I benchmarked it against four baselines that don't verify anything, and they all scored exactly zero on hallucination detection while the verification pipeline caught every real error in the test set. Along the way the benchmark itself found two real bugs in my claim extractor, which I fixed and re-measured — Claim F1 went from 0.70 to 1.00 on the fixed issues, and I have an honest follow-up test showing exactly where that fix stops generalizing."

What this leads with: real numbers, a real bug-fix loop, and an honest boundary — not a feature list. That combination is what a technical interviewer is actually listening for.

### To a non-technical stakeholder or executive

Lead with the business problem, not the architecture: "Employees increasingly ask AI assistants questions instead of searching documents themselves, but those assistants can produce a wrong answer stated with total confidence — a wrong number in a report, a misstated policy. This platform adds a step most AI systems skip: after generating an answer, it checks every important claim in that answer against the real evidence, and tells you exactly how much to trust the result and why. It doesn't just answer — it shows its work."

What to avoid: architecture diagrams, model names, or the sixteen-capability platform vision — none of that is what a business stakeholder needs to decide whether the problem is real and the approach is sound.

### To a potential research collaborator or advisor

Lead with the open questions, not the finished implementation: "I have a working system that separates deterministic and LLM-based verification, and an evaluation harness with baselines and ablations. The open research questions I'm most interested in are whether trust score calibration actually holds up at scale — does a system reporting 90% trust turn out to be right about 90% of the time — and whether adaptive routing (choosing SQL vs. rules vs. LLM verification per claim, and eventually per whole query) measurably beats a fixed pipeline. I don't have publishable results on either yet; I have the infrastructure to run the experiments."

What matters here: being precise about the difference between "I built something that could support this experiment" and "I ran the experiment and got a result" (Chapter 28 discusses this distinction directly) — a research audience will notice immediately if that line is blurred.

### To a potential teammate or collaborator

Lead with the concrete next task, not the vision: "Here's the frozen architecture and the data contracts — Claim, Evidence, VerificationResult, TrustScore. Here's the Sprint 1 backlog with a task per pod. Here's the Architecture Freeze Policy, so you know what needs an ADR versus what you can just build. Here's the benchmark, so you know exactly what 'this works' means before you open a PR."

What matters here: giving someone a bounded, checkable piece of work rather than the whole sixteen-capability vision — someone joining a project needs a place to start, not a museum tour.

### The one thing to never do, in any of these rooms

Present the long-term roadmap (Chapters 24-25) as if it were already built. Every audience above responds better to "here's what's real, here's what's designed but not built, here's the line between them" than to an impressive-sounding list that turns out to be mostly aspiration once someone asks a follow-up question. That distinction, kept honestly, is the actual credibility asset — more than any individual capability on the roadmap.

---

## Chapter 44: Interview Preparation — Talking About This Project

Chapter 33 gave short templates for different audiences. This chapter goes deeper, specifically for a technical interview, with realistic follow-up questions an interviewer is likely to ask and how to answer them honestly — including the answers where the honest response is "I haven't measured that yet."

### "Walk me through the architecture."

Start with the pipeline, not the platform vision: "A query goes through hybrid retrieval — dense and sparse search combined with Reciprocal Rank Fusion — then the generated answer is broken into atomic claims. Each claim is routed by type: numeric claims go to a deterministic

SQL/Pandas verifier, entity claims to a rules engine, and only narrative claims that need actual language judgment go to an LLM verifier. The results feed a seven-dimension trust score, not a single confidence number." This is roughly 30 seconds and covers the actual differentiator. Only go into the sixteen-capability long-term vision if they ask what's next.

### "Why not just use an LLM to verify everything?"

This is a strong signal question — it's testing whether you understand your own design rationale, not just the mechanics. "Because using an LLM to check a number a second time doesn't add real verification — it's just another chance for the model to be wrong, correlated with the same kind of error the first generation might have made. A calculation against real data is deterministic and 100% reliable given correct source data; an LLM's judgment isn't. I only use the LLM verifier for claims that are genuinely about meaning — where there's no formula to check against."

### "What was the hardest bug you found?"

Answer with the actual story, because it's a better answer than anything invented: "My benchmark found that a churn-rate claim was being compared against revenue-growth ground truth, because my verifier matched only on unit type — 'percent' — not on which specific metric the number referred to. I fixed it by making the fact lookup metric-aware, and Claim F1 went from 0.70 to 0.737. Then I found a second bug — a keyword collision between 'revenue' and 'prior revenue' — fixed that too, and reached 1.0 on my test set. But I didn't stop there: I ran three harder adversarial cases immediately after, and all three broke the fix, for reasons I understood and documented rather than patched with more regex."

### "Your extractor got a perfect score — is claim extraction solved?"

If you're asked this and the honest answer is "no," say so directly — this is exactly the kind of question that tests whether you overclaim results: "No — a perfect score on the same 12 examples that found the original bugs just confirms those specific bugs are fixed. I immediately tested three new adversarial cases targeting different failure modes — cross-sentence pronoun reference, embedded relative clauses — and all three failed, which is expected. Real dependency parsing and coreference resolution are the actual fix, and I have that designed but not yet built, partly because the sandbox I was working in couldn't download the trained model I needed."

### "How would this scale to millions of documents?"

"The in-memory retriever in my teaching implementation wouldn't — that's explicitly a simplification. Production swaps in pgvector or a dedicated vector database with an HNSW or IVF index for approximate nearest-neighbor search, which is what makes sub-second retrieval possible at that scale. The retrieval *algorithm* — hybrid dense+sparse combined via RRF — doesn't change; only the storage and indexing backend does, because I built the Retriever behind a stable interface specifically so that swap doesn't touch anything downstream."

### "What would you do differently if you started over?"

A good honest answer, grounded in what actually happened in this project: "I'd freeze scope earlier. Early on, a lot of architecturally sound ideas kept expanding what I was designing — a bigger platform, more capabilities — before I'd built anything and gotten real evidence about what actually mattered. What fixed it was an explicit rule: no new subsystem without a written problem statement, measurable evidence, and an implementation estimate. Once I had that, and once I had a working benchmark producing real numbers, most future decisions became much easier to make."

### "What's the weakest part of your system right now?"

Don't dodge this — a strong candidate names their own weak point unprompted, before being asked, ideally: "Claim extraction, without question. It's regex-based, which I fixed twice using a real benchmark and measured both fixes, but the round-2 adversarial test showed it still can't handle coreference or arbitrary clause structures. The second weakest point is that my adversarial dataset is only 12 examples — enough to find and fix specific bugs, nowhere near enough to claim general robustness."

### Questions worth asking back

A strong candidate also asks questions that show they understand what's actually hard here: "How do you currently measure whether your RAG system is hallucinating?" or "Do you separate deterministic checks from LLM-based verification, or is everything going through the model?" — questions like these signal you understand the actual problem space, not just that you can describe your own project.

### The one discipline that matters most in any of these answers

Every strong answer above shares one property: it's precise about what's proven with a number versus what's designed but not measured. That precision is worth more in an interview than any individual technical detail, because it's what separates "I built something and can defend every claim about it" from "I built something impressive-sounding and hope you don't ask a follow-up question."

---

## Chapter 45: Project Evolution Log — How This Design Was Stress-Tested

This chapter is unusual for a technical book: it's a narrative account of how TrustGraph's design actually evolved, including the mistakes and corrections, rather than a clean retrospective that hides the process. It's included because the process itself is a worked example of a real engineering discipline — and because a junior engineer benefits more from seeing *how* a design gets this rigorous than from just being handed the final result.

## Phase 1: The idea outgrows "RAG project"

The project began as a simple question: what would a genuinely advanced, differentiated RAG system look like? Early proposals ranged from "the most powerful RAG system" to increasingly specific ideas — data observability platforms, semantic layers, FinOps optimizers. Each was reasonable; none had a clear, singular focus. The turning point was reframing the goal around one real, underserved gap: RAG systems generate confident answers with no mechanism to verify or explain them. That gap — not "more features" — became the project's actual center of gravity.

## Phase 2: Scope expands faster than it ships

Once "verifiable RAG" was the focus, a long sequence of detailed architectural reviews followed — each one technically sound, each one adding real scope: multi-agent verification, cryptographic provenance, a full enterprise platform with fourteen subsystems, then sixteen "Intelligence" capabilities, then a twenty-one module Control Plane. Every individual addition was defensible. Collectively, they were heading toward a multi-year platform, not a project a small team could ship.

This is a genuinely common failure mode, not a sign of poor judgment — ambitious, well-reasoned ideas are seductive precisely because each one sounds correct in isolation. The pattern only becomes visible in aggregate.

## Phase 3: Freezing the architecture

The correction came in stages: first, a hard line between what ships in v0.1 and what's roadmap (this book's Chapters 25-26 preserve that full roadmap, clearly labeled as archived reference, not current work). Then a formal mechanism — ADR-001 and its Architecture Freeze Policy (Chapter 22) — requiring any new subsystem to clear five bars: a written problem statement, measurable evidence, impact analysis, an implementation estimate, and explicit approval. Absent all five, an idea stays on the roadmap, however good it sounds.

This didn't stop new ideas from arriving — they kept arriving, repeatedly, sometimes as the same proposal reworded. What changed was that they stopped automatically becoming scope. A concrete release target (v0.1, tied to a real date) and an explicit scope-protection rule (cut scope before extending the deadline) gave the team something firmer than good intentions to hold the line with.

## Phase 4: The benchmark becomes the actual authority

The most significant shift in the whole project happened once a real benchmark existed. Before it, claims about the system were architectural: "TrustGraph verifies claims," "deterministic routing should reduce hallucinations." After it, claims became measured: a baseline comparison showing every non-verifying system scores exactly zero on hallucination detection while TrustGraph catches every real error; two real bugs found by the benchmark itself, not by code review; a measured improvement from fixing them (Claim F1 0.700 → 0.737 → 1.000); and an honest, immediate follow-up test showing exactly where that fix stops generalizing.

This is the moment the project stopped being primarily a design exercise and became primarily an engineering one. The test for any future idea changed from "does this sound like it would help" to "what does the benchmark say happens if we build this."

### Phase 5: Knowing when to stop discussing and start building

The most repeated piece of feedback across this project's development was, in one form or another, "this is good — now stop reviewing and go build it." That advice was correct every time it was given, and it needed to be given more than once, because returning to design discussion is a comfortable, low-risk activity compared to writing code that might reveal new problems. Recognizing that pull — and choosing to act on evidence instead of continuing to refine the plan — is itself the discipline this chapter is trying to name.

### The lesson, stated plainly

Good architecture emerges from a real tension, not from getting the design right on the first attempt: ambitious ideas need room to be considered, and a shipping team needs a firm boundary around what's actually being built right now. Neither instinct is wrong in isolation. The discipline is having an explicit, written mechanism — a frozen architecture, a scope-protection rule, a benchmark that produces real numbers — that lets ambitious thinking continue on paper while a small, fixed, evidence-tested slice of it actually gets built and shipped.

---

## Chapter 46: Appendix — Design Decision Log

Chapter 30 told the story of how this project's design was stress-tested. This appendix is the structured companion to that narrative — a decision-by-decision log of the specific calls made, what was considered, and why. Where Chapter 30 is prose, this is closer to a changelog: useful when you need to know *why* a specific line in the TRD or ADR-001 says what it does, not just the general shape of the story.

### Decision 1: Scope — RAG project vs. platform

**Considered:** building the most advanced possible RAG system, with proposals ranging from multi-agent verification to full enterprise data platforms. **Decided:** narrow the differentiator to one real, underserved gap — claim-level verification, with deterministic checks for numeric claims — rather than pursuing breadth. **Why:** breadth without a clear center produces a project that's hard to finish and harder to explain in one sentence.

### Decision 2: Verification architecture — agents vs. services

**Considered:** a multi-agent system where a "generator agent" and a "critic agent" debate an answer's correctness. **Decided:** a linear pipeline of typed services (Retriever → Claim Extractor → Verification Router → Verifiers → Trust Scorer), not agents. **Why:** each pipeline stage is independently testable and independently replaceable; a multi-agent debate adds token cost and orchestration complexity without a clear accuracy benefit that was ever measured.

## Decision 3: Numeric verification — LLM vs. deterministic

**Considered:** using an LLM to check every claim, numeric and narrative alike. **Decided:** route numeric claims to deterministic SQL/Pandas checks; reserve the LLM verifier for narrative claims only. **Why:** an LLM checking a number is a second chance for the same kind of error, not an independent check; a calculation against real data is reliable given correct source data. This decision was validated concretely in Chapter 11's benchmark — the `deterministic_verification_rate` dimension exists specifically to make this split visible in the trust score.

## Decision 4: Trust representation — single score vs. multi-dimensional

**Considered:** a single 0-100% confidence value, matching how most systems report certainty. **Decided:** a seven-dimension breakdown (retrieval quality, evidence coverage, claim verification rate, freshness, conflict score, source authority, deterministic verification rate). **Why:** a single number is exactly as opaque as an LLM's own self-reported confidence — the thing this whole project exists to move away from. Chapter 26's dashboard scenarios (0.807 trust with one unsupported claim, 0.893 trust with one contradicted claim among five) directly demonstrate why the breakdown carries more information than the average alone.

## Decision 5: Team structure — technical layers vs. capabilities

**Considered:** organizing engineers by technical layer (a "verification team," a "trust team," an "evaluation team"). **Decided:** capability-based teams (AI Intelligence, Data Platform, Platform Core, Trust & Governance, Experience), where most features can be built inside one team's boundary. **Why:** a layer-based org turns almost every feature into a three-team negotiation; capability boundaries were deliberately drawn to match the Data Contract Specification's object ownership.

## Decision 6: Architecture governance — open design vs. frozen with an escape hatch

**Considered:** continuing to accept well-reasoned architectural proposals as they arrived, indefinitely. **Decided:** freeze the architecture (ADR-001) with an explicit Architecture Freeze Policy — five requirements (problem statement, evidence, impact analysis, implementation estimate, approval) before any new subsystem is accepted. **Why:** every individual proposal considered along the way was reasonable; collectively they were heading toward a multi-year platform. A firm, written mechanism was needed because good intentions alone didn't hold the line — this policy had to be invoked repeatedly, not just written once.

## Decision 7: Release definition — "Phase 1 done" vs. "v0.1 released"

**Considered:** measuring progress against a features-implemented checklist. **Decided:** a single release milestone (v0.1) defined by eight concrete gates — deployable, tested, benchmarked, documented, demoable — not by feature count. **Why:** "implemented" has no natural stopping point; "these eight gates are true" does.

### Decision 8: Evaluation — trust the architecture vs. build a benchmark first

**Considered:** treating a well-designed verification pipeline as self-evidently correct. **Decided:** build a benchmark with baselines (B0-B3) before trusting any claim about the system's effectiveness. **Why:** this is the single highest-leverage decision in the whole project's history — it's what turned "TrustGraph should reduce hallucinations" from an architectural claim into a measured one (all four non-verifying baselines score exactly 0.000 on hallucination detection; TrustGraph catches all seven real errors in the test set).

### Decision 9: Bug fixing — patch broadly vs. fix the specific diagnosed cause

**Considered:** adding more general-purpose text-matching rules every time the benchmark found a new failure. **Decided:** fix exactly the diagnosed root cause each time (unit-only matching → metric-aware matching; keyword collision → specificity ordering), then re-measure before considering the next fix. **Why:** general rule-patching without measurement is how "regex hell" happens — each fix should be scoped to what was actually diagnosed, not extended speculatively to catch other, unconfirmed failure modes.

### Decision 10: Round-2 testing — declare victory vs. actively look for new failures

**Considered:** stopping at the F1 = 1.000 result from the v2 extractor fix. **Decided:** immediately test three new adversarial cases specifically designed to find the *next* failure mode, before claiming the fix generalized. **Why:** a perfect score on the same examples that found the bugs proves the bugs are fixed, not that the extractor is robust — testing this distinction directly is what separates a credible result from an overclaimed one.

### Decision 11: This book's scope — how much to include

**Considered:** a short, high-level summary of the finished design. **Decided:** the full technical depth, including code that wasn't executable in this environment (clearly labeled), the actual bugs found along the way, and the honest boundary of what's proven versus designed. **Why:** per the same principle running through every decision above — a credible account of a verification-focused project should itself model being precise about what's demonstrated versus claimed, not just describe that principle in the abstract.

---

## Chapter 47: Sprint 1 Backlog & Execution Plan

**Sprint Goal:** Each pod ships its service against the frozen contracts, working in parallel against mocked upstream/downstream, per ADR-001. Platform Core unblocks everyone by shipping the contracts package and event bus skeleton first.

**Duration:** 2 weeks (Weeks 1-2 of the 12-week v0.1 plan)

---

### Platform Core (ships Day 1-2, unblocks all other pods)

- ☐ Set up monorepo structure per Engineering Standards §1
- ☐ Implement `libs/contracts` as installable package: Pydantic models for QueryContext, Evidence, Claim, VerificationResult, TrustScore, PipelineEvent, ResponseMetadata + all Request/Response envelopes from OpenAPI spec
- ☐ Contract validation test suite (schema conformance against Data Contract Spec)
- ☐ API Gateway skeleton: FastAPI app, `/query` route stub, bearer auth middleware
- ☐ PipelineEvent emission helper library (`libs/observability`)
- ☐ `docker-compose.yml` for local dev: Postgres + pgvector + Redis
- ☐ CI pipeline: lint, type-check, unit tests, coverage gate on every PR

### AI Intelligence — Retriever

- ☐ Implement `Retriever` interface (dense + BM25 hybrid, RRF combination) against `RetrieveRequest/RetrieveResponse`
- ☐ Stub Evidence Ranker (cross-encoder rerank) behind same interface
- ☐ Unit tests + fixture corpus (10-20 sample documents)
- ☐ Emit `PipelineEvent` for retrieval stage with latency + recall estimate

### AI Intelligence — Claim Extractor

- ☐ LLM-based candidate claim extraction
- ☐ spaCy NER + dependency parser cross-check pass
- ☐ Output typed `Claim` objects, `claim_type` classification
- ☐ Unit tests on known claim/entity extraction cases
- ☐ Emit `PipelineEvent` with extraction_confidence distribution

### AI Intelligence — Verification Router + Verifiers

- ☐ Verification Router: classify claim → dispatch to SQL/Pandas, Rules, or LLM verifier
- ☐ SQL/Pandas deterministic verifier for numeric claims
- ☐ Rules-engine stub verifier for entity claims (Knowledge Graph deferred to Phase 3 per roadmap)
- ☐ LLM verifier for narrative claims
- ☐ Failure taxonomy codes wired into every non-success path
- ☐ Unit tests per verifier type + integration test for router dispatch logic

### Trust & Governance — Trust Scorer

- ☐ Implement 7-dimension TrustScore formula (equal weights v1, per TRD §5)
- ☐ `TrustScoreRequest/TrustScoreResponse` implementation
- ☐ Begin `evaluation/` harness scaffold: adversarial dataset structure, metrics runner stub
- ☐ Define first adversarial test cases: contradictory documents, missing evidence, numeric mismatch

### Experience — Admin Console + Response Composer

- ☐ Response Composer: assembles ResponseMetadata from all upstream results

- ☐ Admin Console shell (React): routing for 3 screens (Executive, Pipeline Explorer, Claim Explorer) — static/mocked data first
- ☐ `/admin/pipeline-events/{trace_id}` and `/admin/responses/{response_id}/claims` endpoints (mocked backing data acceptable this sprint)

---

### Definition of Done (Sprint 1)

- Every pod's service runs standalone with unit tests passing in CI
- All services import and produce valid `libs/contracts` objects (contract tests pass)
- No pod has started work on anything outside its TRD-defined scope (Architecture Freeze Policy holds)
- Demo: one hardcoded query flows through all 5 mocked/stubbed stages end-to-end, even if individual stages are still rough

### Explicitly Out of Scope for Sprint 1

Event bus (Kafka), multi-tenant hierarchy, plugin marketplace, real Knowledge Graph, any Admin Console screen beyond the 3 approved, any capability from Phases 2-10 of the long-term roadmap.

---

## Chapter 48: v0.1 Release Gates

v0.1 is not released until **every** item below is true. If any item is missing, the project status is "not yet released" — not "almost done."

- ☐ End-to-end vertical slice works (query → retrieval → claim extraction → verification → trust score → response)
- ☐ Automated tests pass (unit tests per service, contract tests against the Data Contract Specification)
- ☐ CI/CD pipeline passes on `main`
- ☐ Benchmark suite runs successfully against the adversarial evaluation dataset
- ☐ Trust metrics are generated and visible (overall score + 7 dimensions per response)
- ☐ Admin dashboard demonstrates the full pipeline (Executive, Pipeline Explorer, Claim Explorer screens)
- ☐ Documentation is complete enough that another engineer can clone the repo and run it from `README.md` alone
- ☐ At least one deployment environment is running (local docker-compose acceptable for v0.1)

### Post-Release Sequence

After v0.1 ships, in order:

1. Publish documentation (this book, plus a public repo or demo access as appropriate)
2. Record a technical demo
3. Use it in interviews and portfolio conversations
4. Gather feedback from real users or external reviewers
5. Decide the next capability based on evidence — not the Part II roadmap — via a new ADR

### The Three Questions That Define "Done"

1. **Does it work?** Can another engineer clone the repository and run it?
2. **Can you measure it?** Trust score, verification accuracy, retrieval metrics, latency, cost, failure analysis — all with real numbers, not estimates.
3. **Can you explain it?** Can every architectural decision be defended with either measured evidence or a documented trade-off (see ADRs)?

If yes to all three, the project has achieved its v0.1 objective.

---

---

# Part E: Reference

## Chapter 49: Glossary

| Term | Definition |
|---|---|
| **RAG (Retrieval-Augmented Generation)** | Giving an LLM retrieved source text as context before it generates an answer |
| **Hallucination** | An LLM generating a confident, plausible-sounding statement that is factually wrong or fabricated |
| **Embedding** | A numerical vector representation of text such that similar meanings produce similar vectors |
| **Dense retrieval** | Finding relevant documents by comparing embedding vectors (cosine similarity) |
| **Sparse retrieval (BM25)** | Finding relevant documents by scoring exact word overlap, weighted by word rarity |
| **Hybrid retrieval** | Combining dense and sparse retrieval, typically via Reciprocal Rank Fusion |
| **Reciprocal Rank Fusion (RRF)** | A method for combining multiple ranked lists by summing `1/(k + rank)` across lists |
| **Cross-encoder reranker** | A model that scores query+document pairs jointly for higher-precision re-ranking of a |

| Term | Definition |
|---|---|
| | retrieval shortlist |
| **Claim** | An atomic, independently verifiable statement extracted from a generated answer |
| **Claim extraction** | Breaking a generated answer into individual claims |
| **Verification** | Checking a claim against evidence — deterministically (code) for numeric claims, via LLM judgment for narrative claims |
| **Verification Router** | The component that classifies each claim and dispatches it to the right verifier |
| **Trust Score** | A calibrated, multi-dimensional score summarizing how well a response's claims held up under verification |
| **Deterministic verification** | Verifying a claim with code (SQL/Pandas) instead of an LLM's judgment |
| **PipelineEvent** | An observability record capturing latency, cost, and version info at each pipeline stage |
| **Typed Intermediate Representation (IR)** | Stable, versioned data contracts (Claim, Evidence, VerificationResult, etc.) that let independently-built services interoperate |
| **Failure Taxonomy** | A fixed set of named failure codes (e.g. `NUMERIC_MISMATCH`, `NO_SUPPORT`) instead of a generic "verification failed" |
| **ADR (Architecture Decision Record)** | A written record of a significant architecture decision and its rationale |
| **Architecture Freeze Policy** | The rule that no new subsystem may be added without a written problem statement, evidence, impact analysis, and approval |
| **Vertical Slice** | A complete, working pipeline end-to-end, as opposed to a partially-built horizontal layer |
| **v0.1** | The first release milestone: deployable, documented, tested, benchmarked, demo-ready |
| **Token** | A sub-word unit of text an LLM actually processes; cost and context-window limits are measured in tokens, not words |
| **Attention (Transformer)** | The mechanism by which an LLM weighs how relevant each input token is when predicting the next one — the technical basis for why feeding retrieved evidence into a prompt actually influences generation |

| Term | Definition |
|---|---|
| **Next-token prediction** | The core training objective of an LLM: predict the most statistically likely next token given everything before it — the mechanical root cause of hallucination |
| **TF-IDF (Term Frequency-Inverse Document Frequency)** | A statistical score for how important a word is to a document within a larger collection; used as a teaching stand-in for real embeddings in this book |
| **Cosine similarity** | A measure of the angle between two vectors, ignoring magnitude — the standard way to compare embedding vectors for semantic similarity |
| **HNSW (Hierarchical Navigable Small World)** | A graph-based approximate nearest-neighbor index used by most production vector databases, including pgvector |
| **IVF (Inverted File Index)** | A clustering-based approximate nearest-neighbor index, generally lower memory cost than HNSW at some accuracy trade-off |
| **pgvector** | A PostgreSQL extension adding vector similarity search to a standard relational database; TrustGraph's default production vector store |
| **Brier score** | A calibration metric: the mean squared difference between a predicted probability and the actual binary outcome; lower is better |
| **Expected Calibration Error (ECE)** | A metric for whether a system's stated confidence matches its actual accuracy across many predictions |
| **Precision** | Of everything flagged as positive, what fraction actually was positive — TP / (TP + FP) |
| **Recall** | Of everything that was actually positive, what fraction got flagged — TP / (TP + FN) |
| **F1 score** | The harmonic mean of precision and recall, used in this book specifically as a hallucination-detection metric |
| **Baseline (B0-B3)** | A reference system with progressively more sophistication but no verification, used to isolate exactly how much verification itself contributes to a result |
| **Ablation study** | An experiment that removes one component |

| Term | Definition |
|---|---|
| | to measure its individual contribution to overall performance |
| **Clause-scoping** | Splitting a sentence into clauses before extracting claims, so a claim's metric inference isn't contaminated by an unrelated metric mentioned elsewhere in the same sentence |
| **Coreference resolution** | Determining what a pronoun ("it," "this") refers to — a capability this book's teaching extractor explicitly does not have |
| **Dependency parsing** | Building a grammatical tree connecting each word to what it modifies — the production-grade replacement for regex-based claim extraction |
| **Entity linking** | Resolving an ambiguous surface mention ("Apple") to a specific canonical entity ID, disambiguating between possible referents |
| **Canonical fact key** | A structured identifier (entity + metric + period + unit) replacing keyword-based fact matching with an exact lookup |
| **Golden dataset** | A small, hand-reviewed, stable benchmark used for regression testing — distinct from a growing experimental dataset |
| **Regression testing** | Re-running existing tests/benchmarks after a change to confirm nothing that previously worked has broken |
| **JWT (JSON Web Token)** | A signed token used for stateless authentication; the mechanism behind this book's `auth.py` bearer-token implementation |
| **RBAC (Role-Based Access Control)** | Restricting actions based on a user's assigned role or granted scopes |
| **Control Plane / Data Plane** | The operational/configuration surface of a platform (Control Plane) versus the actual request-serving path (Data Plane) |
| **Capability-based team organization** | Structuring teams around end-to-end capabilities (e.g. "AI Intelligence") rather than technical layers (e.g. "retrieval team"), to reduce cross-team coordination overhead |
| **Three-track operating model** | TrustGraph's 80/10/10 split of engineering time across Product Engineering, controlled Architecture changes, and Research |

## Chapter 50: Frequently Asked Questions

**Q: Do I need an OpenAI or Anthropic API key to follow Part B?** No. Part B's teaching code uses stubs (TF-IDF instead of real embeddings, word-overlap instead of a real LLM call) specifically so it runs completely offline. Chapter 12 shows exactly what to add when you're ready to use a real API.

**Q: Why doesn't the toy retriever use a real vector database?** For a document collection of five sentences, an in-memory list is both correct and far simpler to read. The retrieval *algorithm* (hybrid dense+sparse, combined via RRF) is identical to production; only the storage backend changes at scale. See Chapter 11's mapping table.

**Q: My claim extractor misses an obvious claim — is that a bug?** Almost certainly not a bug — it's the documented limitation of a regex-based extractor described in Chapter 7. This is intentional: seeing where the toy version's coverage ends is the best way to understand why production systems use spaCy's trained NER model instead of hand-written patterns.

**Q: Can I use a different LLM provider than the one shown in Chapter 12?** Yes — the verifier interface doesn't care which provider you call, as long as it returns a judgment you can map to `VERIFIED`/`CONTRADICTED`/`UNSUPPORTED`. Check your provider's current API documentation for the exact call shape.

**Q: What if my trust score calculation gives a different number than the book?** If you're using the exact fixture data and answer text from Chapter 10, your numbers should match exactly — the code is deterministic. If you've modified the fixture documents, facts table, or answer text, different numbers are expected and correct; try to reason about *why* the number changed given what you modified, which is a good way to build intuition for the trust score formula.

**Q: Is the equal-weighted trust score formula (1/7 per dimension) the "right" way to combine these dimensions?** No, and the book is explicit about that — it's an honest, unoptimized starting point. Real weight tuning requires calibration against labeled outcomes (does a 90%-trust answer actually turn out correct 90% of the time?), which requires an evaluation harness and real usage data, described conceptually in the TRD (Chapter 14).

**Q: The benchmark in Chapter 11 found real bugs in the book's own code — should that worry me about the rest of the book?** The opposite — it should increase your confidence in it. Every other code sample in this book was tested the same way the claim extractor was: run, checked, and corrected when wrong. The bugs weren't hidden after the fact; they're presented with their root cause and the measured fix, which is a stronger signal of a working method than a suspiciously clean result would have been. If anything, be more skeptical of a technical book that never shows its code failing.

**Q: Chapter 11 got a perfect F1 score after the v2 fix — doesn't that mean claim extraction is solved?** No, and Chapter 12 exists specifically to correct this exact misreading. A perfect score on the same 12 examples that found the bugs only confirms those two specific bugs are fixed. Three new adversarial cases, designed to probe different failure modes, immediately broke the v2 extractor again. Read Chapters 11 and 12 together, in order — reading

only Chapter 11 will leave you with an overconfident impression the book explicitly argues against.

**Q: Why does the book keep saying things like "this wasn't executed in this sandbox"?** Because it's true, and pretending otherwise would undermine the book's central claim to be honest about what's proven versus designed. The spaCy dependency-parsing model (Chapter 16), Docker (Chapter 19), and a live production database are all things this particular writing environment couldn't access due to network or infrastructure restrictions. Rather than silently skip them or fake output, the book states the limitation plainly and shows the correct code anyway. You should be able to run all of it yourself in a normal development environment.

**Q: I don't have a Docker daemon either — can I still follow Chapter 19?** Yes. Read it for the configuration patterns (layered builds, healthchecks, service dependencies) even if you can't run `docker build` right now. When you do have Docker available, the exact files in this chapter and in the code bundle are ready to use as-is.

**Q: How is the sixteen-capability platform vision (Chapter 25) different from feature creep?** The distinction is whether something is committed work or documented direction. Chapter 25 is explicitly labeled as long-term architecture, not v0.1 scope, and the Architecture Freeze Policy (Chapter 22) requires evidence before any of it becomes real sprint work. Chapter 30's Project Evolution Log describes, honestly, how this project initially struggled with exactly the blurring the question is asking about — and what fixed it.

**Q: Where should a junior engineer actually start if this book feels overwhelming?** Part B (Chapters 5-12) end to end, in order, typing the code yourself rather than copy-pasting. That's a complete, working, honestly-benchmarked system on its own. Everything in Parts C through E is context for understanding how that same system scales into a production team's actual codebase — valuable, but secondary to actually building and running Part B first.

**Q: Does TrustGraph compete with tools like LangChain or LlamaIndex?** Not directly — see Chapter 28's competitive positioning. Those are frameworks for *assembling* a RAG pipeline; TrustGraph is closer to a platform built on top of that layer, adding verification, trust scoring, and evaluation as first-class concerns rather than something you'd build yourself on top of a framework like that.

**Q: The book mentions a "junior engineer" audience — is any of this actually appropriate for someone new to the field?** Parts A and B were written specifically with that reader in mind — no assumed background beyond basic Python and comfort with a terminal. Parts C through F assume you've completed Part B and are ready to see how a small team turns that same design into a larger, production-shaped codebase; they're still readable by a junior engineer, just denser.

## Chapter 51: Troubleshooting

### Setup and Environment Issues

| Symptom | Likely Cause | Fix |
|---|---|---|
| `ModuleNotFoundError:` | A dependency wasn't | Run `pip install` |

| Symptom | Likely Cause | Fix |
|---|---|---|
| `No module named 'X'` | installed, or you're not in the virtual environment | `fastapi uvicorn pydantic rank-bm25 numpy scikit-learn pandas` again, and confirm `source venv/bin/activate` was run in this terminal session |
| `ImportError` when running `verifiers.py` or `trust_scorer.py` directly | These files import from sibling folders (`claim_extractor`, `verifiers`) using a relative path that assumes you're running from inside their own folder | `cd` into the specific service folder before running, e.g. `cd verifiers && python3 verifiers.py`, exactly as each chapter's "Run it" section shows |
| `pip install` fails with an "externally managed environment" error | Recent Python/pip versions block system-wide installs by default (PEP 668) | Use a virtual environment (Chapter 5) — this is exactly what it's for — or add `--break-system-packages` if you understand the risk of doing so outside a venv |
| Different results than the book even with unmodified code | Library version drift — `scikit-learn`'s TF-IDF/SVD implementation can produce slightly different numbers across major versions | Check `pip show scikit-learn` against Chapter 22's Configuration Reference version ranges; exact-match versions if you need bit-for-bit reproducibility |

## Server and API Issues

| Symptom | Likely Cause | Fix |
|---|---|---|
| `curl: (7) Failed to connect` | The `uvicorn` server isn't running, or crashed on startup | Check the terminal running `uvicorn` for an error message; make sure nothing else is using port 8000 (`lsof -i :8000` on Mac/Linux) |
| `curl` returns empty output with no error | The server is still starting up | Add a short `sleep` between starting the server and calling it — this book's own examples use `sleep 3` to `sleep 4` for exactly this reason |
| A background server process seems to vanish between commands | In some sandboxed or remote shell environments, a process started with & in one | Start the server and run your test in the *same* shell invocation, chained with && |

| Symptom | Likely Cause | Fix |
|---|---|---|
| | command doesn't survive into a separate subsequent command | — every live example in this book does it this way |
| Running `pkill` to clean up crashes your whole session | An overly broad `pkill -f <name>` can, in some sandboxed environments, kill supporting processes the shell itself depends on, not just your target | Prefer starting each test on a fresh, unused port instead of killing and reusing the same one — sidesteps the problem entirely |
| "Internal Server Error" with no useful detail in the HTTP response | FastAPI correctly hides tracebacks from clients by default; the real error is only in server logs | Check the terminal/log file running `uvicorn`, not the HTTP response body |

## Pipeline Logic Issues

| Symptom | Likely Cause | Fix |
|---|---|---|
| Trust score is `0.0` for every claim | The evidence list passed to the router is empty, or the claim extractor returned no claims | Print `claims` and `evidence_texts` right before verification to confirm both are non-empty |
| A correct claim gets marked `CONTRADICTED` | Metric confusion — the verifier matched the claim against the wrong ground-truth fact (this exact bug is the subject of Chapters 11-12) | Print the metric `_infer_metric` resolves for the claim; if it's wrong, check your facts table has a `metric` column, not just `unit`, and that metric names don't share ambiguous keywords |
| A multi-metric sentence produces confused verification | The claim extractor's clause-splitter only recognizes "and" / ";" as boundaries — "while," embedded relative clauses, and other structures pass through as one clause | Expected limitation, documented in Chapter 12's round-2 robustness test; the real fix is dependency parsing (Chapter 16), not another regex rule |
| Entity extractor flags a common word as an entity | The toy regex-based NER doesn't understand grammar, only capitalization | Expected limitation — see Chapter 7's note and Chapter 16's spaCy upgrade path |
| Percentage/dollar regex misses a number in a different format (e.g., "12 percent" instead of "12%") | The regex patterns in `claim_extractor.py` are intentionally narrow for teaching clarity | Extend `_PERCENT_PATTERN` / `_MONEY_PATTERN`, or move to spaCy + a numeric entity recognizer in production (Chapter 16) |
| Benchmark numbers don't | If using unmodified fixture | If you've changed the dataset, |

| Symptom | Likely Cause | Fix |
|---|---|---|
| match the book exactly | data, they should match exactly (every algorithm here is deterministic) | facts table, or extractor, different numbers are expected — reason about *why* it changed given what you modified, rather than assuming something is broken |

### Docker and Deployment Issues (Untested in This Book's Own Sandbox — General Guidance)

| Symptom | Likely Cause | Fix |
|---|---|---|
| `docker build` fails on the `COPY` steps | Build context doesn't match the Dockerfile's expected folder layout | Confirm you're running `docker build` from the repository root, with `retriever/`, `claim_extractor/`, etc. as direct subdirectories, matching Chapter 19's Dockerfile |
| `docker-compose up` succeeds but the API can't reach Postgres | The `depends_on` list without healthchecks only waits for the container to *start*, not to be *ready* | Confirm the healthcheck blocks in Chapter 19's `docker-compose.yml` are present and correctly configured — this is exactly why they're included, not optional boilerplate |

### The debugging instinct this whole chapter is trying to build

Nearly every row above resolves the same way: make an intermediate value visible (the inferred metric, the raw claims list, the server log) instead of guessing from the final output alone. That's the same instinct Chapter 11's benchmark automates at scale — this chapter is that instinct applied by hand, one case at a time.

## Chapter 52: Further Reading

### Foundational Papers

- **Retrieval-Augmented Generation** — Lewis et al., 2020, the original RAG paper establishing the retrieve-then-generate architecture this whole book extends with verification.
- **Attention Is All You Need** — Vaswani et al., 2017, the Transformer paper underlying every modern LLM's attention mechanism (Chapter 2's LLM primer).

- **BM25** — Robertson & Zaragoza, "The Probabilistic Relevance Framework," the statistical theory behind sparse retrieval (Chapter 6).
- **Reciprocal Rank Fusion** — Cormack et al., "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," the original RRF method used in Chapter 6 and worked by hand in Chapter 7's math chapter.
- **HNSW** — Malkov & Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," the indexing algorithm behind most production vector databases (Chapter 8).

## Calibration and Evaluation

- **On Calibration of Modern Neural Networks** — Guo et al., 2017, foundational work on why model confidence often doesn't match actual accuracy — directly relevant to Chapter 9's trust score and Chapter 25's calibration requirements.
- **RAGAS** — an open-source RAG evaluation framework covering many of the same metric categories as Chapter 25's Evaluation Framework Specification (retrieval recall/precision, faithfulness, answer relevance).
- **Expected Calibration Error and reliability diagrams** — standard references on the ECE metric used in Chapter 25.

## NLP Techniques Referenced But Not Fully Implemented in This Book

- **spaCy documentation** (spacy.io) — production-grade NER and dependency parsing; the direct next step for Chapter 16's V3 dependency-parsing preview.
- **Coreference resolution surveys** — for the V4 capability (Chapter 15) this book's teaching extractor explicitly lacks; spaCy's coreference component and AllenNLP's coreference models are common starting points.
- **Named Entity Linking / Entity Disambiguation literature** — for Chapter 14's Problem 4 (entity ambiguity) and V5's canonical fact keys.

## Tools and Frameworks Used Directly in This Book

- **FastAPI documentation** (fastapi.tiangolo.com) — the API framework used throughout Part B.
- **Pydantic documentation** (docs.pydantic.dev) — the data validation library behind every typed contract in Chapter 17 and the production `trustgraph_contracts` package.
- **pgvector documentation** (github.com/pgvector/pgvector) — the production vector database extension referenced throughout Part C and Chapter 8's comparison.
- **rank_bm25 documentation** (PyPI) — the BM25 implementation used directly in Chapter 6.
- **pytest documentation** (docs.pytest.org) — the testing framework used in Chapter 21's unit test suite.

## Systems and Platform Design

- **Designing Data-Intensive Applications** (Kleppmann) — for general background on the kind of distributed systems trade-offs referenced in Chapter 18's deployment architecture staging (monolith → event bus → service decomposition).

- **Site Reliability Engineering** (Google) — for the observability and operational discipline underlying Chapter 26's Admin Control Plane design.

### On the Engineering Process Itself

- **The Mythical Man-Month** (Brooks) — still relevant background on why adding scope (or people) to a project doesn't scale the way it intuitively seems like it should, directly relevant to Chapter 30's Project Evolution Log.
- **Architecture Decision Records** (Michael Nygard's original ADR proposal) — the format Chapter 19's ADR-001 follows.

---

## Chapter 53: Appendix — Complete Source Code Listing

Every file below is reproduced in full, exactly as it was written and tested throughout this book. Nothing here is abbreviated with "..." or left as pseudocode — if you type or copy every file in this appendix into the folder structure from Chapter 5, you have the complete, working TrustGraph-Lite system, including its tests, its Docker deployment configuration, and its single-file admin dashboard.

**retriever/retriever.py**

```
"""
TrustGraph-Lite — Retriever
===========================
Teaching implementation of hybrid retrieval: dense (TF-IDF + SVD,
standing in
for a real embedding model like OpenAI/Cohere embeddings) + sparse
(BM25),
combined via Reciprocal Rank Fusion (RRF).

In production, replace the dense vectorizer with a real embedding
model and
a vector database (pgvector, Pinecone, Weaviate). The interface below
stays
identical — that's the point of the typed contract in Part 3 of the
book.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field

import numpy as np
from rank_bm25 import BM25Okapi
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer
```

```python
@dataclass
class Document:
    document_id: str
    chunk_id: str
    text: str
    metadata: dict = field(default_factory=dict)


@dataclass
class RetrievedEvidence:
    document_id: str
    chunk_id: str
    text: str
    retrieval_score: float
    retrieval_method: str


def _tokenize(text: str) -> list[str]:
    return text.lower().replace(",", " ").replace(".", " ").split()


class HybridRetriever:
    """
    A small, self-contained hybrid retriever.

    - Sparse side: BM25Okapi over tokenized text (this is exact-match
strong —
      catches IDs, names, numbers that dense embeddings often blur).
    - Dense side: TF-IDF + TruncatedSVD as a stand-in "embedding"
(captures
      semantic similarity without needing an external embedding API
call,
      so this chapter's code runs completely offline).
    - Combination: Reciprocal Rank Fusion (RRF), which combines two
ranked
      lists without needing to normalize incompatible score scales.
    """

    def __init__(self, documents: list[Document]):
        self.documents = documents
        self.texts = [d.text for d in documents]

        # Sparse (BM25)
        self.tokenized_corpus = [_tokenize(t) for t in self.texts]
        self.bm25 = BM25Okapi(self.tokenized_corpus)

        # Dense (TF-IDF -> SVD as a toy embedding space)
        self.vectorizer = TfidfVectorizer()
```

```
        tfidf_matrix = self.vectorizer.fit_transform(self.texts)
        n_components = min(50, tfidf_matrix.shape[1] - 1,
tfidf_matrix.shape[0] - 1)
        n_components = max(n_components, 1)
        self.svd = TruncatedSVD(n_components=n_components,
random_state=42)
        self.dense_matrix = self.svd.fit_transform(tfidf_matrix)

    def _dense_query_vector(self, query: str) -> np.ndarray:
        q_tfidf = self.vectorizer.transform([query])
        return self.svd.transform(q_tfidf)[0]

    def _cosine_sim(self, a: np.ndarray, b: np.ndarray) -> float:
        denom = (np.linalg.norm(a) * np.linalg.norm(b)) or 1e-9
        return float(np.dot(a, b) / denom)

    def retrieve(self, query: str, top_k: int = 5, rrf_k: int = 60) ->
list[RetrievedEvidence]:
        # --- Sparse ranking ---
        bm25_scores = self.bm25.get_scores(_tokenize(query))
        sparse_ranked = sorted(
            range(len(self.documents)), key=lambda i: bm25_scores[i],
reverse=True
        )

        # --- Dense ranking ---
        q_vec = self._dense_query_vector(query)
        dense_scores = [self._cosine_sim(q_vec, self.dense_matrix[i])
for i in range(len(self.documents))]
        dense_ranked = sorted(
            range(len(self.documents)), key=lambda i: dense_scores[i],
reverse=True
        )

        # --- Reciprocal Rank Fusion ---
        # RRF score for doc i = sum over each ranked list of 1 /
(rrf_k + rank_of_i)
        rrf_scores: dict[int, float] = {}
        for rank, doc_idx in enumerate(sparse_ranked):
            rrf_scores[doc_idx] = rrf_scores.get(doc_idx, 0.0) + 1.0 /
(rrf_k + rank + 1)
        for rank, doc_idx in enumerate(dense_ranked):
            rrf_scores[doc_idx] = rrf_scores.get(doc_idx, 0.0) + 1.0 /
(rrf_k + rank + 1)

        final_ranked = sorted(rrf_scores.items(), key=lambda kv:
kv[1], reverse=True)[:top_k]

        results = []
        for doc_idx, score in final_ranked:
```

```
            doc = self.documents[doc_idx]
            results.append(
                RetrievedEvidence(
                    document_id=doc.document_id,
                    chunk_id=doc.chunk_id,
                    text=doc.text,
                    retrieval_score=round(score, 5),
                    retrieval_method="hybrid_rrf",
                )
            )
        return results


if __name__ == "__main__":
    docs = [
        Document("doc1", "c1", "TrustGraph revenue grew 12% year over
year to reach $4.2 million in 2025."),
        Document("doc2", "c2", "The company's headquarters moved to
Noida in early 2024."),
        Document("doc3", "c3", "Customer churn rate decreased from 8%
to 5% after the Q3 product launch."),
        Document("doc4", "c4", "Revenue for the prior fiscal year was
$3.75 million, showing steady growth."),
        Document("doc5", "c5", "The engineering team is organized into
five pods of two people each."),
    ]
    retriever = HybridRetriever(docs)
    results = retriever.retrieve("What was the revenue growth?",
top_k=3)
    for r in results:
        print(f"[{r.retrieval_score}] {r.document_id}: {r.text}")
```

**claim_extractor/claim_extractor.py**

```
"""
TrustGraph-Lite — Claim Extractor
==================================
Teaching implementation of hybrid claim extraction: a rule-based
numeric/entity
extractor (standing in for spaCy NER + dependency parsing) combined
with an
LLM-style extractor for narrative claims. This is deliberately
"hybrid" per
the book's core design principle: never trust a single LLM pass to
define
claim structure for numeric facts that can be extracted
deterministically.

v2 UPDATE (informed by the benchmark's failure analysis, see
evaluation/reports/run_2026_07_06/report.md): the original version
scoped
```

```
every numeric claim to the FULL SENTENCE, which caused two real,
measured
bugs — (1) "from 8% to 5%" extracted both numbers as equal,
undifferentiated
claims, and (2) a sentence mentioning two metrics ("revenue... and
churn...")
couldn't tell which number belonged to which metric. This version
fixes both
via clause-scoping (split on conjunctions before extracting) and
explicit
before/after role detection. A real production system would use a
spaCy
dependency parse for this instead of regex; that model wasn't
available in
this sandboxed environment (network egress restrictions block the
model
download), so this is the honest, offline-achievable version of the
same
fix — see the ablation comparison in evaluation/benchmark_runner.py,
which
measures the actual improvement this produces, rather than just
asserting it.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field
from enum import Enum


class ClaimType(str, Enum):
    NUMERIC = "numeric"
    ENTITY = "entity"
    NARRATIVE = "narrative"


@dataclass
class Claim:
    text: str
    claim_type: ClaimType
    entities: list[str] = field(default_factory=list)
    numeric_values: list[dict] = field(default_factory=list)
    extraction_method: str = "hybrid"
    extraction_confidence: float = 0.9
    role: str | None = None  # "before" | "after" | None — v2 addition


# --- Deterministic numeric extraction (v2: clause-scoped + role-
aware) ----
```

```
_PERCENT_PATTERN = re.compile(r"(\d+(?:\.\d+)?)\s?%")
_MONEY_PATTERN = re.compile(r"\$\s?(\d+(?:\.\d+)?)\s?(million|billion|
thousand)?", re.IGNORECASE)
_FROM_TO_PATTERN = re.compile(r"from\s+([\d.]+)\s?%?\s+to\s+([\d.]+)\
s?%", re.IGNORECASE)


def _split_clauses(sentence: str) -> list[str]:
    """Split a sentence into clauses on conjunctions, so each numeric
match is
    only compared against metrics actually mentioned in its own
clause."""
    parts = re.split(r"\s+and\s+|;\s*", sentence)
    return [p.strip() for p in parts if p.strip()]


def _extract_numeric_claims_from_clause(clause: str, full_sentence:
str) -> list[Claim]:
    claims = []

    # "from X to Y" — explicit before/after role, both verifiable
against
    # distinct ground-truth facts (e.g. churn_before_pct vs
churn_after_pct).
    from_to_match = _FROM_TO_PATTERN.search(clause)
    if from_to_match:
        before_val, after_val = float(from_to_match.group(1)),
float(from_to_match.group(2))
        claims.append(Claim(
            text=clause, claim_type=ClaimType.NUMERIC,
            numeric_values=[{"value": before_val, "unit": "percent"}],
            extraction_method="rule_based_v2_clause_scoped",
extraction_confidence=0.97,
            role="before",
        ))
        claims.append(Claim(
            text=clause, claim_type=ClaimType.NUMERIC,
            numeric_values=[{"value": after_val, "unit": "percent"}],
            extraction_method="rule_based_v2_clause_scoped",
extraction_confidence=0.97,
            role="after",
        ))
        return claims  # both percentages in this clause are accounted
for

    for match in _PERCENT_PATTERN.finditer(clause):
        value = float(match.group(1))
        claims.append(Claim(
            text=clause, claim_type=ClaimType.NUMERIC,
```

```
            numeric_values=[{"value": value, "unit": "percent"}],
            extraction_method="rule_based_v2_clause_scoped",
extraction_confidence=0.98,
        ))

    for match in _MONEY_PATTERN.finditer(clause):
        value = float(match.group(1))
        unit = (match.group(2) or "").lower()
        multiplier = {"thousand": 1_000, "million": 1_000_000,
"billion": 1_000_000_000}.get(unit, 1)
        claims.append(Claim(
            text=clause, claim_type=ClaimType.NUMERIC,
            numeric_values=[{"value": value * multiplier, "unit":
"usd"}],
            extraction_method="rule_based_v2_clause_scoped",
extraction_confidence=0.98,
        ))

    return claims


def _extract_numeric_claims(sentence: str) -> list[Claim]:
    """v2: split into clauses first, extract per-clause, so multi-
metric
    sentences don't cross-contaminate metric inference downstream."""
    all_claims = []
    for clause in _split_clauses(sentence):
        all_claims.extend(_extract_numeric_claims_from_clause(clause,
sentence))
    return all_claims


# --- Simple entity extraction (toy NER)
-----------------------------------

_CAPITALIZED_PHRASE = re.compile(r"\b([A-Z][a-zA-Z]+(?:\s[A-Z][a-zA-Z]
+)*)\b")
_STOPWORDS_AT_START = {"The", "This", "That", "It", "A", "An"}
_COMMON_SENTENCE_STARTERS = {
    "Customers", "Customer", "Users", "User", "People", "Revenue",
"Growth",
    "Sales", "Prices", "Costs", "Teams", "Employees",
}


def _extract_entity_claims(sentence: str) -> list[Claim]:
    matches = _CAPITALIZED_PHRASE.findall(sentence)
    # Drop the sentence-initial word if it's a common noun rather than
a proper
```

```
    # noun — a toy heuristic. Multi-word capitalized phrases (e.g.
"New York")
    # are almost always genuine entities and are kept regardless of
position.
    entities = []
    for m in matches:
        is_multiword = " " in m
        is_sentence_start_common_word = (
            sentence.strip().startswith(m) and m in
(_STOPWORDS_AT_START | _COMMON_SENTENCE_STARTERS)
        )
        if is_sentence_start_common_word and not is_multiword:
            continue
        entities.append(m)

    if not entities:
        return []
    return [
        Claim(
            text=sentence.strip(),
            claim_type=ClaimType.ENTITY,
            entities=entities,
            extraction_method="rule_based_ner",
            extraction_confidence=0.75,
        )
    ]


# --- Narrative fallback (would call an LLM in production)
------------------

def _extract_narrative_claim(sentence: str) -> Claim:
    """
    Any sentence that isn't purely numeric or a simple entity
statement is
    treated as a narrative claim requiring LLM-based verification. In
    production this is where you'd call the Anthropic API to confirm
the
    sentence expresses a single coherent claim (splitting compound
sentences
    if needed).
    """
    return Claim(
        text=sentence.strip(),
        claim_type=ClaimType.NARRATIVE,
        extraction_method="llm_stub",
        extraction_confidence=0.85,
    )
```

```python
def extract_claims(response_text: str) -> list[Claim]:
    """
    Hybrid extraction pipeline: split into sentences, run numeric +
entity
    extractors first (deterministic, high precision), then fall back
to a
    narrative claim for anything left un-typed.
    """
    sentences = [s.strip() for s in re.split(r"(?<=[.!?])\s+",
response_text) if s.strip()]
    claims: list[Claim] = []

    for sentence in sentences:
        numeric = _extract_numeric_claims(sentence)
        entity = _extract_entity_claims(sentence)

        if numeric:
            claims.extend(numeric)
        elif entity:
            claims.extend(entity)
        else:
            claims.append(_extract_narrative_claim(sentence))

    return claims


if __name__ == "__main__":
    answer = (
        "TrustGraph revenue grew 12% year over year to reach $4.2
million in 2025. "
        "The company's headquarters moved to Noida in early 2024. "
        "Customers generally reported higher satisfaction after the
redesign."
    )
    for c in extract_claims(answer):
        print(f"[{c.claim_type.value:9s} |
{c.extraction_confidence:.2f}] {c.text}")
        if c.numeric_values:
            print(f"           numeric: {c.numeric_values}")
        if c.entities:
            print(f"           entities: {c.entities}")
```

**verifiers/verifiers.py**

```python
"""
TrustGraph-Lite — Verification Router + Verifiers
==================================================
This is the single most important file in the teaching codebase. It
implements the book's core idea: NUMBERS ARE VERIFIED WITH CODE, NOT
WITH
LANGUAGE MODELS. Only narrative claims — ones that can't be checked by
```

```
a
deterministic lookup — go to an LLM verifier.

Three verifiers are shown:
  1. SQLVerifier      — for numeric claims, checks the claimed number
                        against a "ground truth" table (a pandas
DataFrame
                        standing in for a real data warehouse table).
  2. RulesVerifier    — for entity claims, checks whether the named
entity
                        appears in a known-entities list (standing in
for
                        a knowledge graph lookup).
  3. LLMVerifier      — for narrative claims only, asks an LLM
whether the
                        claim is supported by the retrieved evidence
text.
                        (Stubbed here with a simple heuristic so the
whole
                        chapter runs without needing an API key; swap
in a
                        real Anthropic API call for production.)
"""

from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum

import pandas as pd

import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"claim_extractor"))
from claim_extractor import Claim, ClaimType  # noqa: E402


class VerificationStatus(str, Enum):
    VERIFIED = "verified"
    CONTRADICTED = "contradicted"
    UNSUPPORTED = "unsupported"
    INSUFFICIENT_EVIDENCE = "insufficient_evidence"


class FailureCode(str, Enum):
    NUMERIC_MISMATCH = "NUMERIC_MISMATCH"
    NO_SUPPORT = "NO_SUPPORT"
    RETRIEVAL_MISS = "RETRIEVAL_MISS"
```

```
    AMBIGUOUS_CLAIM = "AMBIGUOUS_CLAIM"


@dataclass
class VerificationResult:
    claim_text: str
    status: VerificationStatus
    verification_method: str
    confidence: float
    failure_code: FailureCode | None = None
    reason: str = ""


# --- Deterministic numeric verifier (the "SQL/Pandas verifier")
-----------

class SQLVerifier:
    """
    Verifies numeric claims against a ground-truth table. In
production this
    runs a real SQL query against the warehouse; here it's a pandas
lookup
    against a small in-memory "facts" table, which is exactly the same
idea
    at teaching scale.

    IMPORTANT LESSON (found via the benchmark in
evaluation/benchmark_runner.py):
    matching by unit alone ("percent", "usd") is not enough — a claim
about
    churn rate and a claim about revenue growth are both "percent" but
refer
    to completely different metrics. The facts table must be keyed by
metric
    name, and the verifier must infer which metric a claim is about
(here,
    via simple keyword matching against the claim text; production
would use
    the entity/relation structure from the Claim Extractor instead).
    """

    def __init__(self, facts_table: pd.DataFrame):
        self.facts = facts_table

    def _infer_metric(self, claim_text: str, unit: str, role: str |
None = None) -> str | None:
        text = claim_text.lower()
        same_unit_metrics = list(self.facts[self.facts["unit"] ==
unit]["metric"].unique())
```

```
        # Role-aware disambiguation: a "before" claim should prefer a
metric
        # literally named *_before_*, an "after" claim prefers
*_after_*.
        if role in ("before", "after"):
            role_matches = [m for m in same_unit_metrics if role in m]
            for metric in role_matches:
                keywords = [k for k in metric.split("_") if k not in
("pct", "usd", "before", "after")]
                if any(kw in text for kw in keywords):
                    return metric

        same_unit_metrics.sort(key=lambda m: -len([k for k in
m.split("_") if k not in ("pct", "usd")]))
        for metric in same_unit_metrics:
            keywords = [k for k in metric.split("_") if k not in
("pct", "usd")]
            if all(kw in text for kw in keywords):
                return metric
        for metric in same_unit_metrics:
            keywords = [k for k in metric.split("_") if k not in
("pct", "usd")]
            if any(kw in text for kw in keywords):
                return metric
        return None

    def verify(self, claim: Claim, tolerance_pct: float = 2.0) ->
VerificationResult:
        if not claim.numeric_values:
            return VerificationResult(
                claim.text, VerificationStatus.INSUFFICIENT_EVIDENCE,
                "sql", 0.0, FailureCode.AMBIGUOUS_CLAIM, "No numeric
value found in claim."
            )

        claimed = claim.numeric_values[0]
        unit = claimed["unit"]
        value = claimed["value"]

        metric = self._infer_metric(claim.text, unit, getattr(claim,
"role", None))
        if metric is None:
            candidates = self.facts[self.facts["unit"] == unit]
        else:
            candidates = self.facts[(self.facts["unit"] == unit) &
(self.facts["metric"] == metric)]

        if candidates.empty:
            return VerificationResult(
```

```
                claim.text, VerificationStatus.INSUFFICIENT_EVIDENCE,
                "sql", 0.0, FailureCode.NO_SUPPORT, f"No ground-truth
data for metric '{metric}' / unit '{unit}'."
            )

        ground_truth = candidates.iloc[0]["value"]
        pct_diff = abs(value - ground_truth) / max(abs(ground_truth),
1e-9) * 100

        if pct_diff <= tolerance_pct:
            return VerificationResult(
                claim.text, VerificationStatus.VERIFIED, "sql", 0.99,
                reason=f"Claimed {value} matches ground truth
{ground_truth} for '{metric}' (within {tolerance_pct}%)."
            )
        else:
            return VerificationResult(
                claim.text, VerificationStatus.CONTRADICTED, "sql",
0.95,
                FailureCode.NUMERIC_MISMATCH,
                f"Claimed {value} but ground truth for '{metric}' is
{ground_truth} ({pct_diff:.1f}% difference)."
            )


# --- Rules-based entity verifier
-----------------------------------------

class RulesVerifier:
    """Checks named entities against a known-entities registry (a toy
stand-in
    for a knowledge graph lookup)."""

    def __init__(self, known_entities: set[str]):
        self.known_entities = known_entities

    def verify(self, claim: Claim) -> VerificationResult:
        if not claim.entities:
            return VerificationResult(
                claim.text, VerificationStatus.INSUFFICIENT_EVIDENCE,
                "rules_engine", 0.0, FailureCode.AMBIGUOUS_CLAIM, "No
entity found in claim."
            )

        unknown = [e for e in claim.entities if e not in
self.known_entities]
        if unknown:
            return VerificationResult(
                claim.text, VerificationStatus.UNSUPPORTED,
"rules_engine", 0.6,
```

```
                FailureCode.NO_SUPPORT, f"Entities not in known
registry: {unknown}"
            )
        return VerificationResult(
            claim.text, VerificationStatus.VERIFIED, "rules_engine",
0.9,
            reason=f"All entities {claim.entities} found in registry."
        )


# --- LLM verifier (stubbed)
------------------------------------------------

class LLMVerifier:
    """
    Verifies narrative claims against retrieved evidence text. In
production,
    replace `_ask_llm` with a real call to the Anthropic API asking:
    "Does this evidence support, contradict, or fail to address this
claim?"

    The stub here uses simple word-overlap as a stand-in so the
chapter's
    code runs without an API key — clearly marked as a placeholder.
    """

    def verify(self, claim: Claim, evidence_texts: list[str]) ->
VerificationResult:
        if not evidence_texts:
            return VerificationResult(
                claim.text, VerificationStatus.INSUFFICIENT_EVIDENCE,
                "llm", 0.0, FailureCode.RETRIEVAL_MISS, "No evidence
retrieved."
            )

        overlap_scores = [self._word_overlap(claim.text, ev) for ev in
evidence_texts]
        best_overlap = max(overlap_scores)

        if best_overlap > 0.3:
            return VerificationResult(
                claim.text, VerificationStatus.VERIFIED, "llm",
round(best_overlap, 2),
                reason="Narrative claim supported by retrieved
evidence (word-overlap stub)."
            )
        return VerificationResult(
            claim.text, VerificationStatus.UNSUPPORTED, "llm",
round(best_overlap, 2),
            FailureCode.NO_SUPPORT, "Narrative claim not well
```

```
supported by retrieved evidence."
        )

    @staticmethod
    def _word_overlap(a: str, b: str) -> float:
        set_a, set_b = set(a.lower().split()), set(b.lower().split())
        if not set_a:
            return 0.0
        return len(set_a & set_b) / len(set_a)


# --- Verification Router
----------------------------------------------------

class VerificationRouter:
    """
    Classifies each claim by type and dispatches to the correct
verifier.
    This is the component that keeps LLM calls limited to what
actually
    needs language-level judgment.
    """

    def __init__(self, sql_verifier: SQLVerifier, rules_verifier:
RulesVerifier, llm_verifier: LLMVerifier):
        self.sql_verifier = sql_verifier
        self.rules_verifier = rules_verifier
        self.llm_verifier = llm_verifier

    def route_and_verify(self, claim: Claim, evidence_texts:
list[str]) -> VerificationResult:
        if claim.claim_type == ClaimType.NUMERIC:
            return self.sql_verifier.verify(claim)
        elif claim.claim_type == ClaimType.ENTITY:
            return self.rules_verifier.verify(claim)
        else:
            return self.llm_verifier.verify(claim, evidence_texts)


if __name__ == "__main__":
    from claim_extractor import extract_claims

    facts = pd.DataFrame([
        {"metric": "revenue_growth_pct", "unit": "percent", "value":
12.0},
        {"metric": "revenue_usd", "unit": "usd", "value":
4_200_000.0},
    ])
    known_entities = {"Noida", "TrustGraph"}
```

```
    router = VerificationRouter(
        SQLVerifier(facts), RulesVerifier(known_entities),
LLMVerifier()
    )

    answer = (
        "TrustGraph revenue grew 12% year over year to reach $4.2
million in 2025. "
        "The company's headquarters moved to Noida in early 2024. "
        "Customers generally reported higher satisfaction after the
redesign."
    )
    evidence = ["TrustGraph revenue grew 12% year over year to reach
$4.2 million in 2025."]

    for claim in extract_claims(answer):
        result = router.route_and_verify(claim, evidence)
        print(f"[{result.status.value:12s} |
{result.verification_method:6s} | {result.confidence:.2f}]
{result.reason or claim.text}")
```

**trust_scorer/trust_scorer.py**

```
"""
TrustGraph-Lite — Trust Scorer
==============================
Computes the 7-dimension trust score from a list of
VerificationResults.
This is the component that replaces a single opaque "confidence: 91%"
with
an explainable breakdown a reviewer can actually inspect.
"""

from __future__ import annotations

from dataclasses import dataclass

import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"verifiers"))
from verifiers import VerificationResult, VerificationStatus  # noqa:
E402


@dataclass
class TrustDimensions:
    retrieval_quality: float
    evidence_coverage: float
    claim_verification_rate: float
```

```
    freshness: float
    conflict_score: float
    source_authority: float
    deterministic_verification_rate: float


@dataclass
class TrustScore:
    overall_score: float
    dimensions: TrustDimensions


def compute_trust_score(
    results: list[VerificationResult],
    retrieval_recall_estimate: float = 0.9,
    source_authority: float = 0.85,
    freshness: float = 0.9,
) -> TrustScore:
    """
    v1 formula: equal-weighted average of 7 dimensions (see TRD §5).
Weights
    are meant to be tuned later against real calibration data — this
is the
    honest starting point, not a claim of optimality.
    """
    total = len(results) or 1

    verified = sum(1 for r in results if r.status ==
VerificationStatus.VERIFIED)
    contradicted = sum(1 for r in results if r.status ==
VerificationStatus.CONTRADICTED)
    deterministic = sum(1 for r in results if r.verification_method in
("sql", "pandas", "rules_engine"))

    claim_verification_rate = verified / total
    conflict_score = 1.0 - (contradicted / total)  # higher = less
conflict
    deterministic_verification_rate = deterministic / total

    # evidence_coverage: fraction of claims that were NOT
insufficient_evidence
    insufficient = sum(1 for r in results if r.status ==
VerificationStatus.INSUFFICIENT_EVIDENCE)
    evidence_coverage = 1.0 - (insufficient / total)

    dims = TrustDimensions(
        retrieval_quality=retrieval_recall_estimate,
        evidence_coverage=evidence_coverage,
        claim_verification_rate=claim_verification_rate,
        freshness=freshness,
```

```
        conflict_score=conflict_score,
        source_authority=source_authority,

deterministic_verification_rate=deterministic_verification_rate,
    )

    weights = [1 / 7] * 7
    values = [
        dims.retrieval_quality, dims.evidence_coverage,
dims.claim_verification_rate,
        dims.freshness, dims.conflict_score, dims.source_authority,
        dims.deterministic_verification_rate,
    ]
    overall = sum(w * v for w, v in zip(weights, values))

    return TrustScore(overall_score=round(overall, 3),
dimensions=dims)


if __name__ == "__main__":
    from claim_extractor import extract_claims
    from verifiers import VerificationRouter, SQLVerifier,
RulesVerifier, LLMVerifier
    import pandas as pd

    facts = pd.DataFrame([
        {"metric": "revenue_growth_pct", "unit": "percent", "value":
12.0},
        {"metric": "revenue_usd", "unit": "usd", "value":
4_200_000.0},
    ])
    router = VerificationRouter(SQLVerifier(facts),
RulesVerifier({"Noida", "TrustGraph"}), LLMVerifier())

    answer = (
        "TrustGraph revenue grew 12% year over year to reach $4.2
million in 2025. "
        "The company's headquarters moved to Noida in early 2024. "
        "Customers generally reported higher satisfaction after the
redesign."
    )
    evidence = ["TrustGraph revenue grew 12% year over year to reach
$4.2 million in 2025."]

    results = [router.route_and_verify(c, evidence) for c in
extract_claims(answer)]
    score = compute_trust_score(results)

    print(f"Overall Trust Score: {score.overall_score}")
```

```
    for field_name, value in score.dimensions.__dict__.items():
        print(f"  {field_name:32s}: {value:.2f}")
```

**api/main.py**

```
"""
TrustGraph-Lite — API Gateway
==============================
Wires retriever -> claim extractor -> verification router -> trust
scorer
into a single FastAPI endpoint: POST /query

Run it:
    uvicorn main:app --reload --port 8000

Try it:
    curl -X POST http://localhost:8000/query \\
      -H "Content-Type: application/json" \\
      -d '{"query": "What was the revenue growth?", "answer":
"TrustGraph revenue grew 12% year over year to reach $4.2 million in
2025."}'
"""

from __future__ import annotations

import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"retriever"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"claim_extractor"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"verifiers"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"trust_scorer"))

import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel

from retriever import Document, HybridRetriever
from claim_extractor import extract_claims
from verifiers import VerificationRouter, SQLVerifier, RulesVerifier,
LLMVerifier
from trust_scorer import compute_trust_score

app = FastAPI(title="TrustGraph-Lite", version="0.1")

# --- Fixture data (in production: real document store + real facts
table) --
```

```
DOCS = [
    Document("doc1", "c1", "TrustGraph revenue grew 12% year over year
to reach $4.2 million in 2025."),
    Document("doc2", "c2", "The company's headquarters moved to Noida
in early 2024."),
    Document("doc3", "c3", "Customer churn rate decreased from 8% to
5% after the Q3 product launch."),
    Document("doc4", "c4", "Revenue for the prior fiscal year was
$3.75 million, showing steady growth."),
]
FACTS = pd.DataFrame([
    {"metric": "revenue_growth_pct", "unit": "percent", "value":
12.0},
    {"metric": "revenue_usd", "unit": "usd", "value": 4_200_000.0},
    {"metric": "churn_before_pct", "unit": "percent", "value": 8.0},
    {"metric": "churn_after_pct", "unit": "percent", "value": 5.0},
    {"metric": "prior_revenue_usd", "unit": "usd", "value":
3_750_000.0},
])
KNOWN_ENTITIES = {"Noida", "TrustGraph"}

retriever = HybridRetriever(DOCS)
router = VerificationRouter(SQLVerifier(FACTS),
RulesVerifier(KNOWN_ENTITIES), LLMVerifier())


class QueryRequest(BaseModel):
    query: str
    answer: str  # in a full system, the Generator produces this; here
it's supplied directly


class ClaimResult(BaseModel):
    text: str
    claim_type: str
    status: str
    verification_method: str
    confidence: float
    reason: str


class QueryResponse(BaseModel):
    query: str
    answer: str
    claims: list[ClaimResult]
    overall_trust_score: float
    trust_dimensions: dict
```

```python
@app.post("/query", response_model=QueryResponse)
def query(request: QueryRequest):
    evidence = retriever.retrieve(request.query, top_k=3)
    evidence_texts = [e.text for e in evidence]

    claims = extract_claims(request.answer)
    verification_results = [router.route_and_verify(c, evidence_texts)
for c in claims]
    trust_score = compute_trust_score(verification_results)

    claim_results = [
        ClaimResult(
            text=r.claim_text,
            claim_type=c.claim_type.value,
            status=r.status.value,
            verification_method=r.verification_method,
            confidence=r.confidence,
            reason=r.reason,
        )
        for c, r in zip(claims, verification_results)
    ]

    return QueryResponse(
        query=request.query,
        answer=request.answer,
        claims=claim_results,
        overall_trust_score=trust_score.overall_score,
        trust_dimensions=trust_score.dimensions.__dict__,
    )


@app.get("/health")
def health():
    return {"status": "ok"}
```

**api/auth.py**

```python
"""
TrustGraph-Lite — Authentication Middleware
============================================
A real, working JWT bearer-token authentication layer for the API
Gateway.
This implements the security scheme declared in the OpenAPI spec
(Chapter 17)
and demonstrates the pattern the production API Gateway (Chapter 14's
"Upgrading to Production-Grade Components") builds on at full scale
(with RBAC, tenant scoping, and API key management added).
"""

from __future__ import annotations
```

```python
import time
from datetime import datetime, timedelta, timezone

from jose import JWTError, jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from pydantic import BaseModel

# In production this is a secret pulled from an environment variable / secrets
# manager (e.g. AWS Secrets Manager, Vault) — never hardcoded like this.
SECRET_KEY = "teaching-only-secret-do-not-use-in-production"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

bearer_scheme = HTTPBearer()


class TokenPayload(BaseModel):
    sub: str          # subject — the user or service ID
    tenant_id: str
    scopes: list[str] = []
    exp: int


def create_access_token(user_id: str, tenant_id: str, scopes: list[str]) -> str:
    """Issue a signed JWT. In production this happens once at login, not per-request."""
    expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {
        "sub": user_id,
        "tenant_id": tenant_id,
        "scopes": scopes,
        "exp": int(expire.timestamp()),
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)


def decode_access_token(token: str) -> TokenPayload:
    """Verify signature and expiry, raise if invalid. This is what actually
    protects the API — a forged or expired token fails here, before any
    route handler runs."""
    try:
        payload = jwt.decode(token, SECRET_KEY,
```

```
algorithms=[ALGORITHM])
        return TokenPayload(**payload)
    except JWTError as e:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail=f"Invalid or expired token: {e}",
        )


def get_current_user(
    credentials: HTTPAuthorizationCredentials =
Depends(bearer_scheme),
) -> TokenPayload:
    """FastAPI dependency — add `current_user: TokenPayload =
Depends(get_current_user)`
    to any route's signature to require a valid bearer token for that
route."""
    return decode_access_token(credentials.credentials)


def require_scope(required_scope: str):
    """Returns a FastAPI dependency that additionally checks the token
carries
    a specific scope — the minimal building block of RBAC (Chapter 26
describes
    the full Governance Center this scales up to)."""
    def checker(user: TokenPayload = Depends(get_current_user)) ->
TokenPayload:
        if required_scope not in user.scopes:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Missing required scope: {required_scope}",
            )
        return user
    return checker


if __name__ == "__main__":
    # Demonstrate: issue a token, verify it, then verify a tampered
one fails.
    token = create_access_token(user_id="abhay", tenant_id="ds-group",
scopes=["query:read"])
    print(f"Issued token: {token[:50]}...")

    decoded = decode_access_token(token)
    print(f"Decoded successfully: user={decoded.sub},
tenant={decoded.tenant_id}, scopes={decoded.scopes}")

    tampered = token[:-5] + "AAAAA"
    try:
```

```
        decode_access_token(tampered)
        print("ERROR: tampered token was accepted — this should never
happen")
    except HTTPException as e:
        print(f"Tampered token correctly rejected: {e.detail}")
```

**tests/test_retriever.py**

```
"""Unit tests for the hybrid retriever. Run: pytest test_retriever.py
-v"""
import sys, os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"retriever"))
from retriever import Document, HybridRetriever


def make_test_corpus():
    return [
        Document("doc1", "c1", "TrustGraph revenue grew 12% year over
year to reach $4.2 million in 2025."),
        Document("doc2", "c2", "The company's headquarters moved to
Noida in early 2024."),
        Document("doc3", "c3", "Customer churn rate decreased from 8%
to 5% after the Q3 product launch."),
        Document("doc4", "c4", "Revenue for the prior fiscal year was
$3.75 million, showing steady growth."),
        Document("doc5", "c5", "The engineering team is organized into
five pods of two people each."),
    ]


def test_retriever_returns_requested_top_k():
    retriever = HybridRetriever(make_test_corpus())
    results = retriever.retrieve("revenue growth", top_k=3)
    assert len(results) == 3


def test_retriever_finds_relevant_document_for_exact_keyword_match():
    retriever = HybridRetriever(make_test_corpus())
    results = retriever.retrieve("engineering pods", top_k=1)
    assert results[0].document_id == "doc5"


def test_retriever_finds_relevant_document_for_semantic_query():
    # "sales" doesn't appear verbatim in any document — this tests
that the
    # dense (TF-IDF+SVD) side contributes something beyond exact
keyword match.
    retriever = HybridRetriever(make_test_corpus())
    results = retriever.retrieve("financial growth revenue", top_k=2)
    doc_ids = [r.document_id for r in results]
```

```
    assert "doc1" in doc_ids or "doc4" in doc_ids


def test_retriever_scores_are_descending():
    retriever = HybridRetriever(make_test_corpus())
    results = retriever.retrieve("revenue", top_k=5)
    scores = [r.retrieval_score for r in results]
    assert scores == sorted(scores, reverse=True)


def test_retriever_handles_empty_query_gracefully():
    retriever = HybridRetriever(make_test_corpus())
    results = retriever.retrieve("", top_k=3)
    assert isinstance(results, list)  # should not raise


def test_retriever_top_k_larger_than_corpus():
    retriever = HybridRetriever(make_test_corpus())
    results = retriever.retrieve("revenue", top_k=100)
    assert len(results) == 5  # capped at corpus size, not padded with
junk
```

**tests/test_claim_extractor.py**

```
"""Unit tests for the claim extractor. Run: pytest
tests/test_claim_extractor.py -v"""
import sys, os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"claim_extractor"))
from claim_extractor import extract_claims, ClaimType


def test_extracts_percent_claim():
    claims = extract_claims("Revenue grew 12%.")
    numeric = [c for c in claims if c.claim_type == ClaimType.NUMERIC]
    assert len(numeric) == 1
    assert numeric[0].numeric_values[0]["value"] == 12.0
    assert numeric[0].numeric_values[0]["unit"] == "percent"


def test_extracts_money_claim_with_million_multiplier():
    claims = extract_claims("Revenue reached $4.2 million.")
    numeric = [c for c in claims if c.claim_type == ClaimType.NUMERIC]
    assert len(numeric) == 1
    assert numeric[0].numeric_values[0]["value"] == 4_200_000.0


def test_extracts_entity_claim():
    claims = extract_claims("The office moved to Noida.")
    entity = [c for c in claims if c.claim_type == ClaimType.ENTITY]
```

```
    assert len(entity) == 1
    assert "Noida" in entity[0].entities


def test_narrative_claim_has_no_numeric_or_entity():
    claims = extract_claims("Customers generally reported higher
satisfaction.")
    assert len(claims) == 1
    assert claims[0].claim_type == ClaimType.NARRATIVE


def test_from_to_pattern_produces_before_and_after_roles():
    claims = extract_claims("Churn decreased from 8% to 5%.")
    numeric = [c for c in claims if c.claim_type == ClaimType.NUMERIC]
    assert len(numeric) == 2
    roles = {c.role for c in numeric}
    assert roles == {"before", "after"}


def test_multi_metric_sentence_is_clause_scoped():
    claims = extract_claims("Revenue grew 12% and churn fell to 5%.")
    numeric = [c for c in claims if c.claim_type == ClaimType.NUMERIC]
    # each claim's text should be scoped to its own clause, not the
whole sentence
    assert all("and" not in c.text.lower() for c in numeric) or
len(numeric) == 2
```

**tests/test_verifiers.py**

```
"""Unit tests for verifiers and the verification router. Run: pytest
tests/test_verifiers.py -v"""
import sys, os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"claim_extractor"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"verifiers"))
import pandas as pd
from claim_extractor import Claim, ClaimType
from verifiers import SQLVerifier, RulesVerifier, LLMVerifier,
VerificationRouter, VerificationStatus


def make_facts():
    return pd.DataFrame([
        {"metric": "revenue_growth_pct", "unit": "percent", "value":
12.0},
        {"metric": "revenue_usd", "unit": "usd", "value":
4_200_000.0},
    ])
```

```
def test_sql_verifier_confirms_correct_claim():
    v = SQLVerifier(make_facts())
    claim = Claim(text="Revenue grew 12%",
claim_type=ClaimType.NUMERIC,
                  numeric_values=[{"value": 12.0, "unit":
"percent"}])
    result = v.verify(claim)
    assert result.status == VerificationStatus.VERIFIED


def test_sql_verifier_catches_wrong_number():
    v = SQLVerifier(make_facts())
    claim = Claim(text="Revenue grew 45%",
claim_type=ClaimType.NUMERIC,
                  numeric_values=[{"value": 45.0, "unit":
"percent"}])
    result = v.verify(claim)
    assert result.status == VerificationStatus.CONTRADICTED


def test_sql_verifier_tolerates_small_rounding_difference():
    v = SQLVerifier(make_facts())
    claim = Claim(text="Revenue grew about 12.1%",
claim_type=ClaimType.NUMERIC,
                  numeric_values=[{"value": 12.1, "unit":
"percent"}])
    result = v.verify(claim, tolerance_pct=2.0)
    assert result.status == VerificationStatus.VERIFIED


def test_rules_verifier_confirms_known_entity():
    v = RulesVerifier({"Noida", "TrustGraph"})
    claim = Claim(text="HQ is in Noida", claim_type=ClaimType.ENTITY,
entities=["Noida"])
    result = v.verify(claim)
    assert result.status == VerificationStatus.VERIFIED


def test_rules_verifier_flags_unknown_entity():
    v = RulesVerifier({"Noida"})
    claim = Claim(text="HQ is in Singapore",
claim_type=ClaimType.ENTITY, entities=["Singapore"])
    result = v.verify(claim)
    assert result.status == VerificationStatus.UNSUPPORTED


def test_router_dispatches_numeric_to_sql():
    router = VerificationRouter(SQLVerifier(make_facts()),
RulesVerifier({"Noida"}), LLMVerifier())
```

```
    claim = Claim(text="Revenue grew 12%",
claim_type=ClaimType.NUMERIC,
                  numeric_values=[{"value": 12.0, "unit":
"percent"}])
    result = router.route_and_verify(claim, [])
    assert result.verification_method == "sql"


def test_router_dispatches_entity_to_rules():
    router = VerificationRouter(SQLVerifier(make_facts()),
RulesVerifier({"Noida"}), LLMVerifier())
    claim = Claim(text="HQ in Noida", claim_type=ClaimType.ENTITY,
entities=["Noida"])
    result = router.route_and_verify(claim, [])
    assert result.verification_method == "rules_engine"


def test_router_dispatches_narrative_to_llm():
    router = VerificationRouter(SQLVerifier(make_facts()),
RulesVerifier({"Noida"}), LLMVerifier())
    claim = Claim(text="Customers were happy",
claim_type=ClaimType.NARRATIVE)
    result = router.route_and_verify(claim, ["Customers were happy
with the redesign."])
    assert result.verification_method == "llm"
```

**tests/test_trust_scorer.py**

```
"""Unit tests for the trust scorer. Run: pytest
tests/test_trust_scorer.py -v"""
import sys, os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"verifiers"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"trust_scorer"))
from verifiers import VerificationResult, VerificationStatus
from trust_scorer import compute_trust_score


def test_all_verified_gives_high_score():
    results = [
        VerificationResult("c1", VerificationStatus.VERIFIED, "sql",
0.99),
        VerificationResult("c2", VerificationStatus.VERIFIED,
"rules_engine", 0.9),
    ]
    score = compute_trust_score(results)
    assert score.overall_score > 0.85
    assert score.dimensions.claim_verification_rate == 1.0
```

```python
def test_all_contradicted_gives_low_score():
    results = [
        VerificationResult("c1", VerificationStatus.CONTRADICTED,
"sql", 0.95),
        VerificationResult("c2", VerificationStatus.CONTRADICTED,
"sql", 0.95),
    ]
    score = compute_trust_score(results)
    assert score.dimensions.conflict_score == 0.0
    assert score.dimensions.claim_verification_rate == 0.0


def test_mixed_results_land_between_extremes():
    results = [
        VerificationResult("c1", VerificationStatus.VERIFIED, "sql",
0.99),
        VerificationResult("c2", VerificationStatus.CONTRADICTED,
"sql", 0.95),
    ]
    score = compute_trust_score(results)
    assert 0.0 < score.overall_score < 1.0


def test_deterministic_verification_rate_counts_only_sql_and_rules():
    results = [
        VerificationResult("c1", VerificationStatus.VERIFIED, "sql",
0.99),
        VerificationResult("c2", VerificationStatus.VERIFIED, "llm",
0.7),
    ]
    score = compute_trust_score(results)
    assert score.dimensions.deterministic_verification_rate == 0.5


def test_empty_results_does_not_crash():
    score = compute_trust_score([])
    assert 0.0 <= score.overall_score <= 1.0
```

**evaluation/benchmark_dataset.py**

```python
"""
A small, hand-labeled benchmark dataset for TrustGraph-Lite.

Each example gives: a query, a generated answer to check, the document
pool
available for retrieval, and ground truth for every claim in the
answer
(whether it is actually correct given the "real" facts below). This is
intentionally small (12 examples) and covers the adversarial
categories
named in the Experimental Evaluation Spec: numeric mismatch, missing
```

```python
evidence, entity errors, and correct answers (so baselines aren't
trivially punished for having nothing to catch).
"""

from dataclasses import dataclass, field


@dataclass
class BenchmarkExample:
    example_id: str
    query: str
    answer_text: str
    category: str  # correct | numeric_mismatch | missing_evidence |
entity_error | ambiguous
    claim_ground_truth: list[bool] = field(default_factory=list)  #
True = claim is actually correct
    all_claims_correct: bool = True  # used for calibration
(Brier/ECE)


# "Ground truth" facts backing this toy dataset (mirrors the SQL
verifier's facts table)
GROUND_TRUTH_FACTS = {
    "revenue_growth_pct": 12.0,
    "revenue_usd": 4_200_000.0,
    "churn_before_pct": 8.0,
    "churn_after_pct": 5.0,
    "prior_revenue_usd": 3_750_000.0,
    "hq_city": "Noida",
}

DOCUMENT_POOL = [
    ("doc1", "TrustGraph revenue grew 12% year over year to reach $4.2
million in 2025."),
    ("doc2", "The company's headquarters moved to Noida in early
2024."),
    ("doc3", "Customer churn rate decreased from 8% to 5% after the Q3
product launch."),
    ("doc4", "Revenue for the prior fiscal year was $3.75 million,
showing steady growth."),
    ("doc5", "The engineering team is organized into five pods of two
people each."),
]

BENCHMARK = [
    BenchmarkExample(
        "ex01", "What was the revenue growth?",
        "TrustGraph revenue grew 12% year over year to reach $4.2
million in 2025.",
        category="correct", claim_ground_truth=[True, True],
```

```
all_claims_correct=True,
    ),
    BenchmarkExample(
        "ex02", "What was the revenue growth?",
        "TrustGraph revenue grew 45% year over year to reach $9
million in 2025.",
        category="numeric_mismatch", claim_ground_truth=[False,
False], all_claims_correct=False,
    ),
    BenchmarkExample(
        "ex03", "How did churn change?",
        "Customer churn rate decreased from 8% to 5% after the Q3
product launch.",
        category="correct", claim_ground_truth=[True, True],
all_claims_correct=True,
    ),
    BenchmarkExample(
        "ex04", "How did churn change?",
        "Customer churn rate decreased from 8% to 1% after the Q3
product launch.",
        category="numeric_mismatch", claim_ground_truth=[True, False],
all_claims_correct=False,
    ),
    BenchmarkExample(
        "ex05", "Where is the company headquartered?",
        "The company's headquarters moved to Noida in early 2024.",
        category="correct", claim_ground_truth=[True],
all_claims_correct=True,
    ),
    BenchmarkExample(
        "ex06", "Where is the company headquartered?",
        "The company's headquarters moved to Singapore in early
2024.",
        category="entity_error", claim_ground_truth=[False],
all_claims_correct=False,
    ),
    BenchmarkExample(
        "ex07", "What was the prior year revenue?",
        "Revenue for the prior fiscal year was $3.75 million, showing
steady growth.",
        category="correct", claim_ground_truth=[True],
all_claims_correct=True,
    ),
    BenchmarkExample(
        "ex08", "What was the company's profit margin?",
        "The company's profit margin was 22% in 2025.",
        category="missing_evidence", claim_ground_truth=[False],
all_claims_correct=False,
    ),
    BenchmarkExample(
```

```
        "ex09", "How is the engineering team organized?",
        "The engineering team is organized into five pods of two
people each.",
        category="correct", claim_ground_truth=[True],
all_claims_correct=True,
    ),
    BenchmarkExample(
        "ex10", "Are customers happy with the product?",
        "Customers generally reported higher satisfaction after the
redesign.",
        category="ambiguous", claim_ground_truth=[False],
all_claims_correct=False,
        # ambiguous/unverifiable narrative claim with no direct
evidence in the pool
    ),
    BenchmarkExample(
        "ex11", "What was the revenue and churn improvement
together?",
        "Revenue grew 12% to $4.2 million and churn decreased from 8%
to 5%.",
        category="correct", claim_ground_truth=[True, True, True,
True], all_claims_correct=True,
    ),
    BenchmarkExample(
        "ex12", "What was the revenue and churn improvement
together?",
        "Revenue grew 12% to $4.2 million and churn decreased from 8%
to 2%.",
        category="numeric_mismatch", claim_ground_truth=[True, True,
True, False], all_claims_correct=False,
    ),
]
```

**evaluation/baselines.py**

```
"""
Baseline systems, per the Experimental Evaluation Spec §2.

B0 — LLM only: no retrieval, no verification. Every claim is assumed
correct
     (this is what a bare LLM answer effectively does — it has no
mechanism
     to flag its own errors).
B1 — Simple vector RAG: dense-only retrieval, no verification.
B2 — Hybrid RAG: hybrid (dense+sparse) retrieval, no verification.
B3 — Hybrid RAG + reranker: same as B2 in this teaching codebase,
since the
     reranker is a stub (Part C, Chapter 12 shows where a real cross-
encoder
     reranker plugs in). Kept as a distinct row for structural
completeness.
```

```
TrustGraph v0.1 — the full pipeline: hybrid retrieval + claim
extraction +
     verification router (deterministic + LLM verifiers) + trust
scoring.

The key structural point: B0-B3 never verify anything, so their
predicted
"claim correctness" is always True. This is what lets the benchmark
show,
concretely, how much verification actually buys you.
"""

import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"retriever"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"claim_extractor"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"verifiers"))
sys.path.append(os.path.join(os.path.dirname(__file__), "..",
"trust_scorer"))

import pandas as pd
from retriever import Document, HybridRetriever
from claim_extractor import extract_claims
from verifiers import VerificationRouter, SQLVerifier, RulesVerifier,
LLMVerifier, VerificationStatus
from trust_scorer import compute_trust_score


def build_documents(document_pool):
    return [Document(doc_id, doc_id, text) for doc_id, text in
document_pool]


class BaselineNoVerification:
    """Shared logic for B0-B3: retrieve (or don't) then assume every
claim is correct."""

    name: str

    def __init__(self, retriever: HybridRetriever | None):
        self.retriever = retriever

    def run(self, query: str, answer_text: str) -> dict:
        claims = extract_claims(answer_text)
        # No verification happens — every claim is assumed correct
(predicted_correct=True)
```

```
        predicted_correct = [True] * len(claims)
        predicted_prob_all_correct = 1.0  # a bare LLM/RAG system
reports no uncertainty by default
        return {
            "predicted_correct": predicted_correct,
            "predicted_prob_all_correct": predicted_prob_all_correct,
            "num_claims": len(claims),
        }


class B0_LLMOnly(BaselineNoVerification):
    name = "B0 (LLM only)"

    def __init__(self):
        super().__init__(retriever=None)  # no retrieval at all


class B1_VectorOnly(BaselineNoVerification):
    name = "B1 (vector-only RAG)"

    def __init__(self, documents):
        # "Vector-only" — in this teaching codebase we approximate by
using the
        # same hybrid retriever, since a true dense-only path would
need a
        # separate ranking function; the point of B1 vs B2 vs B3 here
is
        # structural (retrieval happens, verification doesn't), not a
claim
        # that our dense-only ranking is state-of-the-art.

super().__init__(retriever=HybridRetriever(build_documents(documents))
)


class B2_HybridRAG(BaselineNoVerification):
    name = "B2 (hybrid RAG)"

    def __init__(self, documents):

super().__init__(retriever=HybridRetriever(build_documents(documents))
)


class B3_HybridRerank(BaselineNoVerification):
    name = "B3 (hybrid + reranker)"

    def __init__(self, documents):
```

```
super().__init__(retriever=HybridRetriever(build_documents(documents))
)


class TrustGraphV01:
    """The full pipeline: retrieval + claim extraction + verification
router + trust scoring."""

    name = "TrustGraph v0.1"

    def __init__(self, documents, facts_df: pd.DataFrame,
known_entities: set[str]):
        self.retriever = HybridRetriever(build_documents(documents))
        self.router = VerificationRouter(
            SQLVerifier(facts_df), RulesVerifier(known_entities),
LLMVerifier()
        )

    def run(self, query: str, answer_text: str) -> dict:
        evidence = self.retriever.retrieve(query, top_k=3)
        evidence_texts = [e.text for e in evidence]

        claims = extract_claims(answer_text)
        results = [self.router.route_and_verify(c, evidence_texts) for
c in claims]
        trust_score = compute_trust_score(results)

        predicted_correct = [r.status == VerificationStatus.VERIFIED
for r in results]
        return {
            "predicted_correct": predicted_correct,
            "predicted_prob_all_correct": trust_score.overall_score,
            "num_claims": len(claims),
            "trust_score": trust_score.overall_score,
        }
```

**evaluation/benchmark_runner.py**

```
"""
TrustGraph-Lite — Benchmark Runner
==================================
Runs B0-B3 and the full TrustGraph v0.1 pipeline against the labeled
benchmark dataset, computes metrics, and produces a comparison report.

Run it:
    python3 benchmark_runner.py

This is the smallest possible real implementation of the Experimental
Evaluation Spec's "one command runs the full benchmark and produces a
reproducible report" success criterion (Milestone 1).
"""
```

```python
import time
import json
from datetime import datetime, timezone

import pandas as pd

from benchmark_dataset import BENCHMARK, DOCUMENT_POOL,
GROUND_TRUTH_FACTS
from baselines import B0_LLMOnly, B1_VectorOnly, B2_HybridRAG,
B3_HybridRerank, TrustGraphV01


def claim_f1_for_hallucination_detection(all_predicted_correct:
list[bool], all_ground_truth_correct: list[bool]) -> dict:
    """
    The metric that matters for this project's value proposition: can
the
    system detect a claim that is actually WRONG?
    Positive class = claim is actually incorrect (a hallucination).
    Predicted positive = system flagged it as NOT correct.
    """
    tp = fp = fn = tn = 0
    for predicted_correct, actually_correct in
zip(all_predicted_correct, all_ground_truth_correct):
        predicted_wrong = not predicted_correct
        actually_wrong = not actually_correct
        if predicted_wrong and actually_wrong:
            tp += 1
        elif predicted_wrong and not actually_wrong:
            fp += 1
        elif not predicted_wrong and actually_wrong:
            fn += 1
        else:
            tn += 1

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision +
recall) > 0 else 0.0
    return {"precision": round(precision, 3), "recall": round(recall,
3), "f1": round(f1, 3),
            "tp": tp, "fp": fp, "fn": fn, "tn": tn}


def brier_score(predicted_probs: list[float], actual_outcomes:
list[bool]) -> float:
    """Lower is better. Measures calibration: does a predicted 90%
actually mean ~90% correct?"""
    n = len(predicted_probs)
```

```
    if n == 0:
        return float("nan")
    return round(sum((p - float(o)) ** 2 for p, o in
zip(predicted_probs, actual_outcomes)) / n, 4)


def run_system(system, use_retrieval_signature: bool) -> dict:
    all_predicted_correct = []
    all_ground_truth_correct = []
    predicted_probs = []
    actual_all_correct = []
    latencies = []

    for example in BENCHMARK:
        start = time.perf_counter()
        result = system.run(example.query, example.answer_text)
        latencies.append(time.perf_counter() - start)

        predicted = result["predicted_correct"]
        # Align lengths defensively — extractor may split claims
differently
        # than the hand-labeled ground truth in edge cases.
        n = min(len(predicted), len(example.claim_ground_truth))
        all_predicted_correct.extend(predicted[:n])

all_ground_truth_correct.extend(example.claim_ground_truth[:n])

        predicted_probs.append(result["predicted_prob_all_correct"])
        actual_all_correct.append(example.all_claims_correct)

    metrics =
claim_f1_for_hallucination_detection(all_predicted_correct,
all_ground_truth_correct)
    metrics["brier_score"] = brier_score(predicted_probs,
actual_all_correct)
    metrics["latency_p50_ms"] = round(sorted(latencies)[len(latencies)
// 2] * 1000, 2)
    metrics["latency_p95_ms"] = round(sorted(latencies)
[int(len(latencies) * 0.95)] * 1000, 2)
    return metrics


def main():
    facts_df = pd.DataFrame([
        {"metric": "revenue_growth_pct", "unit": "percent", "value":
GROUND_TRUTH_FACTS["revenue_growth_pct"]},
        {"metric": "revenue_usd", "unit": "usd", "value":
GROUND_TRUTH_FACTS["revenue_usd"]},
        {"metric": "churn_before_pct", "unit": "percent", "value":
GROUND_TRUTH_FACTS["churn_before_pct"]},
```

```
        {"metric": "churn_after_pct", "unit": "percent", "value":
GROUND_TRUTH_FACTS["churn_after_pct"]},
        {"metric": "prior_revenue_usd", "unit": "usd", "value":
GROUND_TRUTH_FACTS["prior_revenue_usd"]},
    ])
    known_entities = {"Noida", "TrustGraph"}

    systems = {
        "B0 (LLM only)": B0_LLMOnly(),
        "B1 (vector-only RAG)": B1_VectorOnly(DOCUMENT_POOL),
        "B2 (hybrid RAG)": B2_HybridRAG(DOCUMENT_POOL),
        "B3 (hybrid + reranker)": B3_HybridRerank(DOCUMENT_POOL),
        "TrustGraph v0.1": TrustGraphV01(DOCUMENT_POOL, facts_df,
known_entities),
    }

    report_rows = []
    for name, system in systems.items():
        metrics = run_system(system, use_retrieval_signature=True)
        metrics["system"] = name
        report_rows.append(metrics)

    manifest = {
        "run_id": f"eval-{datetime.now(timezone.utc).strftime('%Y-%m-
%d-%H%M%S')}",
        "dataset_version": "golden-v1-teaching",
        "dataset_size": len(BENCHMARK),
        "embedding_model": "tfidf-svd-v1-teaching",
        "date": datetime.now(timezone.utc).isoformat(),
    }

    print(f"\n=== TrustGraph-Lite Benchmark Report
({manifest['run_id']}) ===\n")
    print(f"Dataset: {manifest['dataset_version']}
(n={manifest['dataset_size']} examples)\n")
    header = f"{'System':24s} {'F1':>6s} {'Prec':>6s} {'Recall':>7s}
{'Brier':>7s} {'P50(ms)':>9s} {'P95(ms)':>9s}"
    print(header)
    print("-" * len(header))
    for row in report_rows:
        print(f"{row['system']:24s} {row['f1']:>6.3f}
{row['precision']:>6.3f} {row['recall']:>7.3f} "
              f"{row['brier_score']:>7.4f}
{row['latency_p50_ms']:>9.2f} {row['latency_p95_ms']:>9.2f}")

    print("\nMetric definition: F1/Precision/Recall measure
HALLUCINATION DETECTION")
    print("(positive class = a claim that is actually wrong). Higher
is better.")
    print("Brier score measures calibration of the system's overall
```

```
confidence (lower is better).")
    print("\nRaw counts (tp=caught a real error, fp=false alarm,
fn=missed a real error, tn=correctly trusted):")
    for row in report_rows:
        print(f"  {row['system']:24s} tp={row['tp']} fp={row['fp']}
fn={row['fn']} tn={row['tn']}")

    with open("benchmark_report_latest.json", "w") as f:
        json.dump({"manifest": manifest, "results": report_rows}, f,
indent=2)
    print("\nFull report written to benchmark_report_latest.json")


if __name__ == "__main__":
    main()
```

**Dockerfile**

```
# TrustGraph-Lite — Dockerfile
# Not executed in this book's sandbox (no Docker daemon available
here) —
# this is a standard, correct configuration; validate it in your own
# environment with: docker build -t trustgraph-lite .

FROM python:3.12-slim

WORKDIR /app

# Install dependencies first (separate layer so code changes don't
bust the
# dependency cache — one of the most common Docker build-time
optimizations)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy each service
COPY retriever/ ./retriever/
COPY claim_extractor/ ./claim_extractor/
COPY verifiers/ ./verifiers/
COPY trust_scorer/ ./trust_scorer/
COPY api/ ./api/

WORKDIR /app/api

EXPOSE 8000

# Basic healthcheck hitting the /health endpoint built in Chapter 10
HEALTHCHECK --interval=30s --timeout=3s \
  CMD python3 -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')" || exit 1
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

**docker-compose.yml**

```
# TrustGraph-Lite — docker-compose.yml
# Runs the teaching API alongside the production-shaped data stack
from
# Part C (Postgres+pgvector, Redis), so you can see how the teaching
# in-memory version and the real deployment target relate.
# Not executed in this book's sandbox — validate with `docker-compose
up`
# in your own environment.

version: "3.9"

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - LOG_LEVEL=info
    depends_on:
      - postgres
      - redis
    restart: unless-stopped

  postgres:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: trustgraph
      POSTGRES_PASSWORD: trustgraph
      POSTGRES_DB: trustgraph
    ports:
      - "5432:5432"
    volumes:
      - trustgraph_pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U trustgraph"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
```

```
      timeout: 3s
      retries: 5

volumes:
  trustgraph_pg_data:
```

**requirements.txt**

```
# Versions confirmed working in this book's actual test environment.
# Pin exact versions in production; these are the ranges every code
sample
# in this book was actually run against.
fastapi>=0.115,<0.140
uvicorn>=0.30,<0.51
pydantic>=2.9,<2.14
rank-bm25==0.2.2
numpy>=1.26,<2.5
scikit-learn>=1.5,<1.9
pandas>=2.2,<3.1
```

**admin_dashboard.html**

```
<!DOCTYPE html>
<!--
TrustGraph-Lite — Minimal Admin Dashboard
==========================================
A single-file HTML/JS dashboard hitting the /query endpoint from
Chapter 10.
This is the smallest possible real version of the Admin Console's
Claim
Explorer screen — a form to submit a query+answer, and a rendered
claim-by-claim trust breakdown.

Run the API first (Chapter 10), then open this file directly in a
browser.
No build step, no framework — plain HTML/CSS/JS, so a fresher can read
every line of what's happening.
-->
<html lang="en">
<head>
<meta charset="UTF-8">
<title>TrustGraph-Lite — Claim Explorer</title>
<style>
  body { font-family: -apple-system, sans-serif; max-width: 800px;
margin: 40px auto; padding: 0 20px; color: #1a1a1a; }
  h1 { font-size: 1.4rem; }
  textarea, input { width: 100%; padding: 8px; margin: 6px 0 14px;
box-sizing: border-box; font-family: inherit; }
  button { padding: 10px 20px; background: #2563eb; color: white;
border: none; border-radius: 4px; cursor: pointer; }
  button:hover { background: #1d4ed8; }
  .trust-score { font-size: 2rem; font-weight: bold; margin: 16px 0
```

```
4px; }
  .dimensions { display: grid; grid-template-columns: 1fr 1fr; gap:
6px 20px; font-size: 0.9rem; margin-bottom: 20px; }
  .claim { border-left: 4px solid #ccc; padding: 10px 14px; margin:
10px 0; background: #f8f9fa; }
  .claim.verified { border-color: #16a34a; }
  .claim.contradicted { border-color: #dc2626; }
  .claim.unsupported, .claim.insufficient_evidence { border-color:
#d97706; }
  .status-badge { display: inline-block; padding: 2px 8px; border-
radius: 10px; font-size: 0.75rem; font-weight: 600; text-transform:
uppercase; }
  .verified .status-badge { background: #dcfce7; color: #166534; }
  .contradicted .status-badge { background: #fee2e2; color: #991b1b; }
  .unsupported .status-badge, .insufficient_evidence .status-badge {
background: #fef3c7; color: #92400e; }
  .method { color: #666; font-size: 0.85rem; }
  #error { color: #dc2626; margin-top: 10px; }
</style>
</head>
<body>

<h1>TrustGraph-Lite — Claim Explorer</h1>
<p>Submit a query and a generated answer to see the claim-by-claim
trust breakdown.</p>

<label>Query</label>
<input id="query" value="What was the revenue growth?">

<label>Answer to check</label>
<textarea id="answer" rows="3">TrustGraph revenue grew 45% year over
year to reach $9 million in 2025.</textarea>

<button onclick="runQuery()">Check This Answer</button>
<div id="error"></div>
<div id="results"></div>

<script>
async function runQuery() {
  const query = document.getElementById('query').value;
  const answer = document.getElementById('answer').value;
  const errorDiv = document.getElementById('error');
  const resultsDiv = document.getElementById('results');
  errorDiv.textContent = '';
  resultsDiv.innerHTML = 'Checking...';

  try {
    const response = await fetch('http://localhost:8000/query', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
```

```
      body: JSON.stringify({ query, answer })
    });
    if (!response.ok) throw new Error(`API returned ${response.status}`);
    const data = await response.json();
    render(data);
  } catch (err) {
    errorDiv.textContent = `Error: ${err.message}. Is the API running on localhost:8000? (See Chapter 10.)`;
    resultsDiv.innerHTML = '';
  }
}

function render(data) {
  const resultsDiv = document.getElementById('results');
  const scorePercent = Math.round(data.overall_trust_score * 100);
  const scoreColor = scorePercent >= 80 ? '#16a34a' : scorePercent >= 50 ? '#d97706' : '#dc2626';

  let html = `<div class="trust-score" style="color:${scoreColor}">Trust Score: ${scorePercent}%</div>`;
  html += '<div class="dimensions">';
  for (const [key, value] of Object.entries(data.trust_dimensions)) {
    html += `<div><strong>${key.replace(/_/g, ' ')}:</strong> ${Math.round(value * 100)}%</div>`;
  }
  html += '</div>';

  html += '<h3>Claims</h3>';
  for (const claim of data.claims) {
    html += `
      <div class="claim ${claim.status}">
        <span class="status-badge">${claim.status.replace('_', ' ')}</span>
        <span class="method">via ${claim.verification_method}</span>
        <p>${claim.text}</p>
        ${claim.reason ? `<p style="color:#666;font-size:0.85rem">${claim.reason}</p>` : ''}
      </div>`;
  }

  resultsDiv.innerHTML = html;
}
</script>
</body>
</html>
```

---

## Chapter 54: Appendix — Complete OpenAPI Specification

Chapter 30 (API Specification) summarized this as an endpoint table. Reproduced here in full — every request/response schema, every field, every endpoint — as the authoritative machine-readable contract. This is the exact file validated as syntactically correct YAML earlier in this book's development (`python3 -c "import yaml; yaml.safe_load(open('trustgraph-openapi.yaml'))"` parses cleanly), and it's the same file the real production team would load into any OpenAPI tooling (Swagger UI, client code generators, contract testing tools) as-is.

```yaml
openapi: 3.0.3
info:
  title: TrustGraph API
  version: "1.0"
  description: >
    TrustGraph v1 vertical slice API — verifiable RAG pipeline.
    Contracts are frozen per ADR-001 and the Data Contract
Specification v1.0.
    All objects referenced here are defined in full in the Data
Contract Specification.

servers:
  - url: https://api.trustgraph.ai/v1
    description: Production
  - url: http://localhost:8080/v1
    description: Local development

tags:
  - name: Query
    description: Composed end-to-end pipeline (retrieval →
verification → trust score)
  - name: Retrieval
  - name: Claims
  - name: Verification
  - name: Trust
  - name: Admin

paths:

  /query:
    post:
      tags: [Query]
      summary: Run the full verifiable RAG pipeline for a user query
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
```

```yaml
            schema:
              $ref: '#/components/schemas/QueryRequest'
      responses:
        '200':
          description: Full response with claims, verification
results, and trust score
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ResponseMetadata'
        '400':
          $ref: '#/components/responses/BadRequest'
        '401':
          $ref: '#/components/responses/Unauthorized'
        '500':
          $ref: '#/components/responses/InternalError'

  /retrieve:
    post:
      tags: [Retrieval]
      summary: Retrieve evidence for a query context
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/RetrieveRequest'
      responses:
        '200':
          description: Retrieved and (optionally) reranked evidence
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/RetrieveResponse'
        '400':
          $ref: '#/components/responses/BadRequest'

  /claims/extract:
    post:
      tags: [Claims]
      summary: Extract atomic claims from a generated response
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
```

```yaml
              $ref: '#/components/schemas/ClaimExtractionRequest'
      responses:
        '200':
          description: Extracted claims
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ClaimExtractionResponse'

  /verify:
    post:
      tags: [Verification]
      summary: Verify a single claim against evidence
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/VerificationRequest'
      responses:
        '200':
          description: Verification result
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/VerificationResponse'

  /trust-score:
    post:
      tags: [Trust]
      summary: Compute a calibrated trust score from verification
results
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/TrustScoreRequest'
      responses:
        '200':
          description: Computed trust score
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/TrustScoreResponse'
```

```yaml
  /admin/pipeline-events/{trace_id}:
    get:
      tags: [Admin]
      summary: Retrieve the full pipeline event trace for a request
(Pipeline Explorer)
      security:
        - bearerAuth: []
      parameters:
        - name: trace_id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        '200':
          description: Ordered list of pipeline events for this trace
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/PipelineEvent'
        '404':
          description: Trace not found

  /admin/responses/{response_id}/claims:
    get:
      tags: [Admin]
      summary: Retrieve claims, evidence, and verification detail for
a response (Claim Explorer)
      security:
        - bearerAuth: []
      parameters:
        - name: response_id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        '200':
          description: Full claim/verification breakdown
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ResponseMetadata'
        '404':
          description: Response not found
```

```yaml
  /admin/metrics/summary:
    get:
      tags: [Admin]
      summary: Executive dashboard summary metrics
      security:
        - bearerAuth: []
      parameters:
        - name: since
          in: query
          schema:
            type: string
            format: date-time
      responses:
        '200':
          description: Aggregate KPIs (avg trust score, hallucination
rate, latency, cost per request)
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ExecutiveSummary'

components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

  responses:
    BadRequest:
      description: Invalid request payload
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Error'
    Unauthorized:
      description: Missing or invalid authentication
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Error'
    InternalError:
      description: Unhandled server error
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Error'

  schemas:
```

```yaml
    Error:
      type: object
      properties:
        failure_code:
          type: string
          enum: [RETRIEVAL_MISS, EXTRACTION_ERROR,
CONFLICTING_EVIDENCE, STALE_DOCUMENT,
                 NUMERIC_MISMATCH, NO_SUPPORT, LOW_AUTHORITY,
AMBIGUOUS_CLAIM, TIMEOUT, VALIDATION_ERROR]
        message:
          type: string
        trace_id:
          type: string
          format: uuid

    EnvelopeCommon:
      type: object
      properties:
        request_id:
          type: string
          format: uuid
        trace_id:
          type: string
          format: uuid
        tenant_id:
          type: string
        timestamp:
          type: string
          format: date-time
        schema_version:
          type: string
          example: "1.0"

    QueryContext:
      type: object
      required: [query_id, raw_query, tenant_id]
      properties:
        query_id:
          type: string
          format: uuid
        raw_query:
          type: string
        rewritten_query:
          type: string
          nullable: true
        user_id:
          type: string
        tenant_id:
          type: string
        workspace_id:
```

```yaml
      type: string
    authorization_scopes:
      type: array
      items:
        type: string
    retrieval_strategy:
      type: string
      enum: [dense, sparse, hybrid, sql, graph]
    filters:
      type: object
      additionalProperties: true
    trace_id:
      type: string
      format: uuid

QueryRequest:
  allOf:
    - $ref: '#/components/schemas/EnvelopeCommon'
    - type: object
      required: [query_context]
      properties:
        query_context:
          $ref: '#/components/schemas/QueryContext'
        top_k:
          type: integer
          default: 10
        rerank:
          type: boolean
          default: true

Evidence:
  type: object
  properties:
    evidence_id:
      type: string
      format: uuid
    document_id:
      type: string
    chunk_id:
      type: string
    text:
      type: string
    page:
      type: integer
      nullable: true
    paragraph:
      type: integer
      nullable: true
    offset:
      type: array
```

```yaml
      items:
        type: integer
      minItems: 2
      maxItems: 2
    bbox:
      type: array
      items:
        type: number
      nullable: true
    source_hash:
      type: string
    authority_score:
      type: number
      minimum: 0
      maximum: 1
    freshness_timestamp:
      type: string
      format: date-time
    retrieval_score:
      type: number
    retrieval_method:
      type: string
      enum: [dense, sparse, hybrid, reranked]

RetrieveRequest:
  allOf:
    - $ref: '#/components/schemas/EnvelopeCommon'
    - type: object
      required: [query_context, top_k]
      properties:
        query_context:
          $ref: '#/components/schemas/QueryContext'
        top_k:
          type: integer
        rerank:
          type: boolean
        filters:
          type: object
          additionalProperties: true
        authorization_context:
          type: object
          additionalProperties: true

RetrieveResponse:
  type: object
  properties:
    evidence:
      type: array
      items:
        $ref: '#/components/schemas/Evidence'
```

```yaml
        retrieval_recall_estimate:
          type: number
          nullable: true
        pipeline_event:
          $ref: '#/components/schemas/PipelineEvent'

    Claim:
      type: object
      properties:
        claim_id:
          type: string
          format: uuid
        response_id:
          type: string
          format: uuid
        text:
          type: string
        claim_type:
          type: string
          enum: [numeric, entity, definition, policy, narrative,
temporal]
        entities:
          type: array
          items:
            type: string
        relations:
          type: array
          items:
            type: object
            properties:
              subject:
                type: string
              predicate:
                type: string
              object:
                type: string
        numeric_values:
          type: array
          nullable: true
          items:
            type: object
            properties:
              value:
                type: number
              unit:
                type: string
              operation:
                type: string
        temporal_context:
          type: object
```

```
          nullable: true
        extraction_method:
          type: string
          enum: [llm, ner, dependency_parser, hybrid]
        extraction_confidence:
          type: number

    ClaimExtractionRequest:
      allOf:
        - $ref: '#/components/schemas/EnvelopeCommon'
        - type: object
          required: [response_text, response_id]
          properties:
            response_text:
              type: string
            response_id:
              type: string
              format: uuid
            extraction_methods:
              type: array
              items:
                type: string
                enum: [llm, ner, dependency_parser]

    ClaimExtractionResponse:
      type: object
      properties:
        claims:
          type: array
          items:
            $ref: '#/components/schemas/Claim'
        pipeline_event:
          $ref: '#/components/schemas/PipelineEvent'

    VerificationResult:
      type: object
      properties:
        claim_id:
          type: string
          format: uuid
        status:
          type: string
          enum: [verified, contradicted, unsupported,
insufficient_evidence]
        verification_method:
          type: string
          enum: [sql, pandas, knowledge_graph, rules_engine, llm,
human]
        supporting_evidence:
          type: array
```

```yaml
      items:
        type: string
        format: uuid
    contradicting_evidence:
      type: array
      items:
        type: string
        format: uuid
    confidence:
      type: number
    failure_code:
      type: string
      nullable: true
    reason:
      type: string
    verification_time_ms:
      type: integer

VerificationRequest:
  allOf:
    - $ref: '#/components/schemas/EnvelopeCommon'
    - type: object
      required: [claim, evidence]
      properties:
        claim:
          $ref: '#/components/schemas/Claim'
        evidence:
          type: array
          items:
            $ref: '#/components/schemas/Evidence'
        verification_strategy_override:
          type: string
          nullable: true

VerificationResponse:
  type: object
  properties:
    result:
      $ref: '#/components/schemas/VerificationResult'
    pipeline_event:
      $ref: '#/components/schemas/PipelineEvent'

TrustScore:
  type: object
  properties:
    response_id:
      type: string
      format: uuid
    overall_score:
      type: number
```

```yaml
    dimensions:
      type: object
      properties:
        retrieval_quality:
          type: number
        evidence_coverage:
          type: number
        claim_verification_rate:
          type: number
        freshness:
          type: number
        conflict_score:
          type: number
        source_authority:
          type: number
        deterministic_verification_rate:
          type: number
    calibration_version:
      type: string
    computed_at:
      type: string
      format: date-time

TrustScoreRequest:
  allOf:
    - $ref: '#/components/schemas/EnvelopeCommon'
    - type: object
      required: [response_id, verification_results]
      properties:
        response_id:
          type: string
          format: uuid
        verification_results:
          type: array
          items:
            $ref: '#/components/schemas/VerificationResult'
        evidence:
          type: array
          items:
            $ref: '#/components/schemas/Evidence'

TrustScoreResponse:
  type: object
  properties:
    trust_score:
      $ref: '#/components/schemas/TrustScore'
    pipeline_event:
      $ref: '#/components/schemas/PipelineEvent'

PipelineEvent:
```

```yaml
      type: object
      properties:
        event_id:
          type: string
          format: uuid
        trace_id:
          type: string
          format: uuid
        stage:
          type: string
          enum: [retrieval, ranking, claim_extraction, verification,
trust_scoring, generation, composition]
        status:
          type: string
          enum: [started, completed, failed, retried]
        latency_ms:
          type: integer
        token_usage:
          type: object
          nullable: true
          properties:
            input_tokens:
              type: integer
            output_tokens:
              type: integer
        cost_usd:
          type: number
          nullable: true
        model_version:
          type: string
          nullable: true
        prompt_version:
          type: string
          nullable: true
        embedding_model_version:
          type: string
          nullable: true
        error:
          type: object
          nullable: true
          properties:
            failure_code:
              type: string
            message:
              type: string
        emitted_at:
          type: string
          format: date-time

    ResponseMetadata:
```

```yaml
    type: object
    properties:
      response_id:
        type: string
        format: uuid
      request_id:
        type: string
        format: uuid
      answer_text:
        type: string
      claims:
        type: array
        items:
          $ref: '#/components/schemas/Claim'
      verification_results:
        type: array
        items:
          $ref: '#/components/schemas/VerificationResult'
      trust_score:
        $ref: '#/components/schemas/TrustScore'
      evidence_used:
        type: array
        items:
          $ref: '#/components/schemas/Evidence'
      pipeline_events:
        type: array
        items:
          $ref: '#/components/schemas/PipelineEvent'
      model_versions:
        type: object
        additionalProperties:
          type: string
      generated_at:
        type: string
        format: date-time

  ExecutiveSummary:
    type: object
    properties:
      requests_per_minute:
        type: number
      avg_trust_score:
        type: number
      hallucination_rate:
        type: number
      avg_latency_ms:
        type: number
      cache_hit_rate:
        type: number
      cost_per_request_usd:
```

```yaml
  type: number
verification_success_rate:
  type: number
top_failure_reasons:
  type: array
  items:
    type: object
    properties:
      failure_code:
        type: string
      count:
        type: integer
```